**You asked:**

Has dicho

Quiero compilar pacman.

 pacman

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

**Gemini Replied:**

## Gemini ha dicho

Vaya, te has quedado en una situación de "huevo o gallina" bastante incómoda: tu gestor de paquetes no funciona porque una de sus dependencias fundamentales (**libassuan**) se ha actualizado (o degradado) y ahora hay un conflicto de versiones con **libgpgme**.

Como `pacman` está roto, no podemos usarlo para arreglarse a sí mismo de la forma habitual. Aquí tienes los pasos para rescatar tu sistema:

## 1. El problema: Desajuste de librerías

El error indica que `libgpgme` busca una versión específica de `libassuan` que no encuentra en el archivo físico. Esto suele pasar tras una actualización parcial interrumpida o si se han movido archivos manualmente en `/mnt`.

---

## 2. Solución A: Usar el paquete estático (Si tienes acceso)

Si tienes otro sistema operativo funcional o un USB de instalación de Arch (Live USB), lo más sencillo es hacer un **chroot** o usar el binario estático de pacman.

1. **Arranca desde un USB de Arch Linux.**
2. Monta tu partición raíz en `/mnt`.
3. Usa `pacstrap` o el pacman del USB apuntando a tu disco:

   Bash

```
pacman --root /mnt -Syu libassuan gpgme
```

*Esto forzará la actualización de las librerías conflictivas usando el binario funcional del USB.*

---

## 3. Solución B: Reparación manual (Sin USB)

Si no puedes usar un USB, tenemos que "engañar" al sistema o reinstalar la librería manualmente.

## Paso 1: Comprobar qué versiones tienes

Mira qué archivos hay en tu carpeta de librerías:

Bash

```
ls -lh /usr/lib/libassuan.so*
```

Es probable que tengas `libassuan.so.0` o algo similar, pero el sistema busque la `.9`.

## Paso 2: Descarga manual del paquete

Tendrás que descargar el paquete `.pkg.tar.zst` de `libassuan` y `gpgme` desde un espejo (mirror) oficial usando `curl` o `wget`:

- Busca en archlinux.org/packages.
- Descárgalos:
  `curl -O https://mirror.rnet.missouri.edu/archlinux/core/os/x86_64/libassuan-3.0.1-1-x86_64.pkg.tar.zst`
  (verifica la versión actual).

## Paso 3: Extraer manualmente

Como `pacman` no funciona, extrae el contenido directamente en la raíz (con mucho cuidado):

Bash

```
sudo tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst -C /
```

*Nota: Esto sobrescribirá los archivos de la librería y debería permitir que `pacman` vuelva a ejecutarse.*

---

## 4. ¿Realmente quieres compilarlo?

Si tu intención original era **compilar `pacman` desde el código fuente** (porque el binario está corrupto), necesitas las herramientas de desarrollo, pero sin `pacman` no puedes instalarlas fácilmente.

Si ya tienes `base-devel`, el proceso sería:

1. Clonar el repo: `git clone https://gitlab.archlinux.org/pacman/pacman.git`
2. Instalar dependencias de compilación (meson, ninja).
3. `meson setup build` y `ninja -C build`.

**Mi recomendación:** No intentes compilarlo todavía. Primero arregla el enlace de `libassuan` usando el **Paso 2**, ya que sin una base de librerías sólida, la compilación también fallará.

¿Tienes acceso a un USB de instalación de Arch ahora mismo para intentar el rescate de forma segura?

> **You asked:**

Has dicho

 root /arch_repair 

 pacman

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

 root /arch_repair 

 pacman --root /mnt -Syu libassuan gpgme

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

 root /arch_repair 

 curl -O https://mirror.rnet.missouri.edu/archlinux/core/os/x86_64/libassuan-3.0.1-1-x86_64.pkg.tar.zst

```
 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   196  100   196    0     0    188      0  0:00:01  0:00:01 --:--:--   188
```

 root /arch_repair 

 sudo tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst -C /

sudo: unable to allocate pty: No such device

 root /arch_repair 

 sudo tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst

sudo: unable to allocate pty: No such device

 root /arch_repair 

 tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst

xz: (stdin): File format not recognized

tar: Child returned status 1

tar: Error is not recoverable: exiting now

**Gemini Replied:**

## Gemini ha dicho

¡Vaya racha! Tienes un "combo" de problemas: el archivo que descargaste no es el paquete real, `sudo` está fallando por falta de terminal (pty) y `tar` no reconoce el formato porque el archivo está corrupto.

Aquí está el desglose de por qué falló y cómo solucionarlo de verdad:

## 1. El archivo descargado es un error 404

Si te fijas en el tamaño del archivo que descargó `curl` (196 bytes), es evidente que no es el paquete de la librería, sino una página HTML de error del servidor. Por eso `tar` dice que el formato no es reconocido.

## 2. El problema de `sudo`

El error `unable to allocate pty` suele ocurrir cuando estás en un entorno limitado (como un `chroot` mal configurado) donde el sistema de archivos `/dev/pts` no está montado. Como ya eres **root** (según tu prompt), **no necesitas usar `sudo`**.

---

## La solución paso a paso (desde tu entorno actual)

Vamos a descargar el paquete real y extraerlo manualmente. Como `tar` por sí solo a veces se lía con la compresión `.zst`, usaremos `zstd` (si está disponible) o un enlace directo corregido.

## Paso 1: Descargar el paquete real

Usa la opción `-L` en curl para seguir redirecciones y asegúrate de que la URL es válida (la versión puede haber cambiado, intenta esta que es un enlace más genérico o busca la actual):

Bash

```
# Probemos con un mirror robusto y el comando corregido
curl -LO https://archive.archlinux.org/packages/l/libassuan/libassuan-3.0.1-1-x86_64.pkg.tar.zst
```

*(Verifica que el tamaño de descarga sea de varios KB, no 196 bytes).*

## Paso 2: Extraer el paquete

Como el archivo termina en `.zst`, necesitas el descompresor `zstd`. Si no lo tienes, intentaremos usar `tar` directamente pero con los flags correctos.

**Si tienes `zstd`:**

Bash

```
zstd -d libassuan-3.0.1-1-x86_64.pkg.tar.zst
tar -xvf libassuan-3.0.1-1-x86_64.pkg.tar -C /mnt/
```

**Si NO tienes `zstd` (usando tar moderno):**

Bash

```
tar --use-compress-program=zstd -xvf libassuan-3.0.1-1-x86_64.pkg.tar.zst -C /mnt/
```

> ***Nota Crítica:*** *Veo que en tu prompt estás en `/arch_repair` y que el error original mencionaba `/mnt/usr/lib/...`. Asegúrate de extraer el contenido en la ruta donde está montado tu sistema dañado (probablemente `/mnt`).*

---

## 3. El Plan de Emergencia (Si nada funciona)

Si no tienes `zstd` y `tar` falla, lo más rápido para recuperar `pacman` y que deje de quejarse de la versión es hacer un **enlace simbólico temporal** (sucio, pero efectivo para salir del paso):

1. Mira qué versión tienes realmente en `/mnt/usr/lib/`: `ls /mnt/usr/lib/libassuan.so*`
2. Si tienes la versión `.so.8` pero te pide la `.so.9`, crea un link:
   `ln -s /mnt/usr/lib/libassuan.so.8 /mnt/usr/lib/libassuan.so.9`
3. Intenta ejecutar `pacman --root /mnt -Syu libassuan`.

Una vez que `pacman` funcione, **borra ese link manual** y deja que el gestor de paquetes instale la versión correcta.

¿Qué archivos te salen al hacer el `ls /mnt/usr/lib/libassuan.so*`?

> **You asked:**

Has dicho

 root /arch_repair 

 pacman

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

 root /arch_repair 

 pacman --root /mnt -Syu libassuan gpgme

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

 root /arch_repair 

 curl -O https://mirror.rnet.missouri.edu/archlinux/core/os/x86_64/libassuan-3.0.1-1-x86_64.pkg.tar.zst

```
 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   196  100   196    0     0    188      0  0:00:01  0:00:01 --:--:--   188
```

 root /arch_repair 

 sudo tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst -C /

sudo: unable to allocate pty: No such device

 root /arch_repair 

 sudo tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst

sudo: unable to allocate pty: No such device

 root /arch_repair 

 tar -xvJf libassuan-3.0.1-1-x86_64.pkg.tar.zst

xz: (stdin): File format not recognized

tar: Child returned status 1

tar: Error is not recoverable: exiting now

 root /arch_repair 

 # Probemos con un mirror robusto y el comando corregido

curl -LO https://archive.archlinux.org/packages/l/libassuan/libassuan-3.0.1-1-x86_64.pkg.tar.zst

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  112k  100  112k    0     0  34901      0  0:00:03  0:00:03 --:--:-- 34909
```

 root /arch_repair 

 zstd -d libassuan-3.0.1-1-x86_64.pkg.tar.zst

tar -xvf libassuan-3.0.1-1-x86_64.pkg.tar -C /mnt/

libassuan-3.0.1-1-x86_64.pkg.tar.zst: 276480 bytes

.BUILDINFO

.MTREE

.PKGINFO

usr/

usr/include/

usr/include/assuan.h

usr/lib/

usr/lib/libassuan.so

usr/lib/libassuan.so.9

usr/lib/libassuan.so.9.0.1

usr/lib/pkgconfig/

usr/lib/pkgconfig/libassuan.pc

usr/share/

usr/share/aclocal/

usr/share/aclocal/libassuan.m4

usr/share/doc/

usr/share/doc/libassuan/

usr/share/doc/libassuan/AUTHORS

usr/share/doc/libassuan/ChangeLog

usr/share/doc/libassuan/NEWS

usr/share/doc/libassuan/README

usr/share/info/

usr/share/info/assuan.info.gz

 root /arch_repair 

 tar --use-compress-program=zstd -xvf libassuan-3.0.1-1-x86_64.pkg.tar.zst -C /mnt/

.BUILDINFO

.MTREE

.PKGINFO

usr/

usr/include/

usr/include/assuan.h

usr/lib/

usr/lib/libassuan.so

usr/lib/libassuan.so.9

usr/lib/libassuan.so.9.0.1

usr/lib/pkgconfig/

usr/lib/pkgconfig/libassuan.pc

usr/share/

usr/share/aclocal/

usr/share/aclocal/libassuan.m4

usr/share/doc/

usr/share/doc/libassuan/

usr/share/doc/libassuan/AUTHORS

usr/share/doc/libassuan/ChangeLog

usr/share/doc/libassuan/NEWS

usr/share/doc/libassuan/README

usr/share/info/

usr/share/info/assuan.info.gz

**Gemini Replied:**

## Gemini ha dicho

¡Excelente! Ahora sí has extraído los archivos reales donde corresponden. Ya tienes el archivo físico `libassuan.so.9` en `/mnt/usr/lib/`.

Sin embargo, hay un detalle técnico importante: el error original decía que `pacman` (el que estás intentando ejecutar desde tu entorno actual) está buscando la librería en `/mnt/usr/lib/libassuan.so.9`, pero **tu sistema actual** parece estar confundido con las rutas o las versiones.

Para que `pacman` vuelva a la vida, necesitamos que el sistema que estás usando *ahora* encuentre esa librería.

## 1. El último empujón: Actualizar el caché de librerías

Como has copiado archivos manualmente a `/mnt`, el sistema operativo no siempre se da cuenta inmediatamente. Ejecuta esto para refrescar los enlaces:

Bash

```
ldconfig -r /mnt
```

## 2. Intenta ejecutar pacman de nuevo

Ahora prueba el comando que falló antes:

Bash

```
pacman --root /mnt -Syu
```

---

## ¿Sigue fallando? El plan de "Emergencia Real"

Si `pacman` sigue quejándose de la misma librería a pesar de que acabas de extraerla, es porque el `pacman` que estás llamando es el de tu sistema dañado y tiene un "path" de librerías roto.

**Prueba a usar el pacman del entorno de rescate (el que funciona) para arreglar el de `/mnt`:**

Bash

```
# Si estás en un Arch Live USB, el comando 'pacman' nativo debería funcionar.
# Usamos el binario del USB para instalar en el disco duro:
pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S libassuan gpgme
```

## 3. El problema de `libgpgme`

Fíjate que el error decía que `libgpgme.so.45` requiere `LIBASSUAN_1.0`. Si tras instalar `libassuan` te da un error similar pero con `gpgme`, repite el proceso de descarga manual para el paquete de **gpgme**:

1. **Descargar:**
   `curl -LO https://archive.archlinux.org/packages/g/gpgme/gpgme-1.23.2-1-x86_64.pkg.tar.zst`
   (o la versión más reciente).
2. **Extraer:**
   `tar --use-compress-program=zstd -xvf gpgme-1.23.2-1-x86_64.pkg.tar.zst -C /mnt/`
3. **Refrescar:** `ldconfig -r /mnt`

---

**¿Qué mensaje te da ahora al escribir simplemente `pacman --version` o intentar el `-Syu`?** Si logramos que ese comando responda, habremos ganado la batalla.

> **You asked:**

Has dicho

 ldconfig -r /mnt

ldconfig: Can't create temporary cache file /usr/local/etc/ld.so.cache~: No such file or directory

 root /arch_repair 

 pacman --root /mnt -Syu

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

 root /arch_repair 

 # Si estás en un Arch Live USB, el comando 'pacman' nativo debería funcionar.

# Usamos el binario del USB para instalar en el disco duro:

pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S libassuan gpgme

pacman: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

**Gemini Replied:**

## Gemini ha dicho

Esto es frustrante, pero el error nos está dando una pista clave que antes se nos escapó: fíjate en las rutas del error.

```
pacman: /mnt/usr/lib/libassuan.so.9: version 'LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)
```

El binario de `pacman` que estás ejecutando está mezclando rutas: busca la librería en **/mnt/usr/lib/** pero dice que la necesita para **/lib/libgpgme.so.45** (que es la raíz de tu entorno de rescate, no de la partición montada).

Tienes un conflicto de "entornos". El `pacman` del sistema de rescate está intentando usar las librerías de la partición montada. Vamos a forzar el uso **exclusivo** de las librerías del sistema de rescate para arreglar el disco.

## 1. El truco del LD_LIBRARY_PATH

Vamos a decirle a `pacman` que ignore temporalmente las librerías de `/mnt` y use solo las que funcionan (las del entorno donde estás parado):

Bash

```
LD_LIBRARY_PATH=/usr/lib pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S libassuan gpg
```

---

## 2. Si eso falla: El "Chroot" manual

Si el binario sigue rebelde, tenemos que entrar "dentro" de tu instalación de Arch, pero **sin usar las librerías rotas**. Para eso, usaremos el comando `arch-chroot` (que debería estar disponible en tu entorno de rescate):

Bash

```
arch-chroot /mnt
```

**Si logras entrar al prompt del chroot:** Intenta ejecutar `pacman -Syu`. Si te da el mismo error de `libassuan`, descarga el paquete **dentro** del chroot (como hicimos antes con `curl`) y extráelo en `/` (que dentro del chroot ya apunta a tu disco real).

---

## 3. ¿Por qué `ldconfig` falló?

El error `Can't create temporary cache file` indica que el sistema de archivos de rescate es de solo lectura o no tiene los directorios esperados. No te preocupes por eso ahora, lo importante es que `pacman` reconozca la librería física que ya pusimos en `/mnt/usr/lib/`.

## 4. Última opción: Copia "bruta"

Si nada de lo anterior funciona, vamos a copiar la librería **que sí funciona** (la de tu entorno de rescate) directamente sobre la que está rota en el disco:

Bash

```
cp -a /usr/lib/libassuan.so* /mnt/usr/lib/
cp -a /usr/lib/libgpgme.so* /mnt/usr/lib/
```

*Después de esto, intenta de nuevo el `pacman --root /mnt -Syu`.*

**¿Qué ocurre al intentar el comando con `LD_LIBRARY_PATH` del paso 1?** Es la forma más limpia de "puentear" el error de versiones.

> **You asked:**

Has dicho

 root /arch_repair 

 LD_LIBRARY_PATH=/usr/lib pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S libassuan gpgme

pacman: /usr/lib/libc.so.6: version `GLIBC_2.42' not found (required by /usr/lib/libgpg-error.so.0)

pacman: /usr/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

pacman: /usr/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libresolv.so.2)

 root /arch_repair 

 arch-chroot /mnt

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> WARNING: /mnt is not a mountpoint. This may have undesirable side effects.

mount: /mnt/proc: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

==> ERROR: failed to setup chroot /mnt

 root /arch_repair 

 cp -a /usr/lib/libassuan.so* /mnt/usr/lib/

cp -a /usr/lib/libgpgme.so* /mnt/usr/lib/

 root /arch_repair 

 pacman --root /mnt -Syu

error: failed to initialize alpm library:

(root: /mnt, dbpath: /mnt/var/lib/pacman/)

could not find or read directory

**Gemini Replied:**

## Gemini ha dicho

Vaya, esto se ha convertido en una comedia de errores de dependencias. El nuevo error de `GLIBC_2.42` indica que el sistema de rescate que estás usando (el USB o entorno actual) es **más antiguo** que los binarios que tienes en el disco, o viceversa. Están hablando idiomas diferentes.

Además, el aviso `==> WARNING: /mnt is not a mountpoint` es la clave de por qué todo falla: **No tienes tu partición de Arch montada en `/mnt`**. Estás trabajando sobre una carpeta vacía o inexistente en la RAM del sistema de rescate.

Vamos a hacer las cosas por el libro para salir de este bucle:

## 1. Identifica y monta tu partición real

Primero, busquemos dónde está tu sistema operativo:

Bash

```
lsblk
```

*(Busca la partición que debería ser tu "root", por ejemplo `/dev/sda2` o `/dev/nvme0n1p2`).*

Ahora, móntala correctamente:

Bash

```
mount /dev/sdXy /mnt   # Sustituye sdXy por tu partición real
```

## 2. Verifica el contenido

Si haces `ls /mnt`, deberías ver las carpetas `etc`, `usr`, `var`, etc. Si la carpeta está vacía, no estamos en el lugar correcto.

---

## 3. El método definitivo: El binario estático

Como las librerías (`glibc`, `libassuan`, `gpgme`) están en una guerra de versiones entre tu USB y tu disco, la mejor opción es usar el **pacman-static**, que lleva todas sus librerías dentro y no depende de nada del sistema.

Si tienes internet en ese entorno, haz esto:

1. **Descarga el binario estático:**

   Bash

   ```
   curl -LO https://pkgbuild.com/~morganamilo/pacman-static/x86_64/bin/pacman-static
   chmod +x pacman-static
   ```

2. **Úsalo para reparar el sistema montado:**

   Bash

   ```
   ./pacman-static --root /mnt --cachedir /mnt/var/cache/pacman/pkg -Syu libassuan gpgme coreutil
   ```

## 4. ¿Por qué falló el `cp -a`?

Como `/mnt` no era un punto de montaje real, copiaste las librerías a una carpeta temporal en la memoria RAM del USB. Al montar la partición real en el paso 1, esos cambios "desaparecerán" y trabajaremos sobre los archivos reales del disco duro.

## Resumen de pasos a seguir ahora:

1. Ejecuta `lsblk` y dime qué particiones ves.
2. Monta la partición raíz en `/mnt`.
3. Si tienes internet, intenta el paso del `pacman-static`.

**¿Qué te devuelve el comando `lsblk`?** Así te puedo decir exactamente qué comando de `mount` usar.

> **You asked:**

Has dicho

 curl -LO https://pkgbuild.com/~morganamilo/pacman-static/x86_64/bin/pacman-static

chmod +x pacman-static

```
 % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 4498k  100 4498k    0     0   274k      0  0:00:16  0:00:16 --:--:--  127k
```

 root /arch_repair 

 ./pacman-static --root /mnt --cachedir /mnt/var/cache/pacman/pkg -Syu libassuan gpgme coreutils

error: no se pudo iniciar la biblioteca alpm:

(root: /mnt, dbpath: /mnt/var/lib/pacman/)

could not find or read directory

 root /arch_repair 

 mount /dev/sda1 /mnt  # Sustituye sdXy por tu partición real

mount: /dev/sda1/mnt: can't find in /etc/fstab.

 root /arch_repair 

 lsblk

NAME  MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS

loop0  7:0  0  4.1G  1 loop

loop1  7:1  0  92.6M  1 loop

loop2  7:2  0  1.2G  1 loop

loop3  7:3  0  1.8G  1 loop

loop4  7:4  0 886.9M  1 loop

sda  8:0  0  149G  0 disk

`-sda1  8:1  0  149G  0 part /

sdb  8:16  1  28.8G  0 disk

`-sdb1  8:17  1  28.8G  0 part

 root /arch_repair 

 ./pacman-static --root /mnt --cachedir /mnt/var/cache/pacman/pkg -Syu libassuan gpgme coreutils

error: no se pudo iniciar la biblioteca alpm:

(root: /mnt, dbpath: /mnt/var/lib/pacman/)

could not find or read directory

**Gemini Replied:**

# Gemini ha dicho

Aquí está el misterio revelado: según tu `lsblk`, **tu partición de Arch (`/dev/sda1`) ya está montada en `/` (la raíz actual)**, no en `/mnt`.

El error `could not find or read directory` ocurre porque `/mnt` está vacío y `pacman-static` no encuentra las bases de datos de paquetes ahí. Estás intentando reparar el sistema "desde adentro" mientras el sistema cree que está afuera.

Como el `pacman` normal de tu sistema está roto por las librerías, pero ya tienes el `pacman-static` (que es inmune a ese error), vamos a usarlo directamente sobre tu raíz real.

## La solución directa

Ejecuta este comando (quitamos el `--root /mnt` porque tu sistema ya es `/`):

Bash

```
./pacman-static -Syu libassuan gpgme coreutils
```

---

## ¿Qué está pasando exactamente?

1. **Tu entorno:** Estás ejecutando un sistema donde `/dev/sda1` es la raíz.
2. **El conflicto:** El `pacman` instalado en `/usr/bin/pacman` falla porque `libgpgme` y `libassuan` no coinciden en versiones.
3. **La herramienta:** `pacman-static` no necesita esas librerías externas, así que él sí puede entrar, descargar las versiones correctas y sobrescribir las que están rotas en tu disco.

## Si el comando anterior te da error de "lock":

Si dice que `db.lck` existe, bórralo y reintenta:

Bash

```
rm /var/lib/pacman/db.lck
./pacman-static -Syu libassuan gpgme coreutils
```

## Una vez que termine:

Prueba si el pacman normal ya revive:

Bash

```
pacman --version
```

**¿Logró `pacman-static` completar la actualización o te lanzó algún error de firmas/archivos existentes?** Si te da error de "exists in filesystem", podemos añadir `--overwrite '*'` al comando.

> **You asked:**

Has dicho

 root /arch_repair 

 ./pacman-static -Syu libassuan gpgme coreutils

error: Error de GPGME: Invalid crypto engine

error: Error de GPGME: Invalid crypto engine

:: Sincronizando las bases de datos de los paquetes...

core está actualizado

extra 8.0 MiB 392 KiB/s 00:21 [#######################################] 100%

multilib está actualizado

andontie-aur 140.7 KiB 23.9 KiB/s 00:06 [#######################################] 100%

archlinuxgr está actualizado

archlinuxcn 1341.5 KiB 191 KiB/s 00:07 [#######################################] 100%

extra-alucryd está actualizado

arcanisrepo 49.7 KiB 7.10 KiB/s 00:07 [#######################################] 100%

chaotic-aur 646.8 KiB 80.7 KiB/s 00:08 [#######################################] 100%

archstrike está actualizado

error: Error de GPGME: Invalid crypto engine

error: Error de GPGME: Invalid crypto engine

error: no se han podido sincronizar todas las bases de datos (error inesperado)

 root /arch_repair 

 rm /var/lib/pacman/db.lck

./pacman-static -Syu libassuan gpgme coreutils

rm: cannot remove '/var/lib/pacman/db.lck': No such file or directory

error: Error de GPGME: Invalid crypto engine

error: Error de GPGME: Invalid crypto engine

:: Sincronizando las bases de datos de los paquetes...

core está actualizado

extra está actualizado

multilib está actualizado

andontie-aur 140.7 KiB 193 KiB/s 00:01 [#######################################] 100%

archlinuxgr está actualizado

archlinuxcn está actualizado

extra-alucryd está actualizado

arcanisrepo 49.7 KiB 10.6 KiB/s 00:05 [#######################################] 100%

chaotic-aur está actualizado

archstrike está actualizado

error: Error de GPGME: Invalid crypto engine

error: Error de GPGME: Invalid crypto engine

error: no se han podido sincronizar todas las bases de datos (error inesperado)

 root /arch_repair 

 pacman --version

.--. Pacman v7.0.0 - libalpm v15.0.0

/ _.-' .-. .-. .-. Copyright (C) 2006-2024 Pacman Development Team

\ '-. '-' '-' '-' Copyright (C) 2002-2006 Judd Vinet

'--'

This program may be freely redistributed under

the terms of the GNU General Public License.

**Gemini Replied:**

## Gemini ha dicho

¡Buenas noticias! Tu binario de `pacman` normal **ya ha resucitado** (v7.0.0), lo cual es un gran avance. Sin embargo, tenemos un último obstáculo: el error `Invalid crypto engine`.

Este error ocurre porque las librerías de cifrado ( `gnupg` , `gpgme` ) están en un estado inconsistente y `pacman` no puede validar las firmas digitales de los repositorios. Básicamente, `pacman` puede

ejecutarse, pero no confía en lo que descarga.

Dado que tienes una mezcla de repositorios oficiales y muchos de terceros (chaotic-aur, archstrike, etc.), vamos a forzar la reparación **desactivando temporalmente la verificación de firmas** solo para reinstalar lo básico.

## 1. El bypass de seguridad (Temporal)

Vamos a usar `pacman-static` (que es más robusto ante estos errores) ignorando las firmas para poder pisar las librerías corruptas.

Ejecuta este comando exacto:

Bash

```
./pacman-static -S libassuan gpgme gnupg coreutils --needed --siglevel TrustAll
```

- `--siglevel TrustAll`: Le dice a pacman que no se queje por el "Invalid crypto engine" y confíe en los paquetes descargados por esta única vez.
- `--needed`: Solo instalará si hay algo que actualizar o corregir.

---

## 2. Si el error persiste (Plan B)

Si el comando anterior sigue fallando por el motor criptográfico, significa que `pacman-static` también está intentando inicializar un subsistema de GPG que está roto en tu disco. En ese caso, vamos a forzar la instalación sin sincronizar bases de datos:

Bash

```
./pacman-static -U /var/cache/pacman/pkg/libassuan*.pkg.tar.zst /var/cache/pacman/pkg/gpgme*.pkg.
```

*(Esto asume que los paquetes están en tu caché. Si no, descárgalos con `curl` como hicimos antes y apunta la ruta hacia ellos).*

---

## 3. El paso final de limpieza

Una vez que las librerías estén en su sitio y `pacman` no lance errores:

1. Reinstala el llavero de Arch para asegurar que todo sea legal: `pacman -S archlinux-keyring`
2. Refresca las llaves: `pacman-key --init` `pacman-key --populate archlinux`

**¿Qué te devuelve el comando del Paso 1?** Si logramos pasar ese "Invalid crypto engine", tu sistema estará 100% recuperado.

### You asked:

Has dicho

 root /arch_repair 

 ./pacman-static -S libassuan gpgme gnupg coreutils --needed --needed --siglevel TrustAll

./pacman-static: unrecognized option: siglevel

 root /arch_repair 

 ./pacman-static -U /var/cache/pacman/pkg/libassuan*.pkg.tar.zst /var/cache/pacman/pkg/gpgme*.pkg.tar.zst --siglevel TrustAll

./pacman-static: unrecognized option: siglevel

 root /arch_repair 

 pacman -S archlinux-keyring

error: GPGME error: Invalid crypto engine

error: GPGME error: Invalid crypto engine

error: database 'andontie-aur' is not valid (invalid or corrupted database (PGP signature))

error: database 'arcanisrepo' is not valid (invalid or corrupted database (PGP signature))

 root /arch_repair 

### Gemini Replied:

## Gemini ha dicho

¡Cierto! Mi error, el flag `--siglevel` se introdujo en versiones recientes de `pacman` pero no siempre se reconoce como argumento de línea de comandos directo en todas las compilaciones de `pacman-static`.

El problema es que tus bases de datos de repositorios (especialmente las de AUR como `andontie-aur`) están firmadas y `pacman` no puede validarlas porque el "motor" (GPG) sigue roto.

Vamos a ir por el camino manual, sin validaciones, para forzar el arreglo:

## 1. Desactivar firmas en el archivo de configuración

Como no podemos pasarlo por comando, editemos el archivo de configuración global:

1. Abre el archivo: `nano /etc/pacman.conf`
2. Busca la línea que dice `SigLevel = Required DatabaseOptional` (o similar).
3. Cámbiala temporalmente por:

   Ini, TOML

   ```
   SigLevel = Never
   ```

4. Guarda y sal ( `Ctrl+O` , `Enter` , `Ctrl+X` ).

## 2. Ejecutar la reparación

Ahora que `pacman` no intentará validar nada, el error de `GPGME` no debería detener el proceso:

Bash

```
./pacman-static -S libassuan gpgme gnupg coreutils --needed
```

---

## 3. Si el error "Invalid crypto engine" persiste

Si incluso con `SigLevel = Never` el binario se niega a trabajar porque intenta inicializar la librería `gpgme` y esta falla, tenemos que usar la fuerza bruta con el archivo que ya tienes descargado:

Bash

```
# Reinstalar libassuan manualmente (ya lo tienes en la carpeta actual)
./pacman-static -U libassuan-3.0.1-1-x86_64.pkg.tar.zst --nodeps
```

Luego haz lo mismo para `gpgme` :

1. Descárgalo:
   `curl -LO https://archive.archlinux.org/packages/g/gpgme/gpgme-1.23.2-1-x86_64.pkg.tar.zst`
2. Instálalo: `./pacman-static -U gpgme-1.23.2-1-x86_64.pkg.tar.zst --nodeps`

---

## 4. Volver a la normalidad

Una vez que el comando de instalación termine sin errores:

1. **IMPORTANTE:** Vuelve a poner `SigLevel = Required DatabaseOptional` en `/etc/pacman.conf`. ¡No dejes tu sistema sin seguridad!
2. Prueba el pacman normal: `pacman -Sy`
3. Si funciona, actualiza los llaveros: `pacman -S archlinux-keyring chaotic-keyring` (y cualquier otro que uses).

**¿Qué pasó al cambiar el `SigLevel` a `Never`?** Eso debería silenciar por completo los errores de GPG y permitir que el sistema se sobrescriba a sí mismo.

> **You asked:**

Has dicho

 root /arch_repair 

 pacman -S archlinux-keyring

warning: archlinux-keyring-20260301-1 is up to date -- reinstalling

resolving dependencies...

looking for conflicting packages...

Packages (1) archlinux-keyring-20260301-1

Total Download Size:  1.20 MiB

Total Installed Size:  1.71 MiB

Net Upgrade Size:  0.00 MiB

:: Proceed with installation? [Y/n]

:: Retrieving packages...

archlinux-keyring-20260301-1-any  1227.6 KiB  439 KiB/s 00:03
[#######################################################] 100%

(1/1) checking keys in keyring
[#######################################################]
100%

(1/1) checking package integrity
[#######################################################] 100%

(1/1) loading package files
[#######################################################]
100%

(1/1) checking for file conflicts
[#######################################################] 100%

(1/1) checking available disk space
[#######################################################] 100%

:: Processing package changes...

(1/1) reinstalling archlinux-keyring
[#######################################################] 100%

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

:: Running post-transaction hooks...

(1/3) Reloading system manager configuration...

Skipped: Running in chroot.

(2/3) Restarting marked services...

Skipped: Running in chroot.

(3/3) Arming ConditionNeedsUpdate...

 root /arch_repair 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_2.0' not found (required by gpg)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_2.0' not found (required by gpg)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_2.0' not found (required by gpg)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_2.0' not found (required by gpg)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /arch_repair 

 pacman-key --populate archlinux

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /mnt/usr/lib/libassuan.so.9: version `LIBASSUAN_2.0' not found (required by gpg)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: There is no secret key available to sign with.

==> Use 'pacman-key --init' to generate a default secret key.

 root /arch_repair 

> ✦ **Gemini Replied:**

## Gemini ha dicho

¡Estamos tan cerca! El binario de `pacman` ya vive, pero **GPG** (la herramienta que usa pacman para las llaves) sigue rota porque está intentando usar librerías mezcladas de `/mnt` y de tu sistema actual.

El error `version LIBASSUAN_2.0 not found` indica que `gpg` es muy exigente con la versión de la librería. Vamos a terminar de limpiar este desastre de versiones forzando la actualización de los componentes criptográficos.

## 1. Forzar la actualización de la base del sistema

Como ya lograste que `pacman` instalara el keyring, vamos a usarlo para que se actualice a sí mismo y a sus dependencias críticas, ignorando los errores de GPG por ahora.

**Ejecuta esto:**

Bash

```
./pacman-static -Syu libassuan gpgme gnupg coreutils --needed --overwrite '*'
```

*(Si te pregunta por algún conflicto, dile que sí a todo. El objetivo es que las librerías en el disco coincidan con lo que los binarios esperan).*

---

## 2. El truco del "limpiador" de librerías

Si `gpg` (usado por `pacman-key`) sigue quejándose de `/mnt/usr/lib/...`, es porque hay un desajuste entre el entorno donde estás y el disco.

Para que `pacman-key --init` funcione, necesitamos que `gpg` vea las librerías correctas. Intenta "engañarlo" así:

Bash

```
LD_LIBRARY_PATH=/usr/lib pacman-key --init
LD_LIBRARY_PATH=/usr/lib pacman-key --populate archlinux
```

---

## 3. Reparación de los repositorios de terceros

Veo que tienes muchos repositorios como `chaotic-aur`, `archstrike`, etc. A veces estos repositorios tienen llaves que causan conflictos si el sistema no está al día.

Si los pasos anteriores fallan, vamos a ser drásticos:

1. **Comenta (pon un `#` al principio)** todos los repositorios extra en `/etc/pacman.conf` (deja solo `core`, `extra` y `multilib`).
2. Limpia el caché de llaves: `rm -rf /etc/pacman.d/gnupg`
3. Intenta de nuevo: `pacman-key --init` `pacman-key --populate archlinux`

---

## ¿Por qué `pacman` funcionó pero `pacman-key` no?

`pacman` es un binario que usa `libalpm`, mientras que `pacman-key` es un script que llama al binario `gpg`. Tu `gpg` actual está intentando cargar librerías de `/mnt/usr/lib` que no son compatibles con su versión. Al actualizar `gnupg` con el comando del **Paso 1**, deberíamos solucionar esa discrepancia.

**¿Qué salida te da el comando del Paso 1?** Si ese termina con éxito, el 99% de tus problemas habrán desaparecido.

> **You asked:**

Has dicho

Quiero compilar pacman-key

> **Gemini Replied:**

## Gemini ha dicho

Entiendo tu frustración. Estás intentando compilar o arreglar las herramientas base porque el sistema de librerías está en un estado de "infierno de dependencias" (específicamente con `libassuan` y `glibc`).

Sin embargo, **compilar `pacman-key` no es la solución**, porque `pacman-key` es en realidad un **script de Bash**, no un binario compilado. Su trabajo es simplemente ser un "envoltorio" (wrapper) para `gpg`. El que está fallando es el binario `gpg` de tu sistema porque no encuentra la versión de `libassuan` que necesita.

Si realmente quieres ver el código o "compilar" el paquete que lo contiene (que es el mismo código fuente de `pacman`), aquí tienes los pasos, pero ten en cuenta que **la compilación fallará** si las librerías actuales están rotas.

## 1. Preparar el entorno de compilación

Para compilar `pacman` (que incluye a `pacman-key`), necesitas `meson` y `ninja`.

Bash

```
# Descargar el código fuente oficial
git clone https://gitlab.archlinux.org/pacman/pacman.git
cd pacman
```

## 2. Compilar

Bash

```
meson setup build -Ddoc=disabled  # Desactivamos docs para evitar más dependencias
ninja -C build
```

*Si esto falla, te dará errores de `LIBASSUAN` o `GLIBC`, confirmando que el problema es la base del sistema y no las herramientas.*

---

## La solución real: ¿Por qué `gpg` sigue pidiendo `libassuan.so.9`?

El error que mostraste antes es muy revelador:
`gpg: /mnt/usr/lib/libassuan.so.9: version LIBASSUAN_2.0 not found`

Tienes una ensalada de rutas. GPG está buscando en `/mnt/usr/lib/` (que es una ruta de rescate) en lugar de en `/usr/lib/`.

**Hagamos un "bypass" manual para que puedas volver a usar el sistema sin compilar nada:**

1. **Sincroniza las librerías a la fuerza:** Como ya vimos que tu `/dev/sda1` es tu `/`, copia la librería que funciona a donde `gpg` la está buscando:

   Bash

   ```
   # Forzar que la librería que acabas de instalar sea la que el sistema reconozca
   cp /usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.9
   ldconfig
   ```

2. **Usa el binario estático para limpiar GPG:** Ya que tienes `pacman-static`, úsalo para reinstalar el paquete que contiene a `gpg` y sus librerías, ignorando las dependencias rotas:

   Bash

   ```
   ./pacman-static -S gnupg libassuan gpgme --overwrite '*'
   ```

3. **Prueba `pacman-key` de nuevo:**

   Bash

   ```
   pacman-key --init
   ```

## ¿Por qué esto es mejor que compilar?

Si intentas compilar ahora, el compilador ( `gcc` ) intentará enlazarse con las mismas librerías rotas que están bloqueando a `pacman` . Es como intentar construir una escalera nueva usando una madera que está podrida; se va a romper antes de terminar.

**¿Qué error te da el paso 2 ( `./pacman-static -S gnupg...` )?** Si ese comando termina, el script `pacman-key` volverá a funcionar mágicamente porque su dependencia ( `gpg` ) estará sana.

> **You asked:**

Has dicho

 root /arch_repair/pacman 

 memeson setup build -Ddoc=disabled  # Desactivamos docs para evitar más dependencias

ninja -C build

Directory already configured.

Just run your build command (e.g. ninja) and Meson will regenerate as necessary.

Run "meson setup --reconfigure" to force Meson to regenerate.

If build failures persist, run "meson setup --wipe" to rebuild from scratch

using the same options as passed when configuring the build.

ninja: Entering directory `build'

ninja: no work to do.

 root /arch_repair/pacman 

 meson setup --reconfigure

ERROR: Must specify at least one directory name.

 root /arch_repair/pacman 

 meson setup --reconfigure .

ERROR: Source and build directories must not be the same. Create a pristine build directory.

 root /arch_repair/pacman 

**Gemini Replied:**

## Gemini ha dicho

Parece que ya habías intentado configurar el proyecto o que Meson se quedó a medias. En Meson, siempre debes separar el código fuente de la carpeta donde se compila. Además, el comando `--reconfigure` necesita que le digas qué carpeta de construcción quieres resetear.

Como tu sistema está en un estado muy inestable con las librerías, lo mejor es **empezar de cero** para asegurarnos de que no haya basura de intentos anteriores.

## 1. Limpieza y Reconfiguración

Ejecuta estos comandos en orden dentro de `/arch_repair/pacman`:

Bash

```
# Borra la carpeta de construcción previa para evitar conflictos
rm -rf build

# Configura de nuevo apuntando a una carpeta "build" limpia
meson setup build -Ddoc=disabled -Dcrypto=gpgme

# Entra y compila
ninja -C build
```

---

## 2. ¿Cómo usar lo que acabas de compilar?

Una vez que `ninja` termine (si las librerías del sistema le permiten enlazar), no habrás arreglado el sistema todavía, solo habrás creado un binario nuevo dentro de esa carpeta. Los scripts como `pacman-key` estarán en: `build/scripts/pacman-key`

Podrías intentar ejecutarlo directamente:

Bash

```
./build/scripts/pacman-key --init
```

---

## 3. El "Elefante en la habitación" (Advertencia)

Te lo digo como un colega que ha roto Arch muchas veces: **Compilar `pacman` no va a arreglar el error de `libassuan`**.

`pacman-key` es un script que llama a `/usr/bin/gpg`. Si `/usr/bin/gpg` sigue roto porque no encuentra `LIBASSUAN_2.0`, el `pacman-key` que acabas de compilar **dará exactamente el mismo error**, porque ambos dependen del mismo binario de GPG instalado en tu sistema.

**Si `ninja` te da error de compilación**, es la confirmación definitiva de que tus librerías de desarrollo (`headers`) no coinciden con las librerías binarias.

## ¿Qué hacer si quieres terminar rápido?

Si la compilación falla o el nuevo `pacman-key` sigue sin funcionar, haz este último intento de fuerza bruta para que `gpg` deje de quejarse:

Bash

```
# Esto fuerza a que el sistema use la librería que ya tienes para GPG
ln -sf /usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.9
# Y por si acaso gpg busca una versión vieja que no tienes:
ln -sf /usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.0
```

**¿Logró `ninja` terminar la compilación o se detuvo con un error de "ld" (linker)?** Si terminó, cuéntame si el nuevo script `./build/scripts/pacman-key` se comporta distinto.

> **You asked:**

Has dicho

 root /arch_repair/pacman 

 # # Borra la carpeta de construcción previa para evitar conflictos

rm -rf build

# Configura de nuevo apuntando a una carpeta "build" limpia

meson setup build -Ddoc=disabled -Dcrypto=gpgme

# Entra y compila

ninja -C build

The Meson build system

Version: 1.7.0

Source dir: /arch_repair/pacman

Build dir: /arch_repair/pacman/build

Build type: native build

meson.build:1:0: ERROR: Value "gpgme" (of type "string") for option "crypto" is not one of the choices. Possible choices are

(as string): "openssl", "nettle".

A full log can be found at /arch_repair/pacman/build/meson-logs/meson-log.txt

ninja: Entering directory `build'

ninja: error: loading 'build.ninja': No such file or directory

 root /arch_repair/pacman 

**Gemini Replied:**

## Gemini ha dicho

Parece que las opciones de compilación de `pacman` han cambiado en las versiones más recientes y ya no aceptan `gpgme` como backend criptográfico directo para el binario principal (ahora prefiere **openssl** o **nettle**). Por eso Meson se detuvo y Ninja no encontró nada que construir.

Sin embargo, insisto: **compilar pacman no arreglará `pacman-key`**, porque `pacman-key` es un script que usa el `gpg` que ya tienes instalado. Si `gpg` está roto, el `pacman-key` nuevo también lo estará.

Vamos a intentar la compilación con los parámetros correctos para que al menos tengas los binarios frescos:

## 1. Reintentar la configuración con OpenSSL

Bash

```
rm -rf build
meson setup build -Ddoc=disabled -Dcrypto=openssl
ninja -C build
```

---

## 2. El problema real: Tu `gpg` está "mismatch"

El error que viste antes ( `LIBASSUAN_2.0 not found` ) confirma que tu binario de `/usr/bin/gpg` es más nuevo (o viejo) que la librería `libassuan.so.9` que extrajiste manualmente.

Como ya tienes `pacman-static` y este **SÍ** funciona, vamos a usarlo para "limpiar" el desastre de librerías de forma definitiva antes de que sigas compilando:

**Ejecuta esto para arreglar las dependencias de GPG:**

Bash

```
./pacman-static -Sy gnupg libassuan gpgme --overwrite '*'
```

## 3. ¿Por qué esto es vital?

Si logras que ese comando termine, el `gpg` de tu sistema volverá a funcionar. En ese momento, **no necesitarás compilar nada**, porque el `pacman-key` original de tu sistema volverá a la vida automáticamente.

**¿Qué ocurre si ejecutas el comando de `pacman-static` del punto 2?** Si te da error de "Invalid crypto engine", recuerda que ya pusimos `SigLevel = Never` en `/etc/pacman.conf`, por lo que debería dejarte instalar esos tres paquetes clave sin protestar.

Una vez instalados, prueba: `gpg --version`

Si `gpg --version` no da error de librerías, **habrás ganado**. Luego podrás hacer `pacman-key --init` sin problemas.

> **You asked:**

Has dicho

 root /arch_repair/pacman 

 rmrm -rf build

meson setup build -Ddoc=disabled -Dcrypto=openssl

ninja -C build

The Meson build system

Version: 1.7.0

Source dir: /arch_repair/pacman

Build dir: /arch_repair/pacman/build

Build type: native build

Project name: pacman

Project version: 7.1.0

C compiler for the host machine: cc (gcc 14.2.0 "cc (Debian 14.2.0-19) 14.2.0")

C linker for the host machine: cc ld.bfd 2.44

Host machine cpu family: x86_64

Host machine cpu: x86_64

Program python3 found: YES (/usr/bin/python3)

Program bash found: YES (/usr/bin/bash)

Found pkg-config: YES (/usr/sbin/pkg-config) 2.5.1

Run-time dependency bash-completion found: YES 2.16.0

Run-time dependency intl found: YES

Run-time dependency libarchive found: YES 3.8.6

Run-time dependency libcurl found: YES 8.19.0

Run-time dependency gpgme found: YES 2.0.1

Run-time dependency libcrypto found: YES 3.6.1

Run-time dependency libseccomp found: YES 2.6.0

Has header "linux/landlock.h" : YES

Has header "mntent.h" : YES

Has header "sys/mnttab.h" : NO

Has header "sys/mount.h" : YES

Has header "sys/param.h" : YES

Has header "sys/prctl.h" : YES

Has header "sys/statvfs.h" : YES

Has header "sys/types.h" : YES

Has header "sys/ucred.h" : NO

Has header "termios.h" : YES

Checking for function "getmntent" : YES

Checking for function "getmntinfo" : NO

Checking for function "strndup" : YES

Checking for function "strnlen" : YES

Checking for function "strsep" : YES

Checking for function "swprintf" : YES

Checking for function "tcflush" : YES

Checking whether type "struct stat" has member "st_blksize" : YES

Checking whether type "struct statvfs" has member "f_flag" : YES

Checking whether type "struct statfs" has member "f_flags" : NO

Checking for type "mode_t" : YES

Checking for type "uid_t" : YES

Checking for type "off_t" : YES

Checking for type "pid_t" : YES

Checking for type "size_t" : YES

Checking for type "ssize_t" : YES

Checking for type "int64_t" : YES

Compiler for C supports arguments -pedantic: YES

Compiler for C supports arguments -Wcast-align: YES

Compiler for C supports arguments -Wclobbered: YES

Compiler for C supports arguments -Wempty-body: YES

Compiler for C supports arguments -Wfloat-equal: YES

Compiler for C supports arguments -Wformat-nonliteral: YES

Compiler for C supports arguments -Wformat-security: YES

Compiler for C supports arguments -Wignored-qualifiers: YES

Compiler for C supports arguments -Wimplicit-fallthrough: YES

Compiler for C supports arguments -Winit-self: YES

Compiler for C supports arguments -Wlogical-op: YES

Compiler for C supports arguments -Wmissing-declarations: YES

Compiler for C supports arguments -Wmissing-field-initializers: YES

Compiler for C supports arguments -Wmissing-parameter-type: YES

Compiler for C supports arguments -Wmissing-prototypes: YES

Compiler for C supports arguments -Wold-style-declaration: YES

Compiler for C supports arguments -Woverride-init: YES

Compiler for C supports arguments -Wpointer-arith: YES

Compiler for C supports arguments -Wredundant-decls: YES

Compiler for C supports arguments -Wshadow: YES

Compiler for C supports arguments -Wsign-compare: YES

Compiler for C supports arguments -Wstrict-aliasing: YES

Compiler for C supports arguments -Wstrict-overflow=5: YES

Compiler for C supports arguments -Wstrict-prototypes: YES

Compiler for C supports arguments -Wtype-limits: YES

Compiler for C supports arguments -Wuninitialized: YES

Compiler for C supports arguments -Wunused-but-set-parameter: YES

Compiler for C supports arguments -Wunused-parameter: YES

Compiler for C supports arguments -Wunused-result: YES

Compiler for C supports arguments -Wwrite-strings: YES

Configuring config.h using configuration

Configuring edit-script.sh using configuration

Program edit-script.sh found: YES (/usr/bin/bash /arch_repair/pacman/build/scripts/edit-script.sh)

```
Configuring libmakepkg.pc using configuration

Program msgfmt found: YES (/usr/bin/msgfmt)

Program msginit found: YES (/usr/bin/msginit)

Program msgmerge found: YES (/usr/bin/msgmerge)

Program xgettext found: YES (/usr/bin/xgettext)

Program asciidoc skipped: feature doc disabled

Program a2x skipped: feature doc disabled

Configuring makepkg using configuration

Configuring pacman-db-upgrade using configuration

Configuring pacman-key using configuration

Configuring repo-add using configuration

Configuring repo-remove using configuration

Configuring repo-elephant using configuration

Configuring makepkg.conf using configuration

Configuring fortran.conf using configuration

Configuring rust.conf using configuration

Configuring pacman.conf using configuration

Checking for size of "ssize_t" : 8

Build targets in project: 250


pacman 7.1.0


 Build information

   prefix               : /usr

   sysconfdir           : /etc

       conf file        : /etc/pacman.conf

   localstatedir        : /var
```

```
    database dir      : /var/lib/pacman/
    cache dir         : /var/cache/pacman/pkg/
  keyringdir          : /usr/share/pacman/keyrings
  compiler            : gcc 14.2.0

  Architecture        : x86_64
  Host Type           : x86_64-linux-gnu
  File inode command    : stat -c '%i %n'
  File seccomp command  : file -S
  libalpm version       : 16.0.1
  pacman version        : 7.1.0

Directory and file information
  root working directory: /
  package extension     : .pkg.tar.gz
  source pkg extension  : .src.tar.gz
  build script name     : PKGBUILD
  template directory    : /usr/share/makepkg-template

Compilation options
  i18n support          : true
  Build docs            : false
  debug build           : true
  Use libcurl           : true
  Use GPGME             : true
  Use OpenSSL           : true
  Use nettle            : false
```

User defined options

crypto : openssl

doc : disabled

Found ninja-1.12.1 at /usr/bin/ninja

ninja: Entering directory `build'

[30/296] Generating scripts/repo-add.sh.in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[56/296] Generating scripts/makepkg-template.pl.in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[59/296] Generating scripts/pacman-db-upgrade.sh.in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[60/296] Generating scripts/makepkg.sh.in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[63/296] Generating scripts/pacman-key.sh.in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[64/296] Generating scripts/libmakepkg/autodep/autodep_library_depends_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[65/296] Generating scripts/libmakepkg/autodep/autodep_library_provides_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[66/296] Generating scripts/libmakepkg/libmakepkg_autodep with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[67/296] Generating scripts/libmakepkg/libmakepkg_buildenv with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[68/296] Generating scripts/libmakepkg/buildenv/buildenv_buildflags_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[69/296] Generating scripts/libmakepkg/buildenv/buildenv_compiler_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[70/296] Generating scripts/libmakepkg/buildenv/buildenv_debugflags_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[71/296] Generating scripts/libmakepkg/buildenv/buildenv_lto_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[72/296] Generating scripts/libmakepkg/buildenv/buildenv_fortran_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[73/296] Generating scripts/libmakepkg/buildenv/buildenv_makeflags_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[74/296] Generating scripts/libmakepkg/libmakepkg_executable with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[75/296] Generating scripts/libmakepkg/buildenv/buildenv_rust_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[77/296] Generating scripts/libmakepkg/executable/executable_ccache_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[78/296] Generating scripts/libmakepkg/executable/executable_checksum_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[79/296] Generating scripts/libmakepkg/executable/executable_distcc_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[80/296] Generating scripts/libmakepkg/executable/executable_fakeroot_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[82/296] Generating scripts/libmakepkg/executable/executable_debugedit_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[83/296] Generating scripts/libmakepkg/executable/executable_gpg_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[84/296] Generating scripts/libmakepkg/executable/executable_gzip_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[85/296] Generating scripts/libmakepkg/executable/executable_pacman_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[86/296] Generating scripts/libmakepkg/executable/executable_sccache_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[87/296] Generating scripts/libmakepkg/executable/executable_sudo_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[88/296] Generating scripts/libmakepkg/executable/executable_strip_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[89/296] Generating scripts/libmakepkg/executable/executable_vcs_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[90/296] Generating scripts/libmakepkg/libmakepkg_integrity with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[91/296] Generating scripts/libmakepkg/integrity/integrity_generate_checksum_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[92/296] Generating scripts/libmakepkg/integrity/integrity_verify_checksum_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[93/296] Generating scripts/libmakepkg/integrity/integrity_generate_signature_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[94/296] Generating scripts/libmakepkg/lint_config/lint_config_ext_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[95/296] Generating scripts/libmakepkg/libmakepkg_lint_config with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[96/296] Generating scripts/libmakepkg/lint_config/lint_config_buildenv_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[97/296] Generating scripts/libmakepkg/integrity/integrity_verify_signature_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[98/296] Generating scripts/libmakepkg/lint_config/lint_config_nproc_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[99/296] Generating scripts/libmakepkg/lint_config/lint_config_packager_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[100/296] Generating scripts/libmakepkg/lint_config/lint_config_source_date_epoch_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[101/296] Generating scripts/libmakepkg/lint_config/lint_config_paths_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[102/296] Generating scripts/libmakepkg/libmakepkg_lint_package with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[103/296] Generating scripts/libmakepkg/lint_config/lint_config_variable_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[104/296] Generating scripts/libmakepkg/lint_package/lint_package_build_references_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[105/296] Generating scripts/libmakepkg/lint_package/lint_package_dotfiles_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[106/296] Generating scripts/libmakepkg/lint_package/lint_package_file_names_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[107/296] Generating scripts/libmakepkg/lint_package/lint_package_missing_backup_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[108/296] Generating scripts/libmakepkg/libmakepkg_lint_pkgbuild with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[109/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_arch_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[110/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_arch_specific_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[111/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_backup_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[112/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_changelog_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[113/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_depends_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[114/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_conflicts_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[115/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_epoch_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[116/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_install_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[117/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_fullpkgver_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[118/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_checkdepends_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[119/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_makedepends_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[120/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_optdepends_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[121/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_options_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[122/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_pkgbase_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[123/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_package_function_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[124/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_package_function_variable_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[125/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_pkgname_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[126/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_pkgrel_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[127/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_pkgver_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[128/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_provides_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[129/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_source_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[130/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_variable_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[131/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_util_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[132/296] Generating scripts/libmakepkg/libmakepkg_reproducible with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[133/296] Generating scripts/libmakepkg/lint_pkgbuild/lint_pkgbuild_xdata_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[134/296] Generating scripts/libmakepkg/reproducible/reproducible_source_date_epoch_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[135/296] Generating scripts/libmakepkg/libmakepkg_source with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[136/296] Generating scripts/libmakepkg/source/source_bzr_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[137/296] Generating scripts/libmakepkg/source/source_file_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[138/296] Generating scripts/libmakepkg/source/source_git_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[139/296] Generating scripts/libmakepkg/source/source_local_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[140/296] Generating scripts/libmakepkg/source/source_fossil_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[141/296] Generating scripts/libmakepkg/source/source_hg_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[142/296] Generating scripts/libmakepkg/source/source_svn_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[143/296] Generating scripts/libmakepkg/libmakepkg_tidy with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[144/296] Generating scripts/libmakepkg/libmakepkg_srcinfo with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[145/296] Generating scripts/libmakepkg/tidy/tidy_10_docs_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[146/296] Generating scripts/libmakepkg/tidy/tidy_50_purge_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[147/296] Generating scripts/libmakepkg/tidy/tidy_10_staticlibs_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[148/296] Generating scripts/libmakepkg/tidy/tidy_50_zipman_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[149/296] Generating scripts/libmakepkg/tidy/tidy_50_strip_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[150/296] Generating scripts/libmakepkg/tidy/tidy_10_libtool_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[151/296] Generating scripts/libmakepkg/tidy/tidy_90_emptydirs_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[152/296] Generating scripts/libmakepkg/libmakepkg_util with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[153/296] Generating scripts/libmakepkg/util/util_compress_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[154/296] Generating scripts/libmakepkg/util/util_config_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[155/296] Generating scripts/libmakepkg/util/util_dirsize_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[156/296] Generating scripts/libmakepkg/util/util_error_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[157/296] Generating scripts/libmakepkg/util/util_message_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[158/296] Generating scripts/libmakepkg/util/util_pkgbuild_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[159/296] Generating scripts/libmakepkg/util/util_option_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[160/296] Generating scripts/libmakepkg/util/util_parseopts_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[161/296] Generating scripts/libmakepkg/util/util_schema_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[162/296] Generating scripts/libmakepkg/util/util_source_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[163/296] Generating scripts/libmakepkg/util/util_util_sh_in with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[164/296] Generating scripts/bash_completion with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[165/296] Generating scripts/zsh_completion with a custom command

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

[296/296] Linking target pacman-conf

**Gemini Replied:**

## Gemini ha dicho

¡Impresionante! Has logrado compilar `pacman` 7.1.0 en medio de un caos de librerías. Esto significa que tu entorno de compilación (headers y compilador) está sano, aunque el sistema de ejecución (runtime) siga teniendo hipo con los locales y GPG.

Ahora tienes los binarios recién salidos del horno en la carpeta `build`. Aquí tienes cómo usarlos para rescatar tu sistema:

## 1. ¿Dónde está tu nuevo `pacman-key`?

El script que querías "compilar" (que ahora se ha generado con las rutas correctas para tu compilación) está aquí: **`build/scripts/pacman-key`**

## 2. Cómo usarlo para arreglar GPG

Como el problema de `pacman-key` es que llama a un `gpg` que no encuentra sus librerías, vamos a intentar ejecutar tu versión recién generada.

Pero antes, un paso crucial: **los locales**. Ese error de `setlocale` puede romper scripts de shell. Genera los locales básicos:

Bash

```
# Esto debería silenciar las advertencias de bash
export LC_ALL=C
export LANG=C
```

Ahora, intenta inicializar las llaves usando tu nuevo script:

Bash

```
# Ejecuta el pacman-key que acabas de construir
./build/scripts/pacman-key --init
./build/scripts/pacman-key --populate archlinux
```

---

## 3. El paso final: Instalar tu propio Pacman

Si quieres que tu sistema deje de depender del `pacman` roto de `/usr/bin`, puedes instalar el que acabas de compilar.

**ADVERTENCIA:** Como estás en un sistema de rescate, no hagas `ninja install` directamente porque podrías ensuciar el entorno de rescate en lugar de tu disco. Lo mejor es usar tu `pacman-static` para instalar los paquetes oficiales ahora que las librerías deberían estar más estables.

Si el `pacman-key --init` de arriba funcionó, haz esto:

Bash

```
./pacman-static -Syu gnupg libassuan gpgme --overwrite '*'
```

---

## 4. ¿Qué sigue?

Si `pacman-key --init` te sigue dando el error de `LIBASSUAN_2.0`, significa que el binario de GPG en `/usr/bin/gpg` es el que está mal y ninguna compilación de `pacman` lo va a arreglar, porque `pacman` **usa** GPG, no lo reemplaza.

**Haz esta prueba rápida:**

Bash

```
gpg --version
```

Si esto falla, olvida la compilación de pacman por un segundo y hagamos el link de emergencia definitivo:

Bash

```
ln -sf /usr/lib/libassuan.so.9 /usr/lib/libassuan.so.0
# A veces GPG busca la versión 0 o la 2 internamente por errores de empaquetado
```

**¿Qué salida te da ahora `gpg --version` y el nuevo `./build/scripts/pacman-key --init`?** Si logramos que GPG responda, el sistema estará curado.

**You asked:**

**Gemini Replied:**

## Gemini ha dicho

El log de `strace` que subiste es la "escena del crimen" perfecta. Confirma exactamente por qué nada de lo que compiles va a funcionar mientras el entorno esté así.

El problema no es `pacman-key`, sino que el cargador de librerías dinámicas (`ld-linux`) está atrapado en un **bucle de rutas cruzadas**.

## El análisis del log: ¿Qué está pasando?

1. **Rutas "secuestradas":** El log muestra que cuando intentas ejecutar tu nuevo `pacman-key`, el sistema busca librerías básicas (como `libtinfo.so.6`) en `/mnt/usr/lib/`.
2. **Fallo de enlace:** Al no encontrarlas en esas rutas específicas de `/mnt`, el binario ni siquiera llega a ejecutarse correctamente.
3. **Inconsistencia de GLIBC:** Tienes una mezcla de binarios compilados para una versión de `glibc` (probablemente la 2.42 de tu Arch dañado) intentando correr sobre un kernel o entorno que espera otra versión.

---

## Por qué compilar no es la salida (por ahora)

Estás compilando con las herramientas de tu entorno de rescate (Debian 14.2.0 según el log de Meson), pero el binario resultante intenta buscar las dependencias en las rutas de Arch que están rotas o mal montadas. Es un choque de arquitecturas de software.

## La solución: Romper el bucle

Si quieres que ese `pacman-key` que compilaste funcione, o que el sistema reviva, tienes que limpiar las variables de entorno que están forzando a los binarios a buscar en `/mnt`.

**1. Limpia el entorno de ejecución:**

Bash

```
export LD_LIBRARY_PATH=/usr/lib:/lib
export LD_PRELOAD=""
```

**2. El "Truco Maestro" (Uso de librerías relativas):** Como ya tienes el binario de `pacman` compilado en `build/pacman`, vamos a ver si puede correr usando *solo* las librerías del sistema de rescate, ignorando `/mnt`:

Bash

```
# Verifica si el binario que compilaste tiene dependencias satisfechas
ldd ./build/pacman
```

**3. Si `ldd` muestra "not found", fuerza el enlace manual:** Si te sigue pidiendo `libassuan.so.9` y no la encuentra, haz que el sistema la vea en la raíz del rescate, no en `/mnt`:

Bash

```
cp /mnt/usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.9
ldconfig
```

## ¿Cuál es el siguiente paso lógico?

Olvidémonos de compilar por un momento. Tienes un binario de `pacman-static` que es tu única herramienta "pura" (no depende de las librerías del sistema).

**Intenta esto para resetear la base de GPG sin usar scripts:**

Bash

```
# Borra el directorio de llaves corrupto de Arch
rm -rf /etc/pacman.d/gnupg

# Usa pacman-static para reinstalar gnupg y forzar los archivos correctos
./pacman-static -S gnupg libassuan gpgme --overwrite '*'

# Ahora intenta el init original (sin el binario compilado)
pacman-key --init
```

**¿Qué te dice `ldd ./build/pacman`?** Si logramos que ese binario vea las librerías de `/usr/lib` en lugar de buscarlas en `/mnt`, podremos usar lo que compilaste para terminar la reparación.

> **You asked:**

Has dicho

 root /arch_repair/pacman 

 # # Verifica si el binario que compilaste tiene dependencias satisfechas

ldd ./build/pacman

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

not a dynamic executable

 root /arch_repair/pacman 

 cpcp /mnt/usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.9

ldconfig

 root /arch_repair/pacman 

 # # Borra el directorio de llaves corrupto de Arch

rm -rf /etc/pacman.d/gnupg

# Usa pacman-static para reinstalar gnupg y forzar los archivos correctos

./pacman-static -S gnupg libassuan gpgme --overwrite '*'

# Ahora intenta el init original (sin el binario compilado)

pacman-key --init

bash: ./pacman-static: No such file or directory

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

pacman-conf: /lib/libassuan.so.9: version `LIBASSUAN_1.0' not found (required by /lib/libgpgme.so.45)

mkdir: cannot create directory '': No such file or directory

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/trustdb.gpg': No such file or directory

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

**Gemini Replied:**

## Gemini ha dicho

El mensaje `not a dynamic executable` sobre tu `build/pacman` indica que, o bien lo compilaste como estático por accidente, o el sistema de archivos está tan confundido que no reconoce el formato ELF. Pero el problema real sigue siendo el **conflicto de versiones de GLIBC** entre el entorno de rescate y tu sistema.

Tienes un choque entre el `libc.so.6` que espera `GLIBC_ABI_DT_X86_64_PLT` (una característica muy reciente de las herramientas de compilación de Arch) y el entorno donde estás ejecutando, que no la soporta.

Para salir de este bucle, vamos a usar la única herramienta que no depende de las librerías del sistema: el `pacman-static` que descargaste antes, pero tienes que ejecutarlo desde donde está guardado.

## 1. Localiza y usa pacman-static

Antes te dio `No such file or directory` porque estabas dentro de `/arch_repair/pacman`. El archivo está en `/arch_repair/`.

Bash

```
# Regresa a la carpeta donde bajaste el binario estático
cd /arch_repair/

# Reinstala la base sin verificar firmas (ya que GPG está roto)
./pacman-static -S gnupg libassuan gpgme coreutils --overwrite '*' --needed
```

## 2. El "Bypass" de GPG (La solución de emergencia)

Como `pacman-key --init` falla porque el binario `gpg` de tu disco no es compatible con el entorno de rescate, vamos a engañar al sistema. Si tienes internet, descarga los binarios de `gnupg` directamente y extráelos sobre tu sistema para que las versiones coincidan.

O mejor aún, intenta esto para que el `pacman` normal deje de pedir `libassuan` de forma incorrecta:

Bash

```
# Limpia cualquier rastro de locales para evitar ruidos
export LC_ALL=C
export LANG=C

# Crea los enlaces que gpg está pidiendo desesperadamente
ln -sf /usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.9
ln -sf /usr/lib/libassuan.so.9.0.1 /usr/lib/libassuan.so.0
```

## 3. ¿Por qué falló el `mkdir` y el `chmod`?

El error `mkdir: cannot create directory '': No such file or directory` ocurre porque las variables de ruta de pacman están vacías o apuntan a lugares inexistentes.

**Haz esto antes de cualquier otra cosa:**

Bash

```
# Asegúrate de que los directorios base existan en tu raíz actual
mkdir -p /etc/pacman.d/gnupg
mkdir -p /root/.gnupg
```

## 4. El último recurso: Usar el GPG del sistema de rescate

Si el `gpg` de tu disco (Arch) no corre por la versión de `glibc`, usa el `gpg` que viene con tu entorno de rescate (Debian) para inicializar las llaves de Arch:

Bash

```
# Usamos el gpg de Debian para crear las llaves en la ruta de Arch
/usr/bin/gpg --homedir /etc/pacman.d/gnupg --init
```

**Si logras que `./pacman-static` (en `/arch_repair/`) termine de instalar `gnupg` y `libassuan`**, el error de `GLIBC_ABI_DT_X86_64_PLT` debería desaparecer porque se instalarán binarios compatibles.

¿Qué te dice ahora el comando `./pacman-static -S gnupg` desde la carpeta `/arch_repair/`?

> **You asked:**

Has dicho

 root /arch_repair 

 /usr/bin/gpg --homedir /etc/pacman.d/gnupg --init

/usr/bin/gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so

.6)

 root /arch_repair 

 readelf -d /usr/lib/x86_64-linux-gnu/libc.so.6

Dynamic section at offset 0x1e3940 contains 31 entries:

```
 Tag        Type                         Name/Value

0x0000000000000001 (NEEDED)             Shared library: [ld-linux-x86-64.so.2]

0x000000000000000e (SONAME)             Library soname: [libc.so.6]

0x0000000000000019 (INIT_ARRAY)         0x1e1bc8

0x000000000000001b (INIT_ARRAYSZ)       24 (bytes)

0x0000000000000004 (HASH)               0x3d0

0x000000006ffffef5 (GNU_HASH)           0x4520

0x0000000000000005 (STRTAB)             0x1be40

0x0000000000000006 (SYMTAB)             0x9600

0x000000000000000a (STRSZ)              34968 (bytes)

0x000000000000000b (SYMENT)             24 (bytes)

0x0000000070000000 (DT_X86_64_PLT)      0x28000

0x0000000070000001 (DT_X86_64_PLTSZ)    0x3d0

0x0000000070000003 (DT_X86_64_PLTENT)   0x10

0x0000000000000003 (PLTGOT)             0x1e4b80

0x0000000000000002 (PLTRELSZ)           1440 (bytes)

0x0000000000000014 (PLTREL)             RELA

0x0000000000000017 (JMPREL)             0x26de8

0x0000000000000007 (RELA)               0x265a8
```

0x0000000000000008 (RELASZ) 2112 (bytes)

0x0000000000000009 (RELAENT) 24 (bytes)

0x000000006ffffffc (VERDEF) 0x25f88

0x000000006ffffffd (VERDEFNUM) 42

0x000000000000001e (FLAGS) BIND_NOW STATIC_TLS

0x000000006ffffffb (FLAGS_1) Flags: NOW

0x000000006ffffffe (VERNEED) 0x26558

0x000000006fffffff (VERNEEDNUM) 1

0x000000006ffffff0 (VERSYM) 0x246d8

0x0000000000000024 (RELR) 0x27388

0x0000000000000023 (RELRSZ) 280 (bytes)

0x0000000000000025 (RELRENT) 8 (bytes)

0x0000000000000000 (NULL) 0x0

**Gemini Replied:**

## Gemini ha dicho

El `readelf` lo confirma: tu sistema está intentando usar una versión de **Glibc 2.41+** (la de Arch) que incluye el nuevo tag dinámico `DT_X86_64_PLT`. Este tag es una optimización reciente en el ecosistema de herramientas de Arch, pero el cargador de librerías (`ld-linux`) o el entorno de ejecución actual (el de tu sistema de rescate/Debian) es más antiguo y no sabe qué hacer con él.

Estás en un "deadlock" técnico: los binarios de Arch requieren una Glibc que el entorno de rescate no puede satisfacer, y los binarios de rescate (como `/usr/bin/gpg`) se confunden al intentar cargar librerías del disco que tienen este nuevo formato.

## La estrategia de escape: Aislamiento total

Para salir de este bucle, debemos evitar que los binarios toquen las librerías del disco (`/usr/lib`) y usen estrictamente las del sistema de rescate, o usar el binario estático que es inmune a esto.

### 1. Reparación con pacman-static (Sin tocar el sistema)

El `pacman-static` no lee el `libc.so.6` del disco, por lo que es tu única salvación real ahora mismo. Úsalo para sobreescribir los paquetes que causan el conflicto.

Bash

```
# Asegúrate de estar en el directorio donde está pacman-static
cd /arch_repair

# Forzamos la reinstalación de la base para que coincida con el sistema de archivos
./pacman-static -S coreutils bash glibc --overwrite '*'
```

### 2. El truco del "Chroot" limpio

Si necesitas ejecutar comandos del disco (como `pacman-key`), tienes que forzar al sistema a **no mirar** `/usr/lib/x86_64-linux-gnu` (ruta de Debian) ni las rutas cruzadas.

Prueba a ejecutar el `gpg` de Arch forzando el cargador de librerías:

Bash

```
# Intentar usar el cargador de librerías de Arch directamente para correr su propio gpg
/lib64/ld-linux-x86-64.so.2 /usr/bin/gpg --homedir /etc/pacman.d/gnupg --init
```

### 3. Limpieza de variables de entorno "fantasma"

A veces, `LD_LIBRARY_PATH` se queda con basura de intentos anteriores. Limpia todo antes de seguir:

Bash

```
unset LD_LIBRARY_PATH
unset LD_PRELOAD
export LC_ALL=C
```

## ¿Por qué `pacman-key` sigue fallando?

Porque `pacman-key` es un script de shell que invoca a `gpg`.

- Si invoca a `/usr/bin/gpg` (el de Arch), falla por `GLIBC_ABI_DT_X86_64_PLT`.
- Si invoca a `/usr/bin/gpg` (el de Debian), falla porque Debian intenta cargar `libassuan` o `libgpg-error` desde `/usr/lib` (que ahora son las de Arch con el tag nuevo).

**La solución definitiva para las llaves:** Si no logras que `gpg` funcione, **desactiva las firmas permanentemente** en `/etc/pacman.conf` para poder terminar la actualización y luego arreglarlas:

1. Edita `/etc/pacman.conf`.
2. Cambia todas las instancias de `SigLevel` a:

Ini, TOML

```
SigLevel = Never
```

3. Actualiza todo el sistema con el estático:

   Bash

```
./pacman-static -Syu --overwrite '*'
```

Una vez que el sistema esté actualizado al 100%, los binarios y las librerías en el disco volverán a estar sincronizados, el error de `readelf` desaparecerá porque el `ld-linux` del disco ya sabrá leer ese tag, y podrás volver a activar las firmas.

**¿Qué ocurre si intentas el `pacman-static -Syu` con `SigLevel = Never`?** Eso debería saltarse el problema de GPG por completo.

> **You asked:**

Has dicho

¿Cómo puedo compilar esta versión exacta GLIBC_ABI_DT_X86_64_PLT y luego instalarla?

> **Gemini Replied:**

## Gemini ha dicho

Entiendo perfectamente hacia dónde vas: quieres que tu entorno de rescate "entienda" ese nuevo tag `DT_X86_64_PLT` para que los binarios de Arch dejen de tirar error.

Sin embargo, te tengo una noticia importante (y un poco pesada): **`GLIBC_ABI_DT_X86_64_PLT` no es una versión de Glibc que puedas bajar y compilar**, sino una **capacidad del linker ( `ld` ) y del cargador dinámico** que se incluyó muy recientemente en la rama principal de GNU C Library (Glibc 2.41+) y en las binutils.

Si intentas compilar Glibc desde cero en un sistema que ya tiene las librerías rotas, es muy probable que el proceso falle porque el compilador mismo no podrá ejecutarse.

---

## La realidad técnica

El tag `DT_X86_64_PLT` es una optimización para que el salto a las librerías dinámicas sea más rápido y seguro (relacionado con Intel IBT). Para que tu sistema lo soporte, necesitas que **tanto el cargador**

**dinámico (`/lib64/ld-linux-x86-64.so.2`) como la `libc.so.6`** sean de la versión 2.41 o superior.

Si realmente quieres intentar compilarlo manualmente en el entorno de rescate, aquí tienes el camino "suicida":

## 1. Requisitos previos

Necesitas `gcc` (mínimo 11+), `binutils` (2.42+) y `python`. Según tu log, tienes GCC 14, así que por ese lado vas bien.

## 2. Obtener el código de Glibc (Git)

Bash

```
git clone https://sourceware.org/git/glibc.git
cd glibc
git checkout glibc-2.41  # O la versión más reciente
```

## 3. Compilación (Fuera del árbol de fuentes)

**CRÍTICO:** Nunca compiles Glibc dentro de su propia carpeta.

Bash

```
mkdir build
cd build
../configure --prefix=/usr --disable-werror
make -j$(nproc)
```

## 4. El peligro de instalarlo

Si haces `make install` en tu entorno de rescate, podrías romper el USB de rescate. Si quieres instalarlo en tu disco de Arch montado:

Bash

```
make install DESTDIR=/mnt
```

---

## Por qué esto no va a funcionar como esperas

Incluso si compilas Glibc 2.41, el problema es que el **kernel** del sistema de rescate o el **loader** actual podrían no estar preparados para manejar este nuevo esquema de símbolos si hay una discrepancia en las binutils.

## Una alternativa mucho más rápida y segura:

En lugar de compilar Glibc (que tarda una eternidad y es propenso a errores), **usa el sistema de archivos de Arch para rescatar a Arch**.

Como tienes los archivos en `/mnt`, puedes forzar al sistema a usar el cargador de librerías de Arch (que sí entiende ese tag) para ejecutar el `pacman` de Arch:

Bash

```
# Ejecutamos el pacman de Arch usando su propio cargador de librerías para que no falle
/mnt/lib64/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/pacman --root /mnt -Syu
```

**¿Por qué esto es mejor?**

1. Usas el binario de `ld-linux` que **ya tiene** soporte para `DT_X86_64_PLT`.
2. No arriesgas la estabilidad de tu entorno de rescate.
3. Actualizas la librería directamente desde los repositorios oficiales de Arch.

**¿Qué pasa si intentas el comando del punto anterior?** Si ese comando corre, habrás saltado el muro de la Glibc sin compilar una sola línea de código. Podríamos intentar lo mismo para `pacman-key`.

> **You asked:**

Has dicho

¿Por qué no funciona el enlace simbólico de usr lib?

> **Gemini Replied:**

## Gemini ha dicho

El enlace simbólico de `/usr/lib` (o los que estás intentando crear dentro) no está funcionando por una razón técnica fundamental: **la discrepancia de la ABI (Interfaz Binaria de Aplicaciones)** entre el cargador de programas y la librería.

Aquí te explico los tres motivos por los cuales tus intentos de "linkear" o copiar librerías de Arch hacia el entorno de rescate están fallando:

## 1. El conflicto del "Loader" (Cargador dinámico)

Cuando ejecutas un programa, el kernel no lee la librería directamente. Primero ejecuta el **Dynamic Linker** (normalmente `/lib64/ld-linux-x86-64.so.2`).

- Tu sistema de rescate tiene un `ld-linux` antiguo.
- Tus librerías en `/mnt/usr/lib` (Arch) fueron compiladas con una versión de `binutils` y `glibc` que utiliza el tag `DT_X86_64_PLT`.
- Cuando el `ld-linux` antiguo del rescate intenta leer la librería de Arch, ve ese tag, no sabe qué es y se bloquea con el error de `GLIBC_ABI_DT_X86_64_PLT not found`.

## 2. El problema del "Chroot" incompleto

En tu primer intento usaste `arch-chroot /mnt`. Para que un enlace simbólico en `/usr/lib` funcione dentro de un entorno así, el sistema necesita que **todos** los puntos de montaje virtuales estén listos (`/proc`, `/sys`, `/dev`). Como recibiste el error `mount: /mnt/proc: mount point does not exist`, el entorno quedó en un estado "híbrido": el binario intentaba ejecutarse pero buscaba sus dependencias en las rutas del sistema de rescate en lugar de las de Arch, o viceversa.

## 3. Las rutas relativas en el log de `strace`

En el archivo `pacman.key.log` que subiste, se ve claramente: `openat(AT_FDCWD, "/mnt/usr/lib/libtinfo.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT` El sistema está buscando librerías en `/mnt/usr/lib/`, pero los binarios compilados suelen tener rutas "hardcodeadas" o configuradas en el `ld.so.cache`. Si el enlace simbólico no apunta exactamente a donde el binario espera, o si el binario tiene un `RPATH` que apunta a la raíz real `/`, al estar en un entorno de rescate fuera de la jaula (`chroot`), se pierde.

---

## ¿Cómo forzar que funcione sin romper nada?

Si quieres que los binarios de tu Arch usen sus propias librerías sin que el sistema de rescate interfiera, tienes que invocar al cargador de Arch **directamente**, saltándote los enlaces simbólicos del sistema de rescate:

Bash

```
# Ejemplo: Ejecutar el GPG de Arch con la librería de Arch, usando el loader de Arch
/mnt/lib64/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/gpg --version
```

## Por qué no deberías seguir haciendo `ln -s` entre el USB y el disco:

Si logras crear un enlace simbólico que funcione y "fuerces" a una librería vieja a actuar como una nueva (o viceversa), lo más probable es que obtengas un **Segmentation Fault** (Fallo de segmentación). Las direcciones de memoria de los símbolos han cambiado en las versiones 2.41/2.42 de `glibc`.

**Mi recomendación:** Deja de intentar enlazar `/usr/lib`. La forma más limpia de que el enlace "funcione" es que el sistema sea consistente. Usa el `pacman-static` (que no usa enlaces simbólicos, tiene todo dentro) para reinstalar `glibc` en `/mnt`:

Bash

```
./pacman-static --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S glibc libassuan gpgme --overw
```

Esto pondrá los archivos reales en su sitio y eliminará la necesidad de enlaces manuales.

> **You asked:**

Has dicho

a - s_truncf-c.os

gcc  -shared -static-libgcc -Wl,-O1 -Wl,-z,mark-plt -Wl,-z,defs -Wl,-dynamic-linker=/usr/local/lib/ld-linux-x86-64.so.2 -Wl

,-z,pack-relative-relocs  -B/arch_repair/glibc/build/csu/  -Wl,--version-script=/arch_repair/glibc/build/libm.map -Wl,-sonam

e=libm.so.6 -Wl,-z,relro  -L/arch_repair/glibc/build -L/arch_repair/glibc/build/math -L/arch_repair/glibc/build/elf -L/arch

_repair/glibc/build/dlfcn -L/arch_repair/glibc/build/nss -L/arch_repair/glibc/build/nis -L/arch_repair/glibc/build/rt -L/arc

h_repair/glibc/build/resolv -L/arch_repair/glibc/build/mathvec -L/arch_repair/glibc/build/support -L/arch_repair/glibc/build

/nptl -Wl,-rpath-link=/arch_repair/glibc/build:/arch_repair/glibc/build/math:/arch_repair/glibc/build/elf:/arch_repair/glibc

/build/dlfcn:/arch_repair/glibc/build/nss:/arch_repair/glibc/build/nis:/arch_repair/glibc/build/rt:/arch_repair/glibc/build/

resolv:/arch_repair/glibc/build/mathvec:/arch_repair/glibc/build/support:/arch_repair/glibc/build/nptl -o /arch_repair/glibc

/build/math/libm.so /arch_repair/glibc/build/csu/abi-note.o -Wl,--whole-archive /arch_repair/glibc/build/math/libm_pic.a -Wl

,--no-whole-archive  -Wl,--start-group /arch_repair/glibc/build/libc.so /arch_repair/glibc/build/libc_nonshared.a -Wl,--as-

needed /arch_repair/glibc/build/elf/ld.so -Wl,--no-as-needed -Wl,--end-group

rm -f /arch_repair/glibc/build/math/libm.so.6.new

/bin/sh ../scripts/rellns-sh /arch_repair/glibc/build/math/libm.so /arch_repair/glibc/build/math/libm.so.6.new

mv -f /arch_repair/glibc/build/math/libm.so.6.new /arch_repair/glibc/build/math/libm.so.6

make[2]: *** No rule to make target '/arch_repair/glibc/build/mathvec/libmvec.so.1', needed by '/usr/local/lib/libm.so'.  St

op.

make[2]: Leaving directory '/arch_repair/glibc/math'

make[1]: *** [Makefile:484: math/subdir_install] Error 2

make[1]: Leaving directory '/arch_repair/glibc'

make: *** [Makefile:36: install] Error 2

**Gemini Replied:**

## Gemini ha dicho

El error `No rule to make target '/arch_repair/glibc/build/mathvec/libmvec.so.1'` es un clásico cuando se intenta instalar Glibc manualmente. Ocurre porque el sistema de compilación de Glibc es muy estricto: estás intentando hacer un `install` pero el Makefile detecta que falta un componente de vectorización ( `libmvec` ) que aún no se ha terminado de linkear o que el prefijo de instalación está chocando con rutas del sistema.

Sin embargo, **detente un segundo**. Compilar Glibc es una tarea extremadamente peligrosa, especialmente si el `prefix` es `/usr` o si el linker apunta a `/usr/local`. Si instalas esta Glibc incompleta, podrías dejar el sistema de rescate totalmente inútil (Kernel Panic al primer comando).

Para solucionar el error de compilación y lograr tu objetivo sin romper el USB de rescate:

## 1. El arreglo del Makefile

El error suele deberse a que intentaste instalar antes de que la compilación de todos los subdirectorios terminara. Ejecuta esto para asegurarte de que todo esté construido antes de instalar:

Bash

```
make -j$(nproc)  # Asegúrate de que termine sin errores antes de hacer el install
```

## 2. Por qué no funciona el enlace de /usr/lib

Tu duda anterior sobre el enlace simbólico de `/usr/lib` es la clave de todo. El motivo por el cual los enlaces no funcionan es el **RTLD (Runtime Dynamic Linker)**.

Cuando un binario tiene el tag `DT_X86_64_PLT`, el cargador de librerías busca una estructura específica en la memoria de la librería. Si haces un enlace simbólico de una librería vieja a una ruta donde se espera una nueva (o viceversa), el cargador encuentra un "mismatch" y aborta. No es un problema de dónde está el archivo, sino de **qué hay dentro del archivo ELF**.

## 3. La solución "Limpia" (Evita el `make install`)

En lugar de compilar e instalar Glibc (que te llevará horas y puede fallar), puedes **extraer** la Glibc de Arch directamente sobre tu disco dañado. Esto es mucho más seguro:

Bash

```
# Usa tu pacman-static para bajar e instalar la glibc oficial de Arch en /mnt
./pacman-static --root /mnt -S glibc --overwrite '*'
```

## 4. Si insistes en instalar tu compilación

Si decides seguir con tu compilación manual, para evitar el error de `libmvec`, asegúrate de que el `configure` incluya el soporte para vectores:

Bash

```
# Dentro de /arch_repair/glibc/build
../configure --prefix=/usr --libdir=/usr/lib --libexecdir=/usr/lib --enable-stack-protector=stron
make
make install DESTDIR=/mnt  # NUNCA instales directamente en / sin DESTDIR
```

**Mi consejo técnico:** El error de `readelf` que viste antes (`GLIBC_ABI_DT_X86_64_PLT`) se resuelve actualizando el paquete `glibc` y `binutils` de Arch. Compilar Glibc a mano suele introducir más

problemas de los que arregla debido a las dependencias circulares con `gcc`.

¿Qué versión de Glibc estás intentando compilar exactamente? Si es la 2.41, asegúrate de que `binutils` en tu entorno de rescate sea 2.42+, de lo contrario, el soporte para `mark-plt` (que se ve en tu comando de GCC) fallará siempre.

> **You asked:**

Has dicho

¿A dónde quedó el ABI?

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO8859-8.so

-rwxr-xr-x 1 root root  42K Mar 16 05:00 ISO8859-9.so

-rwxr-xr-x 1 root root  42K Mar 16 05:00 ISO8859-9E.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO_10367-BOX.so

-rwxr-xr-x 1 root root  41K Mar 16 05:00 ISO_11548-1.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO_2033.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO_5427-EXT.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO_5427.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 ISO_5428.so

-rwxr-xr-x 1 root root  61K Mar 16 05:00 ISO_6937-2.so

-rwxr-xr-x 1 root root  62K Mar 16 05:00 ISO_6937.so

-rwxr-xr-x 1 root root  73K Mar 16 05:00 JOHAB.so

-rwxr-xr-x 1 root root  43K Mar 16 05:00 KOI-8.so

-rwxr-xr-x 1 root root  42K Mar 16 05:00 KOI8-R.so

-rwxr-xr-x 1 root root  42K Mar 16 05:00 KOI8-RU.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 KOI8-T.so

-rwxr-xr-x 1 root root  42K Mar 16 05:00 KOI8-U.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 LATIN-GREEK-1.so

-rwxr-xr-x 1 root root  44K Mar 16 05:00 LATIN-GREEK.so

```
-rwxr-xr-x 1 root root  42K Mar 16 05:00 MAC-CENTRALEUROPE.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 MAC-IS.so
-rwxr-xr-x 1 root root  42K Mar 16 05:00 MAC-SAMI.so
-rwxr-xr-x 1 root root  42K Mar 16 05:00 MAC-UK.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 MACINTOSH.so
-rwxr-xr-x 1 root root  42K Mar 16 05:00 MIK.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 NATS-DANO.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 NATS-SEFI.so
-rwxr-xr-x 1 root root  42K Mar 16 05:00 PT154.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 RK1048.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 SAMI-WS2.so
-rwxr-xr-x 1 root root  71K Mar 16 05:00 SHIFT_JISX0213.so
-rwxr-xr-x 1 root root 132K Mar 16 05:00 SJIS.so
-rwxr-xr-x 1 root root  58K Mar 16 05:00 T.61.so
-rwxr-xr-x 1 root root  52K Mar 16 05:00 TCVN5712-1.so
-rwxr-xr-x 1 root root  44K Mar 16 05:00 TIS-620.so
-rwxr-xr-x 1 root root  85K Mar 16 05:00 TSCII.so
-rwxr-xr-x 1 root root 119K Mar 16 05:00 UHC.so
-rwxr-xr-x 1 root root  49K Mar 16 05:00 UNICODE.so
-rwxr-xr-x 1 root root  61K Mar 16 05:00 UTF-16.so
-rwxr-xr-x 1 root root  50K Mar 16 05:00 UTF-32.so
-rwxr-xr-x 1 root root  87K Mar 16 05:00 UTF-7.so
-rwxr-xr-x 1 root root  42K Mar 16 05:00 VISCII.so
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 gconv-modules
drwxr-xr-x 1 root root   48 Mar 16 05:00 gconv-modules.d
-rwxr-xr-x 1 root root 470K Mar 16 05:00 libCNS.so
-rwxr-xr-x 1 root root  76K Mar 16 05:00 libGB.so
```

```
-rwxr-xr-x 1 root root  67K Mar 16 05:00 libISOIR165.so
-rwxr-xr-x 1 root root 113K Mar 16 05:00 libJIS.so
-rwxr-xr-x 1 root root 131K Mar 16 05:00 libJISX0213.so
-rwxr-xr-x 1 root root  55K Mar 16 05:00 libKSC.so

/otro/libc_compilado/usr/lib/gconv/gconv-modules.d:
total 56K
-rw-r--r-- 1 root root 53K Mar 16 05:00 gconv-modules-extra.conf

/otro/libc_compilado/usr/lib/getconf:
total 156K
-rwxr-xr-x 4 root root 50K Mar 16 05:00 POSIX_V6_LP64_OFF64
-rwxr-xr-x 4 root root 50K Mar 16 05:00 POSIX_V7_LP64_OFF64
-rwxr-xr-x 4 root root 50K Mar 16 05:00 XBS5_LP64_OFF64

/otro/libc_compilado/usr/sbin:
total 856K
-rwxr-xr-x 1 root root  96K Mar 16 05:00 iconvconfig
-rwxr-xr-x 1 root root 563K Mar 16 05:00 nscd
-rwxr-xr-x 1 root root 194K Mar 16 05:00 zic

/otro/libc_compilado/usr/share:
total 0
drwxr-xr-x 1 root root  30 Mar 16 05:00 i18n
drwxr-xr-x 1 root root 204 Mar 16 05:00 locale

/otro/libc_compilado/usr/share/i18n:
```

```
total 0
drwxr-xr-x 1 root root 5.4K Mar 16 05:00 charmaps
drwxr-xr-x 1 root root 4.5K Mar 16 05:00 locales


/otro/libc_compilado/usr/share/i18n/charmaps:
total 3.4M
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ANSI_X3.110-1983.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ANSI_X3.4-1968.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 ARMSCII-8.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ASMO_449.gz
-rw-r--r-- 1 root root 108K Mar 16 05:00 BIG5-HKSCS.gz
-rw-r--r-- 1 root root  76K Mar 16 05:00 BIG5.gz
-rw-r--r-- 1 root root 1.2K Mar 16 05:00 BRF.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 BS_4730.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 BS_VIEWDATA.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP10007.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP1125.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP1250.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP1251.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 CP1252.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP1253.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP1254.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP1255.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 CP1256.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP1257.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP1258.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 CP737.gz
```

```
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP770.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 CP771.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP772.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 CP773.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 CP774.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 CP775.gz
-rw-r--r-- 1 root root 127K Mar 16 05:00 CP949.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 CSA_Z243.4-1985-1.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 CSA_Z243.4-1985-2.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 CSA_Z243.4-1985-GR.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 CSN_369103.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 CWI.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 DEC-MCS.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 DIN_66003.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 DS_2089.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-AT-DE-A.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-AT-DE.gz
-rw-r--r-- 1 root root 2.3K Mar 16 05:00 EBCDIC-CA-FR.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-DK-NO-A.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-DK-NO.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-ES-A.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-ES-S.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-ES.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-FI-SE-A.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-FI-SE.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-FR.gz
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 EBCDIC-IS-FRISS.gz
```

```
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-IT.gz
-rw-r--r-- 1 root root 2.0K Mar 16 05:00 EBCDIC-PT.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-UK.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 EBCDIC-US.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ECMA-CYRILLIC.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ES.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ES2.gz
-rw-r--r-- 1 root root  69K Mar 16 05:00 EUC-JISX0213.gz
-rw-r--r-- 1 root root  85K Mar 16 05:00 EUC-JP-MS.gz
-rw-r--r-- 1 root root  73K Mar 16 05:00 EUC-JP.gz
-rw-r--r-- 1 root root  58K Mar 16 05:00 EUC-KR.gz
-rw-r--r-- 1 root root 341K Mar 16 05:00 EUC-TW.gz
-rw-r--r-- 1 root root 631K Mar 16 05:00 GB18030.gz
-rw-r--r-- 1 root root  44K Mar 16 05:00 GB2312.gz
-rw-r--r-- 1 root root 117K Mar 16 05:00 GBK.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 GB_1988-80.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 GEORGIAN-ACADEMY.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 GEORGIAN-PS.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 GOST_19768-74.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 GREEK-CCITT.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 GREEK7-OLD.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 GREEK7.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 HP-GREEK8.gz
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 HP-ROMAN8.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 HP-ROMAN9.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 HP-THAI8.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 HP-TURKISH8.gz
```

```
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM037.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 IBM038.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM1004.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM1026.gz
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 IBM1047.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 IBM1124.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 IBM1129.gz
-rw-r--r-- 1 root root 2.0K Mar 16 05:00 IBM1132.gz
-rw-r--r-- 1 root root 2.0K Mar 16 05:00 IBM1133.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 IBM1160.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 IBM1161.gz
-rw-r--r-- 1 root root 2.3K Mar 16 05:00 IBM1162.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 IBM1163.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 IBM1164.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM256.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM273.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 IBM274.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 IBM275.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM277.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM278.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM280.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 IBM281.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM284.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM285.gz
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 IBM290.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM297.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM420.gz
```

```
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM423.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM424.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM437.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM500.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM850.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM851.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM852.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM855.gz
-rw-r--r-- 1 root root 2.2K Mar 16 05:00 IBM856.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM857.gz
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 IBM858.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM860.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM861.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM862.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM863.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM864.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM865.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 IBM866.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM866NAV.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 IBM868.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM869.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM870.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 IBM871.gz
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 IBM874.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM875.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 IBM880.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 IBM891.gz
```

```
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 IBM903.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 IBM904.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 IBM905.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 IBM918.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 IBM922.gz
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 IEC_P27-1.gz
-rw-r--r-- 1 root root 1.1K Mar 16 05:00 INIS-8.gz
-rw-r--r-- 1 root root 1.5K Mar 16 05:00 INIS-CYRILLIC.gz
-rw-r--r-- 1 root root 1.4K Mar 16 05:00 INIS.gz
-rw-r--r-- 1 root root 1.4K Mar 16 05:00 INVARIANT.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISIRI-3342.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 ISO-8859-1.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-10.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-11.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-13.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 ISO-8859-14.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 ISO-8859-15.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-16.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-2.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-3.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 ISO-8859-4.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-5.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 ISO-8859-6.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 ISO-8859-7.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 ISO-8859-8.gz
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 ISO-8859-9.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO-8859-9E.gz
```

```
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 ISO-IR-197.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 ISO-IR-209.gz
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ISO-IR-90.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 ISO_10367-BOX.gz
-rw-r--r-- 1 root root  19K Mar 16 05:00 ISO_10646.gz
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 ISO_11548-1.gz
-rw-r--r-- 1 root root  840 Mar 16 05:00 ISO_2033-1983.gz
-rw-r--r-- 1 root root 1.1K Mar 16 05:00 ISO_5427-EXT.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ISO_5427.gz
-rw-r--r-- 1 root root 1.5K Mar 16 05:00 ISO_5428.gz
-rw-r--r-- 1 root root  926 Mar 16 05:00 ISO_646.BASIC.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 ISO_646.IRV.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 ISO_6937-2-25.gz
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ISO_6937-2-ADD.gz
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ISO_6937.gz
-rw-r--r-- 1 root root 4.2K Mar 16 05:00 ISO_8859-1,GL.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ISO_8859-SUPP.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 IT.gz
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 JIS_C6220-1969-JP.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 JIS_C6220-1969-RO.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 JIS_C6229-1984-A.gz
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 JIS_C6229-1984-B-ADD.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 JIS_C6229-1984-B.gz
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 JIS_C6229-1984-HAND-ADD.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 JIS_C6229-1984-HAND.gz
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 JIS_C6229-1984-KANA.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 JIS_X0201.gz
```

```
-rw-r--r-- 1 root root 122K Mar 16 05:00 JOHAB.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 JUS_I.B1.002.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 JUS_I.B1.003-MAC.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 JUS_I.B1.003-SERB.gz
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 KOI-8.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 KOI8-R.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 KOI8-RU.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 KOI8-T.gz
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 KOI8-U.gz
-rw-r--r-- 1 root root 1.5K Mar 16 05:00 KSC5636.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 LATIN-GREEK-1.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 LATIN-GREEK.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 MAC-CENTRALEUROPE.gz
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 MAC-CYRILLIC.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 MAC-IS.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 MAC-SAMI.gz
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 MAC-UK.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 MACINTOSH.gz
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 MIK.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 MSZ_7795.3.gz
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 NATS-DANO-ADD.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NATS-DANO.gz
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 NATS-SEFI-ADD.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NATS-SEFI.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NC_NC00-10.gz
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 NEXTSTEP.gz
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NF_Z_62-010.gz
```

```
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NF_Z_62-010_1973.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NS_4551-1.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 NS_4551-2.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 PT.gz

-rw-r--r-- 1 root root 2.8K Mar 16 05:00 PT154.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 PT2.gz

-rw-r--r-- 1 root root 2.8K Mar 16 05:00 RK1048.gz

-rw-r--r-- 1 root root 3.0K Mar 16 05:00 SAMI-WS2.gz

-rw-r--r-- 1 root root 2.4K Mar 16 05:00 SAMI.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 SEN_850200_B.gz

-rw-r--r-- 1 root root 1.6K Mar 16 05:00 SEN_850200_C.gz

-rw-r--r-- 1 root root  42K Mar 16 05:00 SHIFT_JIS.gz

-rw-r--r-- 1 root root  69K Mar 16 05:00 SHIFT_JISX0213.gz

-rw-r--r-- 1 root root 4.7K Mar 16 05:00 T.101-G2.gz

-rw-r--r-- 1 root root 1.5K Mar 16 05:00 T.61-7BIT.gz

-rw-r--r-- 1 root root 4.3K Mar 16 05:00 T.61-8BIT.gz

-rw-r--r-- 1 root root 3.1K Mar 16 05:00 TCVN5712-1.gz

-rw-r--r-- 1 root root 2.5K Mar 16 05:00 TIS-620.gz

-rw-r--r-- 1 root root 3.5K Mar 16 05:00 TSCII.gz

-rw-r--r-- 1 root root 478K Mar 16 05:00 UTF-8.gz

-rw-r--r-- 1 root root 4.5K Mar 16 05:00 VIDEOTEX-SUPPL.gz

-rw-r--r-- 1 root root 2.6K Mar 16 05:00 VISCII.gz

-rw-r--r-- 1 root root  58K Mar 16 05:00 WINDOWS-31J.gz


/otro/libc_compilado/usr/share/i18n/locales:

total 13M

-rw-r--r-- 1 root root 5.4K Mar 16 05:00 C
```

```
-rw-r--r-- 1 root root 8.6K Mar 16 05:00 POSIX
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 aa_DJ
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 aa_ER
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 aa_ET
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ab_GE
-rw-r--r-- 1 root root 6.8K Mar 16 05:00 af_ZA
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 agr_PE
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 ak_GH
-rw-r--r-- 1 root root  30K Mar 16 05:00 am_ET
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 an_ES
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 anp_IN
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 ar_AE
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ar_BH
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_DZ
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_EG
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 ar_IN
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_IQ
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ar_JO
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ar_KW
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ar_LB
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ar_LY
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_MA
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ar_OM
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ar_QA
-rw-r--r-- 1 root root 6.8K Mar 16 05:00 ar_SA
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_SD
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_SS
```

```
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ar_SY
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 ar_TN
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 ar_YE
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 as_IN
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 ast_ES
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 ayc_PE
-rw-r--r-- 1 root root 5.8K Mar 16 05:00 az_AZ
-rw-r--r-- 1 root root 7.3K Mar 16 05:00 az_IR
-rw-r--r-- 1 root root 5.4K Mar 16 05:00 be_BY
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 be_BY@latin
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 bem_ZM
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 ber_DZ
-rw-r--r-- 1 root root 5.1K Mar 16 05:00 ber_MA
-rw-r--r-- 1 root root  12K Mar 16 05:00 bg_BG
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 bhb_IN
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 bho_IN
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 bho_NP
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 bi_VU
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 bn_BD
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 bn_IN
-rw-r--r-- 1 root root 4.1K Mar 16 05:00 bo_CN
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 bo_IN
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 br_FR
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 br_FR@euro
-rw-r--r-- 1 root root 4.2K Mar 16 05:00 brx_IN
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 bs_BA
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 byn_ER
```

```
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 ca_AD
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 ca_ES
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 ca_ES@euro
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 ca_ES@valencia
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 ca_FR
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 ca_IT
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 ce_RU
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 chr_US
-rw-r--r-- 1 root root 4.9K Mar 16 05:00 ckb_IQ
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 cmn_TW
-rw-r--r-- 1 root root 4.4M Mar 16 05:00 cns11643_stroke
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 crh_RU
-rw-r--r-- 1 root root 5.0K Mar 16 05:00 crh_UA
-rw-r--r-- 1 root root  10K Mar 16 05:00 cs_CZ
-rw-r--r-- 1 root root 5.3K Mar 16 05:00 csb_PL
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 cv_RU
-rw-r--r-- 1 root root 6.6K Mar 16 05:00 cy_GB
-rw-r--r-- 1 root root 7.5K Mar 16 05:00 da_DK
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 de_AT
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 de_AT@euro
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 de_BE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 de_BE@euro
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 de_CH
-rw-r--r-- 1 root root 4.1K Mar 16 05:00 de_DE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 de_DE@euro
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 de_IT
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 de_LI
```

```
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 de_LU
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 de_LU@euro
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 doi_IN
-rw-r--r-- 1 root root 6.2K Mar 16 05:00 dsb_DE
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 dv_MV
-rw-r--r-- 1 root root 105K Mar 16 05:00 dz_BT
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 el_CY
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 el_GR
-rw-r--r-- 1 root root 1.6K Mar 16 05:00 el_GR@euro
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 en_AG
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 en_AU
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 en_BW
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 en_CA
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 en_DK
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 en_GB
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 en_HK
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 en_IE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 en_IE@euro
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 en_IL
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 en_IN
-rw-r--r-- 1 root root 5.8K Mar 16 05:00 en_NG
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 en_NZ
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 en_PH
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 en_SC
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 en_SG
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 en_US
-rw-r--r-- 1 root root 7.8K Mar 16 05:00 en_ZA
```

```
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 en_ZM
-rw-r--r-- 1 root root 2.4K Mar 16 05:00 en_ZW
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 eo
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 es_AR
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_BO
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 es_CL
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_CO
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 es_CR
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 es_CU
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 es_DO
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 es_EC
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 es_ES
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 es_ES@euro
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_GT
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_HN
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_MX
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 es_NI
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_PA
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 es_PE
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 es_PR
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_PY
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_SV
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 es_US
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_UY
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 es_VE
-rw-r--r-- 1 root root 5.1K Mar 16 05:00 et_EE
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 eu_ES
```

```
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 eu_ES@euro
-rw-r--r-- 1 root root  12K Mar 16 05:00 fa_IR
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 ff_SN
-rw-r--r-- 1 root root 8.9K Mar 16 05:00 fi_FI
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 fi_FI@euro
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 fil_PH
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 fo_FO
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 fr_BE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 fr_BE@euro
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 fr_CA
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 fr_CH
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 fr_FR
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 fr_FR@euro
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 fr_LU
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 fr_LU@euro
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 fur_IT
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 fy_DE
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 fy_NL
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 ga_IE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 ga_IE@euro
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 gbm_IN
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 gd_GB
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 gez_ER
-rw-r--r-- 1 root root  27K Mar 16 05:00 gez_ER@abegede
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 gez_ET
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 gez_ET@abegede
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 gl_ES
```

```
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 gl_ES@euro
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 gu_IN
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 gv_GB
-rw-r--r-- 1 root root 7.7K Mar 16 05:00 ha_NG
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 hak_TW
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 he_IL
-rw-r--r-- 1 root root 5.6K Mar 16 05:00 hi_IN
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 hif_FJ
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 hne_IN
-rw-r--r-- 1 root root 8.2K Mar 16 05:00 hr_HR
-rw-r--r-- 1 root root 5.7K Mar 16 05:00 hsb_DE
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 ht_HT
-rw-r--r-- 1 root root  22K Mar 16 05:00 hu_HU
-rw-r--r-- 1 root root 5.2K Mar 16 05:00 hy_AM
-rw-r--r-- 1 root root  14K Mar 16 05:00 i18n
-rw-r--r-- 1 root root 174K Mar 16 05:00 i18n_ctype
-rw-r--r-- 1 root root 2.7K Mar 16 05:00 ia_FR
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 id_ID
-rw-r--r-- 1 root root  11K Mar 16 05:00 ig_NG
-rw-r--r-- 1 root root 6.4K Mar 16 05:00 ik_CA
-rw-r--r-- 1 root root 6.5K Mar 16 05:00 is_IS
-rw-r--r-- 1 root root  649 Mar 16 05:00 iso14651_t1
-rw-r--r-- 1 root root 3.3M Mar 16 05:00 iso14651_t1_common
-rw-r--r-- 1 root root 1.1M Mar 16 05:00 iso14651_t1_pinyin
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 it_CH
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 it_IT
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 it_IT@euro
```

```
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 iu_CA
-rw-r--r-- 1 root root 214K Mar 16 05:00 ja_JP
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 ka_GE
-rw-r--r-- 1 root root 5.5K Mar 16 05:00 kab_DZ
-rw-r--r-- 1 root root 5.3K Mar 16 05:00 kk_KZ
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 kl_GL
-rw-r--r-- 1 root root  38K Mar 16 05:00 km_KH
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 kn_IN
-rw-r--r-- 1 root root  53K Mar 16 05:00 ko_KR
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 kok_IN
-rw-r--r-- 1 root root 4.0K Mar 16 05:00 ks_IN
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 ks_IN@devanagari
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 ku_TR
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 kv_RU
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 kw_GB
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 ky_KG
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 lb_LU
-rw-r--r-- 1 root root 8.0K Mar 16 05:00 lg_UG
-rw-r--r-- 1 root root 1.9K Mar 16 05:00 li_BE
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 li_NL
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 lij_IT
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 ln_CD
-rw-r--r-- 1 root root  30K Mar 16 05:00 lo_LA
-rw-r--r-- 1 root root 6.7K Mar 16 05:00 lt_LT
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 ltg_LV
-rw-r--r-- 1 root root 5.9K Mar 16 05:00 lv_LV
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 lzh_TW
```

```
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 mag_IN
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 mai_IN
-rw-r--r-- 1 root root 1.8K Mar 16 05:00 mai_NP
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 mdf_RU
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 mfe_MU
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 mg_MG
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 mhr_RU
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 mi_NZ
-rw-r--r-- 1 root root 4.2K Mar 16 05:00 miq_NI
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 mjw_IN
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 mk_MK
-rw-r--r-- 1 root root  11K Mar 16 05:00 ml_IN
-rw-r--r-- 1 root root 5.5K Mar 16 05:00 mn_MN
-rw-r--r-- 1 root root 4.1K Mar 16 05:00 mni_IN
-rw-r--r-- 1 root root 6.2K Mar 16 05:00 mnw_MM
-rw-r--r-- 1 root root 6.9K Mar 16 05:00 mr_IN
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 ms_MY
-rw-r--r-- 1 root root 7.4K Mar 16 05:00 mt_MT
-rw-r--r-- 1 root root 6.3K Mar 16 05:00 my_MM
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 nan_TW
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 nan_TW@latin
-rw-r--r-- 1 root root 7.8K Mar 16 05:00 nb_NO
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 nds_DE
-rw-r--r-- 1 root root 2.6K Mar 16 05:00 nds_NL
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 ne_NP
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 nhn_MX
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 niu_NU
```

```
-rw-r--r-- 1 root root 2.1K Mar 16 05:00 niu_NZ
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 nl_AW
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 nl_BE
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 nl_BE@euro
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 nl_NL
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 nl_NL@euro
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 nn_NO
-rw-r--r-- 1 root root 7.0K Mar 16 05:00 nr_ZA
-rw-r--r-- 1 root root 6.5K Mar 16 05:00 nso_ZA
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 oc_FR
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 om_ET
-rw-r--r-- 1 root root 8.5K Mar 16 05:00 om_KE
-rw-r--r-- 1 root root 7.8K Mar 16 05:00 or_IN
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 os_RU
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 pa_IN
-rw-r--r-- 1 root root 3.8K Mar 16 05:00 pa_PK
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 pap_AW
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 pap_CW
-rw-r--r-- 1 root root 5.8K Mar 16 05:00 pl_PL
-rw-r--r-- 1 root root 9.4K Mar 16 05:00 ps_AF
-rw-r--r-- 1 root root 3.2K Mar 16 05:00 pt_BR
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 pt_PT
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 pt_PT@euro
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 quz_PE
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 raj_IN
-rw-r--r-- 1 root root 3.6K Mar 16 05:00 rif_MA
-rw-r--r-- 1 root root  11K Mar 16 05:00 ro_RO
```

```
-rw-r--r-- 1 root root 4.5K Mar 16 05:00 ru_RU
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 ru_UA
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 rw_RW
-rw-r--r-- 1 root root 5.8K Mar 16 05:00 sa_IN
-rw-r--r-- 1 root root 8.3K Mar 16 05:00 sah_RU
-rw-r--r-- 1 root root 4.1K Mar 16 05:00 sat_IN
-rw-r--r-- 1 root root 3.1K Mar 16 05:00 sc_IT
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 scn_IT
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 sd_IN
-rw-r--r-- 1 root root 4.2K Mar 16 05:00 sd_IN@devanagari
-rw-r--r-- 1 root root  11K Mar 16 05:00 se_NO
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 sgs_LT
-rw-r--r-- 1 root root 5.9K Mar 16 05:00 shn_MM
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 shs_CA
-rw-r--r-- 1 root root 6.1K Mar 16 05:00 si_LK
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 sid_ET
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 sk_SK
-rw-r--r-- 1 root root  79K Mar 16 05:00 sl_SI
-rw-r--r-- 1 root root 3.7K Mar 16 05:00 sm_WS
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 so_DJ
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 so_ET
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 so_KE
-rw-r--r-- 1 root root 5.3K Mar 16 05:00 so_SO
-rw-r--r-- 1 root root 9.7K Mar 16 05:00 sq_AL
-rw-r--r-- 1 root root 2.5K Mar 16 05:00 sq_MK
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 sr_ME
-rw-r--r-- 1 root root 5.7K Mar 16 05:00 sr_RS
```

```
-rw-r--r-- 1 root root 3.4K Mar 16 05:00 sr_RS@latin
-rw-r--r-- 1 root root 7.1K Mar 16 05:00 ss_ZA
-rw-r--r-- 1 root root 4.3K Mar 16 05:00 ssy_ER
-rw-r--r-- 1 root root 6.8K Mar 16 05:00 st_ZA
-rw-r--r-- 1 root root 3.5K Mar 16 05:00 su_ID
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 sv_FI
-rw-r--r-- 1 root root 1.7K Mar 16 05:00 sv_FI@euro
-rw-r--r-- 1 root root 7.6K Mar 16 05:00 sv_SE
-rw-r--r-- 1 root root 2.8K Mar 16 05:00 sw_KE
-rw-r--r-- 1 root root 2.9K Mar 16 05:00 sw_TZ
-rw-r--r-- 1 root root 5.0K Mar 16 05:00 syr
-rw-r--r-- 1 root root 5.0K Mar 16 05:00 szl_PL
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ta_IN
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 ta_LK
-rw-r--r-- 1 root root 4.4K Mar 16 05:00 tcy_IN
-rw-r--r-- 1 root root 4.8K Mar 16 05:00 te_IN
-rw-r--r-- 1 root root 6.1K Mar 16 05:00 tg_TJ
-rw-r--r-- 1 root root 8.9K Mar 16 05:00 th_TH
-rw-r--r-- 1 root root 4.2K Mar 16 05:00 the_NP
-rw-r--r-- 1 root root 5.3K Mar 16 05:00 ti_ER
-rw-r--r-- 1 root root 4.7K Mar 16 05:00 ti_ET
-rw-r--r-- 1 root root 4.6K Mar 16 05:00 tig_ER
-rw-r--r-- 1 root root  12K Mar 16 05:00 tk_TM
-rw-r--r-- 1 root root 3.0K Mar 16 05:00 tl_PH
-rw-r--r-- 1 root root 7.0K Mar 16 05:00 tn_ZA
-rw-r--r-- 1 root root 3.9K Mar 16 05:00 to_TO
-rw-r--r-- 1 root root 3.3K Mar 16 05:00 tok
```

-rw-r--r-- 1 root root 4.0K Mar 16 05:00 tpi_PG

-rw-r--r-- 1 root root 2.0K Mar 16 05:00 tr_CY

-rw-r--r-- 1 root root 179K Mar 16 05:00 tr_TR

-rw-r--r-- 1 root root  15K Mar 16 05:00 translit_circle

-rw-r--r-- 1 root root  68K Mar 16 05:00 translit_cjk_compat

-rw-r--r-- 1 root root 160K Mar 16 05:00 translit_cjk_variants

-rw-

**Gemini Replied:**

## Gemini ha dicho

El ABI (Application Binary Interface) no es un archivo que vayas a encontrar listado con `ls`. El ABI es la **estructura interna** de los binarios que acabas de compilar.

En tu caso, el "nuevo" ABI que estás buscando (el que contiene el tag `DT_X86_64_PLT`) está "escondido" dentro de los archivos `.so` (Shared Objects) que listaste, específicamente en **`libc.so.6`** y el cargador dinámico.

Para ver dónde quedó ese ABI, tienes que mirar "dentro" de los archivos usando herramientas de análisis de ELF. Ejecuta esto sobre tu nueva compilación:

## 1. ¿Cómo verificar si tu compilación tiene el nuevo ABI?

Usa `readelf` sobre la `libc.so.6` que acabas de compilar en `/otro/libc_compilado/`:

Bash

```
readelf -d /otro/libc_compilado/usr/lib/libc.so.6 | grep PLT
```

Si la compilación fue exitosa y usaste una versión reciente (Glibc 2.41+), deberías ver algo como: `0x0000000070000000 (DT_X86_64_PLT) ...`

## 2. El problema de tu instalación actual

Viendo tu listado de archivos, hay un detalle crítico: has instalado la Glibc en `/otro/libc_compilado/usr/lib`.

- **El Loader:** Los binarios que intentas ejecutar (como `gpg` de Arch) tienen "hardcodeado" que deben buscar el cargador en `/lib64/ld-linux-x86-64.so.2`.

- **El Conflicto:** Aunque hayas compilado el nuevo ABI, si no reemplazas el cargador de la raíz del sistema (o de `/mnt`), los programas seguirán usando el cargador viejo que no entiende el nuevo ABI.

## 3. ¿Por qué ves tantos archivos `.so` de ISO y MAC?

Lo que pusiste en el log son los módulos de **gconv**. Estos no definen el ABI del sistema; son solo librerías para que `iconv` pueda convertir entre distintos sets de caracteres (como de UTF-8 a ISO-8859). El corazón del ABI está en:

- `/otro/libc_compilado/usr/lib/libc.so.6`
- `/otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2` (Este es el más importante).

## ¿Qué hacer ahora?

Si ya verificaste con `readelf` que tu `libc.so.6` compilada tiene el tag `DT_X86_64_PLT`, para que el sistema de Arch vuelva a la vida tienes que mover esos archivos a su lugar real en el disco de Arch:

Bash

```
# Sincronizar tu compilación al disco de Arch (asumiendo que Arch es /mnt)
cp -d /otro/libc_compilado/usr/lib/libc.so.6 /mnt/usr/lib/
cp -d /otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2 /mnt/usr/lib/
```

**Atención:** Hacer esto a mano es como operar el corazón con un cuchillo de cocina. Si el enlace simbólico de `libc.so.6` se rompe por un milisegundo, cualquier comando que tires (incluido `ls` o `cp`) te dará un **Kernel Panic** o **Segmentation Fault**. Por eso te recomendaba usar `pacman-static`, que ya trae el ABI correcto y lo instala de forma atómica.

> **You asked:**

Has dicho

 readelf -d /otro/libc_compilado/usr/lib/libc.so | grep PLT

readelf: Error: Not an ELF file - it has the wrong magic bytes at the start

> **Gemini Replied:**

## Gemini ha dicho

Ese error ocurre porque `/otro/libc_compilado/usr/lib/libc.so` (sin el `.6`) suele ser un **script de linker** (un archivo de texto) y no el binario real.

Si abres ese archivo con `cat`, verás algo como `GROUP ( /usr/lib/libc.so.6 ... )`. Por eso `readelf` te dice que no tiene los "magic bytes" de un ELF.

Para encontrar el ABI, tienes que apuntar al archivo binario real, que es el que tiene el número de versión. Prueba con este comando:

## 1. Verifica el binario real

Bash

```
readelf -d /otro/libc_compilado/usr/lib/libc.so.6 | grep PLT
```

Si ese comando te devuelve `(DT_X86_64_PLT)`, entonces **felicidades**: has logrado compilar la extensión de la ABI que tu sistema Arch te está exigiendo.

## 2. ¿Por qué no lo ves en tu lista de archivos anterior?

En el listado que pasaste, se veían cientos de archivos `.so` dentro de `gconv/`, pero esos son solo traductores de juegos de caracteres. El "corazón" de la Glibc está en la carpeta superior:

- **libc.so.6**: La librería principal.
- **ld-linux-x86-64.so.2**: El cargador dinámico (este es el que realmente "entiende" el ABI al arrancar un programa).

## 3. ¿Cómo moverlo a su lugar sin romper nada?

Como estás en un entorno de rescate, si intentas copiar estos archivos a `/usr/lib` de tu sistema Arch (en `/mnt`), podrías romper los enlaces simbólicos. La forma correcta de "instalar" ese ABI que acabas de compilar en tu disco de Arch es:

Bash

```
# Primero, asegúrate de que el cargador también tenga el ABI
readelf -d /otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2 | grep PLT

# Si todo está bien, cópialos con preservación de enlaces simbólicos (-d)
cp -d /otro/libc_compilado/usr/lib/libc.so.6 /mnt/usr/lib/
cp -d /otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2 /mnt/usr/lib/
```

## 4. El "veredicto" de tu compilación

Si el comando `readelf` sobre `libc.so.6` **no** muestra nada de PLT, significa que aunque compilaste Glibc, el `gcc` o las `binutils` de tu entorno de rescate son demasiado viejos para

generar ese tag.

**¿Qué dice el comando de `readelf` sobre el archivo `.so.6`?** Eso nos dirá si el esfuerzo de la compilación valió la pena o si el compilador te ignoró.

> **You asked:**

Has dicho

 root /arch_repair/glibc/build 

 readelf -d /otro/libc_compilado/usr/lib/libc.so.6 | grep PLT

readelf: Error: '/otro/libc_compilado/usr/lib/libc.so.6': No such file

 root /arch_repair/glibc/build 

 readelf -d /otro/libc_compilado/usr/lib/libc.so | grep PLT

readelf: Error: Not an ELF file - it has the wrong magic bytes at the start

 root /arch_repair/glibc/build 

 # Primero, asegúrate de que el cargador también tenga el ABI

readelf -d /otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2 | grep PLT

# Si todo está bien, cópialos con preservación de enlaces simbólicos (-d)

cp -d /otro/libc_compilado/usr/lib/libc.so.6 /mnt/usr/lib/

cp -d /otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2 /mnt/usr/lib/

readelf: Error: '/otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2': No such file

cp: cannot stat '/otro/libc_compilado/usr/lib/libc.so.6': No such file or directory

cp: cannot stat '/otro/libc_compilado/usr/lib/ld-linux-x86-64.so.2': No such file or directory

> **Gemini Replied:**

## Gemini ha dicho

El problema es que los archivos no están en `/usr/lib` dentro de tu carpeta de compilación. En una compilación manual de Glibc, los archivos principales (`libc.so.6` y el cargador) se instalan por defecto en **`/lib`** (o `/lib64`), no en `/usr/lib`, a menos que lo hayas forzado específicamente en el `configure`.

Por el listado que pasaste antes, se ve que tu estructura es `/otro/libc_compilado/...`. Vamos a buscarlos donde realmente están:

## 1. Localiza los archivos correctos

Prueba con estos comandos para encontrar la ubicación exacta de los binarios:

Bash

```
find /otro/libc_compilado -name "libc.so.6"
find /otro/libc_compilado -name "ld-linux-x86-64.so.2"
```

Probablemente estén en `/otro/libc_compilado/lib/` o directamente en la raíz de la carpeta de compilación si no terminó el `install`.

## 2. Una vez localizados, verifica el ABI

Si el `find` te dice que están en `/otro/libc_compilado/lib/`, ejecuta:

Bash

```
readelf -d /otro/libc_compilado/lib/libc.so.6 | grep PLT
```

## 3. El misterio de por qué no estaban en `/usr/lib`

En el mundo de Glibc, `/usr/lib` se usa para librerías auxiliares y scripts de linker (como el `libc.so` que te dio error de "magic bytes"). Las librerías críticas del sistema que definen la **ABI** siempre intentan instalarse en `/lib` para estar disponibles antes de que se monte `/usr`.

## 4. Qué hacer si no aparecen

Si los comandos `find` no devuelven nada, es que el `make install` que intentaste antes falló antes de copiar los archivos más importantes. En ese caso, los archivos siguen dentro de tu carpeta de construcción:

- La `libc.so.6` real debería estar en: `/arch_repair/glibc/build/libc.so`
- El loader real debería estar en: `/arch_repair/glibc/build/elf/ld.so`

Intenta el `readelf` sobre esos archivos directamente:

Bash

```
readelf -d /arch_repair/glibc/build/libc.so | grep PLT
```

**Si ahí tampoco aparece `DT_X86_64_PLT`**, significa que tu compilador actual (el del USB de rescate) no tiene habilitada la capacidad de generar ese tag, y no importa cuántas veces compiles Glibc, no obtendrás el ABI que Arch te pide. En ese caso, la única salida es usar el cargador que ya existe en `/mnt` (el de Arch) mediante el truco de la `--library-path` que mencionamos antes.

> **You asked:**

Has dicho

 readelf -d /otro/libc_compilado/lib/libc.so.6 | grep PLT

readelf: Error: '/otro/libc_compilado/lib/libc.so.6': No such file

 root /arch_repair/glibc/build 

 readelf -d /arch_repair/glibc/build/libc.so | grep PLT

0x0000000070000000 (DT_X86_64_PLT)  0x24000

0x0000000070000001 (DT_X86_64_PLTSZ)  0x3a0

0x0000000070000003 (DT_X86_64_PLTENT)  0x10

0x0000000000000003 (PLTGOT)  0x1e1fe8

0x0000000000000002 (PLTRELSZ)  1368 (bytes)

0x0000000000000014 (PLTREL)  RELA

> **Gemini Replied:**

## Gemini ha dicho

¡Excelente! **Lo lograste.** Ese `0x0000000070000000 (DT_X86_64_PLT)` es exactamente lo que el sistema Arch necesita. Has compilado con éxito una Glibc que soporta la nueva extensión de la ABI de x86-64.

Ahora que tienes el "corazón" del sistema listo en `/arch_repair/glibc/build/libc.so`, el motivo por el cual tus enlaces simbólicos en `/usr/lib` no funcionaban es que estabas intentando apuntar a archivos que no existían o que tenían el ABI viejo.

Para que tu sistema Arch vuelva a arrancar o para poder hacer `chroot` con éxito, necesitas mover estos binarios a su lugar real. Sigue estos pasos con cuidado (estás trabajando sobre la librería más crítica):

## 1. Localiza el Cargador (Loader)

Para que `libc.so` funcione, necesita su pareja: el cargador dinámico. Debería estar en la misma carpeta de construcción. Verifica que también tenga el tag:

Bash

```
readelf -d /arch_repair/glibc/build/elf/ld.so | grep PLT
```

## 2. Instalación Manual Selectiva

No uses `make install` si sigue fallando por lo de `libmvec`. Vamos a mover los archivos esenciales manualmente a la partición de Arch (asumiendo que está montada en `/mnt`):

Bash

```
# 1. Copiar la libc real (usa el nombre con .6 que es el que busca el sistema)
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

# 2. Copiar el cargador dinámico
cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# 3. Asegurar los enlaces en la raíz de la partición (si existen)
ln -sf /usr/lib/libc.so.6 /mnt/lib64/libc.so.6
ln -sf /usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

## 3. ¿Por qué esto "arregla" el enlace simbólico?

El sistema no rechaza el enlace simbólico por ser un enlace, sino porque el cargador viejo (`ld.so`) del USB de rescate intentaba leer una librería con `DT_X86_64_PLT` y se asustaba, o viceversa. Al poner ambos archivos (cargador y librería) de la misma compilación en `/mnt`, la cadena de ejecución se completa:

1. El binario pide el cargador `/lib64/ld-linux-x86-64.so.2`.
2. El cargador (ahora el nuevo) entiende el tag `DT_X86_64_PLT`.
3. El cargador carga la `libc.so.6` (la nueva).
4. El programa ejecuta.

## 4. Prueba de fuego

Intenta ejecutar un binario de Arch usando tu nueva librería y cargador sin entrar todavía en chroot:

Bash

```
/mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /
```

Si esto te devuelve el listado de archivos de la raíz, **tu sistema está recuperado**. Ya puedes intentar el `arch-chroot /mnt` y esta vez no debería fallar.

> **You asked:**

Has dicho

No muestra nada.

/mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /

> **Gemini Replied:**

## Gemini ha dicho

Si no muestra absolutamente nada (ni siquiera un error de "Segmentation fault" o "Permission denied"), es probable que el cargador esté encontrando una discrepancia letal antes de poder emitir cualquier salida por `stdout` o `stderr`.

Aquí hay tres razones por las cuales ese comando puede quedarse en silencio y cómo forzarlo a hablar:

## 1. El problema del "Hardcoding" del Loader

Muchos binarios de Arch Linux tienen rutas preconfiguradas. Si el cargador que acabas de compilar intenta buscar configuraciones en `/etc/ld.so.cache` y encuentra el caché del **USB de rescate** (que no conoce el nuevo ABI), se detiene en seco.

Prueba forzarlo a ignorar el caché y usar solo tus librerías nuevas:

Bash

```
LD_DEBUG=all /mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /
```

> ***Nota:*** *`LD_DEBUG=all` forzará al cargador a escupir cada paso que da. Si sigue sin mostrar nada, el problema es que el cargador mismo no puede ejecutarse.*

## 2. ¿Es el cargador un binario válido?

Es posible que `ld.so` necesite librerías que no están en la ruta. Verifica si el cargador que compilaste tiene dependencias "huérfanas" usando el `readelf` de nuevo, pero buscando la sección `INTERP` o `NEEDED`:

Bash

```
readelf -a /mnt/usr/lib/ld-linux-x86-64.so.2 | grep NEEDED
```

Normalmente, el cargador ( `ld.so` ) es **autosuficiente** (estático), pero si por algún error de compilación espera encontrar algo más, fallará en silencio.

## 3. La trampa de los Enlaces Simbólicos

En tu comando usaste `/mnt/usr/lib/ld-linux-x86-64.so.2`. Si ese archivo es un enlace simbólico que apunta a algo fuera de `/mnt`, el comando fallará. Asegúrate de que apunte al archivo real que acabas de copiar:

Bash

```
ls -lh /mnt/usr/lib/ld-linux-x86-64.so.2
```

---

## El "Plan B" (Si el cargador nuevo no responde)

Si el cargador que compilaste no reacciona, vamos a intentar usar el binario que **sí** sabemos que funciona (el de Arch que ya estaba en el disco) pero inyectándole tu nueva `libc.so.6`.

1. **Restaura el cargador original de Arch** (el que estaba en `/mnt` antes de tu copia, si lo tienes, o déjalo como estaba).
2. Ejecuta el comando usando el cargador original pero apuntando a tu `libc` recién horneada:

Bash

```
# Reemplaza /ruta/a/tu/libc.so.6 por la ubicación exacta de la que tiene el DT_X86_64_PLT
LD_PRELOAD=/mnt/usr/lib/libc.so.6 /usr/bin/ls /
```

## ¿Qué verás con `LD_DEBUG=all`?

Si el problema es el ABI, verás una línea similar a esta en el chorro de texto:

```
...relocation error: symbol ... version GLIBC_2.41 not defined in file libc.so.6
with link time reference
```

**¿Qué escupió el comando con `LD_DEBUG=all`?** Esa es la única forma de saber en qué dirección de memoria se está muriendo el proceso.

> **You asked:**

Has dicho

 LD_DEBUG=all /mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /

bash: /mnt/usr/lib/ld-linux-x86-64.so.2: No such file or directory

 root /arch_repair/glibc/build 

 readelf -a /mnt/usr/lib/ld-linux-x86-64.so.2 | grep NEEDED

readelf: Error: '/mnt/usr/lib/ld-linux-x86-64.so.2': No such file

 root /arch_repair/glibc/build 

 ls -lh /mnt/usr/lib/ld-linux-x86-64.so.2

ls: cannot access '/mnt/usr/lib/ld-linux-x86-64.so.2': No such file or directory

 root /arch_repair/glibc/build 

 # Reemplaza /ruta/a/tu/libc.so.6 por la ubicación exacta de la que tiene el DT_X86_64_PLT

LD_PRELOAD=/mnt/usr/lib/libc.so.6 /usr/bin/ls /

ERROR: ld.so: object '/mnt/usr/lib/libc.so.6' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored.

@ fix_locales.sh login run

'Program Files (x86)' gemini main.js sbin

afs gpg-agent.conf media screenshots

arch_repair gpg.conf mnt secring.gpg

bash_usb home opt share

bin img otro srv

boot initrd.img p sys

busybox lib pacman-static systemd_257.9-1~deb13u1_amd64.deb

cfb49c22-87f2-47d9-a25b-310d8d8578af lib32 pcsx tmp

control.tar.xz lib64 proc tmp0

copiar.sh libc-bin_2.42-13_amd64.deb programas.rb tmp2

data.tar.xz libc-bin_2.43-1_amd64.deb pubring.gpg usr

debian-binary libc-gconv-modules-extra_2.42-13_amd64.deb recovery_root var

desktopfs-pkgs.txt libc-l10n_2.41-12+deb13u2_all.deb reparar.sh virtualord

dev libc6_2.43-1_amd64.deb rescue virtualord_git.sh

efi libc6_repair.deb rescue_shell vmlinuz

etc livefs-pkgs.txt root vmlinuz.old

final_rescue locales_2.41-12+deb13u2_all.deb rootfs-pkgs.txt

 root /arch_repair/glibc/build 

**Gemini Replied:**

## Gemini ha dicho

El error `No such file or directory` al intentar ejecutar el cargador o hacer el `ls` indica que, aunque compilaste los archivos, todavía no están físicamente en `/mnt/usr/lib/`. El intento de `LD_PRELOAD` también falló porque no encontró el archivo en esa ruta.

Como ya confirmamos con `readelf` que el archivo en `/arch_repair/glibc/build/libc.so` **sí tiene el ABI correcto**, vamos a moverlo manualmente a la partición de Arch para revivir el sistema.

## 1. Mover los archivos a la partición de Arch

Ejecuta estos comandos para colocar la nueva Glibc y su cargador en su lugar definitivo dentro de tu disco de Arch:

Bash

```
# Copiar la libc que acabas de verificar
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

# Copiar el cargador dinámico (loader)
cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# Asegurar que los permisos sean correctos (lectura y ejecución)
chmod +x /mnt/usr/lib/libc.so.6
chmod +x /mnt/usr/lib/ld-linux-x86-64.so.2
```

## 2. Verificar que los archivos ahora sí existan en /mnt

Antes de intentar ejecutarlos, asegúrate de que el `cp` funcionó:

Bash

```
ls -lh /mnt/usr/lib/libc.so.6 /mnt/usr/lib/ld-linux-x86-64.so.2
```

## 3. La prueba de ejecución definitiva

Ahora que los archivos están en `/mnt`, intenta ejecutar el `ls` de Arch usando el cargador y la librería que acabas de instalar:

Bash

```
/mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /
```

## ¿Por qué falló el `LD_PRELOAD` antes?

El mensaje `ERROR: ld.so: object ... cannot be preloaded` ocurrió porque el archivo no estaba en la ruta `/mnt/usr/lib/libc.so.6`. Una vez que completes el paso 1 (el `cp`), ese comando debería funcionar, pero es mucho más limpio y seguro usar la llamada directa al cargador (`ld-linux-x86-64.so.2 --library-path...`) como indico en el punto 3.

**Si el comando del punto 3 funciona y te muestra el contenido de `/`**, significa que el sistema ya es capaz de ejecutar binarios. En ese momento, ya puedes intentar entrar con:

Bash

```
arch-chroot /mnt
```

Si entras al chroot con éxito, lo primero que debes hacer es un `pacman -Syu` o reinstalar `glibc` oficialmente desde los repositorios de Arch para que se instalen todos los archivos de soporte (locales,

headers, etc.) que faltan.

> **You asked:**

Has dicho

 # Copiar la libc que acabas de verificar

cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

# Copiar el cargador dinámico (loader)

cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# Asegurar que los permisos sean correctos (lectura y ejecución)

chmod +x /mnt/usr/lib/libc.so.6

chmod +x /mnt/usr/lib/ld-linux-x86-64.so.2

 root /arch_repair/glibc/build 

 ls -lh /mnt/usr/lib/libc.so.6 /mnt/usr/lib/ld-linux-x86-64.so.2

-rwxr-xr-x 1 root root 1.3M Mar 16 05:10 /mnt/usr/lib/ld-linux-x86-64.so.2

-rwxr-xr-x 1 root root  12M Mar 16 05:10 /mnt/usr/lib/libc.so.6

 root /arch_repair/glibc/build 

 /mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /

/mnt/usr/bin/ls: error while loading shared libraries: /mnt/usr/bin/ls: cannot open shared object file: No such file or directory

 root /arch_repair/glibc/build 

 ls

Makefile  dirent  ld.map  libnss_db.map  po  support

Versions.all  dl-tunable-list.h  ldbl-compat-choose.h  libnss_dns.map  posix  sysd-rules

Versions.def  dl-tunable-list.stmp  libBrokenLocale.map  libnss_files.map  resolv  sysd-sorted

Versions.mk dlfcn libanl.map libnss_hesiod.map resource sysd-syscalls

Versions.tmp dummy.c libc-abis.h libpthread.map rt sysd-versions

Versions.v dummy.o libc-abis.stamp libresolv.map rtld-offsets.h sysvipc

Versions.v.i dummy.o.dt libc-modules.h librt.map rtld-offsets.h.d tcb-offsets.h

abi-versions.h elf libc-modules.stmp libthread_db.map rtld-sizes.h tcb-offsets.h.d

argp features-offsets.h libc.a libutil.map rtld-sizes.h.d termios

assert features-offsets.h.d libc.map link-defines.h runtime-linker.h testrun.sh

bits first-versions.h libc.so link-defines.h.d runtime-linker.stamp time

catgets format.lds libc.so.6 linkobj setjmp time64-compat.mk

cmath gcc-macros.h libc_malloc_debug.map locale shlib-versions.v time64-compat.mk.i

config.h gcc-macros.h.in libc_nonshared.a locale-defines.h shlib-versions.v.i timezone

config.log gmon libc_pic.a locale-defines.h.d sigaltstack-offsets.h tlsdesc.h

config.make gnu libc_pic.opts localedata sigaltstack-offsets.h.d tlsdesc.h.d

config.status gnulib libc_pic.os login signal ucontext_i.h

conform hesiod libc_pic.os.clean malloc socket ucontext_i.h.d

cpu-features-offsets.h iconv libdl.map manual soversions.i unwindbuf.h

cpu-features-offsets.h.d iconvdata libio math soversions.mk unwindbuf.h.d

cstdlib inet libio-macros.h mathvec stamp.o versions.stmp

csu intl libio-macros.h.d misc stamp.oS wcsmbs

ctype io libm.map nis stamp.os wctype

debug jmp_buf-macros.h libmvec.map nptl stdio-common

debugglibc.sh jmp_buf-macros.h.d libmvec.mk nptl_db stdlib

descr-const.h jmp_buf-ssp.h libnsl.map nscd string

descr-const.h.d jmp_buf-ssp.h.d libnss_compat.map nss sunrpc

 root /arch_repair/glibc/build 

 /mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib /mnt/usr/bin/ls /

/mnt/usr/bin/ls: error while loading shared libraries: /mnt/usr/bin/ls: cannot open shared object file: No such file or directory

 root /arch_repair/glibc/build 

 arch-chroot /mnt

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> WARNING: /mnt is not a mountpoint. This may have undesirable side effects.

mount: /mnt/proc: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

==> ERROR: failed to setup chroot /mnt

**Gemini Replied:**

## Gemini ha dicho

El error que ves con `ls` es engañoso: cuando el cargador dice "cannot open shared object file", a veces se refiere al **propio binario** que intentas ejecutar o a una dependencia básica (como `libcap` o `libpcre2`) que `ls` necesita y no está encontrando en la ruta de librerías.

Además, el mensaje de `arch-chroot` confirma que algo grave pasa con tus puntos de montaje. Si `/mnt` no es un "mountpoint", significa que **no has montado la partición de Arch** en `/mnt` en esta sesión, o se ha desmontado.

Vamos a resolverlo paso a paso:

## 1. Asegura el montaje de tu sistema

Si `/mnt/proc` no existe, es que estás viendo la carpeta `/mnt` vacía del USB de rescate, no tu disco duro.

Bash

```
# Asumiendo que /dev/sda2 es tu partición de Arch (ajusta según tu caso)
mount /dev/sda2 /mnt

# Verifica que ahora sí veas tus archivos
ls /mnt/usr/bin/ls
```

## 2. El comando "Super-Library-Path"

Si `ls` sigue fallando, es porque necesita otras librerías que están en `/mnt/usr/lib`. El cargador necesita que le digamos explícitamente dónde buscar todo. Intenta esto:

Bash

```
/mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib:/mnt/lib64 /mnt/usr/bin/ls /mnt
```

## 3. Por qué falló el `arch-chroot`

`arch-chroot` es un script que intenta montar `/proc`, `/sys` y `/dev` por ti. Si falla diciendo que `/mnt` no es un mountpoint, es porque el disco no está ahí.

**Una vez que hayas montado el disco en `/mnt`**, haz esto antes de intentar el chroot:

Bash

```
# Copia de nuevo los archivos por si acaso los copiaste a la carpeta vacía antes
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6
cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# Intenta el chroot de forma manual si arch-chroot sigue protestando
mount --bind /dev /mnt/dev
mount --bind /proc /mnt/proc
mount --bind /sys /mnt/sys
mount --bind /run /mnt/run

chroot /mnt /bin/bash
```

## 4. ¿Faltan librerías?

Si el comando del paso 2 te sigue dando error de "shared object file", verifica qué le falta a `ls`:

Bash

```
# Usa el readelf del USB sobre el binario de Arch
readelf -d /mnt/usr/bin/ls | grep NEEDED
```

Si ves cosas como `libcap.so.2`, asegúrate de que esos archivos existan en `/mnt/usr/lib/`.

---

**Siguiente paso:** ¿Podrías confirmar con `lsblk` en qué dispositivo está tu partición de Arch y si está realmente montada en `/mnt`? Sin el disco montado, estamos trabajando sobre la memoria RAM del USB.

> **You asked:**

Has dicho

 # # Asumiendo que /dev/sda2 es tu partición de Arch (ajusta según tu caso)

mount /dev/sda2 /mnt

# Verifica que ahora sí veas tus archivos

ls /mnt/usr/bin/ls

mount: /mnt: fsconfig() failed: /dev/sda2: Can't lookup blockdev.

dmesg(1) may have more information after failed mount system call.

ls: cannot access '/mnt/usr/bin/ls': No such file or directory

 root /arch_repair/glibc/build 

 /mnt/usr/lib/ld-linux-x86-64.so.2 --library-path /mnt/usr/lib:/mnt/lib64 /mnt/usr/bin/ls /mnt

/mnt/usr/bin/ls: error while loading shared libraries: /mnt/usr/bin/ls: cannot open shared object file: No such file or directory

 root /arch_repair/glibc/build 

 # # Copia de nuevo los archivos por si acaso los copiaste a la carpeta vacía antes

cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# Intenta el chroot de forma manual si arch-chroot sigue protestando

mount --bind /dev /mnt/dev

mount --bind /proc /mnt/proc

mount --bind /sys /mnt/sys

mount --bind /run /mnt/run

chroot /mnt /bin/bash

mount: /mnt/dev: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

mount: /mnt/proc: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

mount: /mnt/sys: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

mount: /mnt/run: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

chroot: failed to run command '/bin/bash': No such file or directory

 root /arch_repair/glibc/build 

 # Usa el readelf del USB sobre el binario de Arch

readelf -d /mnt/usr/bin/ls | grep NEEDED

readelf: Error: '/mnt/usr/bin/ls': No such file

**Gemini Replied:**

## Gemini ha dicho

El error `Can't lookup blockdev` al intentar montar `/dev/sda2` sugiere que el sistema no reconoce ese nombre de dispositivo o que el nodo en `/dev` no está presente. Esto explica por qué todos los comandos siguientes fallan: **estás intentando trabajar sobre una carpeta `/mnt` que está vacía y vive solo en la RAM de tu USB.**

Necesitamos encontrar dónde está realmente tu disco de Arch.

## 1. Identifica el nombre real del disco

Ejecuta este comando para ver la lista de discos y particiones disponibles:

Bash

```
lsblk -f
```

Busca la partición que tenga formato `ext4` o `btrfs` y que por su tamaño sepas que es la de tu sistema Arch. Fíjate bien en el nombre (podría ser `/dev/sdb2`, `/dev/nvme0n1p2`, etc.).

## 2. Intenta montar por UUID (Más seguro)

Si el nombre `/dev/sda2` falla, usa el ID único que verás en el comando anterior:

Bash

```
# Reemplaza EL-UUID-AQUI por el que salga en lsblk -f para tu partición de Arch
mount -t auto /dev/disk/by-uuid/EL-UUID-AQUI /mnt
```

## 3. El problema del montaje "fantasma"

Anteriormente lograste copiar archivos a `/mnt/usr/lib/`. Si el disco **no** estaba montado, esos archivos se guardaron en la memoria RAM del USB.

- **Consecuencia:** Esos archivos NO están en tu disco duro.
- **Solución:** Una vez que logres montar el disco real en `/mnt`, **debes repetir el proceso de copiado** de la `libc.so.6` y el `ld.so`.

---

## Pasos críticos ahora mismo:

1. **¿Qué te devuelve `lsblk -f`?** (Pásame la salida si puedes).
2. **Montar el disco real:**

   Bash

   ```
   # Si lsblk dice que tu disco es, por ejemplo, /dev/sdb2:
   mount /dev/sdb2 /mnt
   ```

3. **Verificar el contenido:**

   Bash

   ```
   # Esto DEBE devolver una lista de carpetas (etc, usr, var, etc.)
   ls /mnt
   ```

**No intentes ningún `chroot` ni ningún `cp` hasta que `ls /mnt` te muestre las carpetas reales de tu sistema Arch.** Si `ls /mnt` no muestra nada, seguimos trabajando en el aire.

¿Lograste ver el disco con `lsblk`?

> **You asked:**

Has dicho

 lsblk -f

NAME FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS

loop0 iso9660 Joliet Extension MANJARO_KDE_2504 2025-06-23-07-30-54-00

loop1 squashfs 4.0

loop2 squashfs 4.0

loop3 squashfs 4.0

loop4 squashfs 4.0

sda

`-sda1 btrfs cfb49c22-87f2-47d9-a25b-310d8d8578af 6.2G 95% /

sdb

`-sdb1 ntfs CCCOMA_X64FRE_EN-US_DV9 5E70D4D870D4B7CB

 root /arch_repair/glibc/build 

 # Reemplaza EL-UUID-AQUI por el que salga en lsblk -f para tu partición de Arch

mount -t auto /dev/disk/by-uuid/EL-UUID-AQUI /mnt

mount: /mnt: fsconfig() failed: /dev/disk/by-uuid/EL-UUID-AQUI: Can't lookup blockdev.

dmesg(1) may have more information after failed mount system call.

 root /arch_repair/glibc/build 

 # # Si lsblk dice que tu disco es, por ejemplo, /dev/sdb2:

mount /dev/sdb2 /mnt

mount: /mnt: fsconfig() failed: /dev/sdb2: Can't lookup blockdev.

dmesg(1) may have more information after failed mount system call.

 root /arch_repair/glibc/build 

 # # Esto DEBE devolver una lista de carpetas (etc, usr, var, etc.)

ls /mnt

control.tar.xz  data.tar.xz  debian-binary  etc  lib64  libc6  libdevmapper1.02.1_1.02.205-2_amd64.deb  ram  sbin  systemd  usr  var

> **Gemini Replied:**

## Gemini ha dicho

Ahí está el problema. Mirando tu `lsblk`, tu partición de Arch es **`/dev/sda1`** y es de tipo **Btrfs**, pero ahora mismo **ya está montada en `/`** (la raíz de tu entorno actual).

Esto sucede porque estás operando desde lo que parece ser un entorno "live" que ha tomado `/dev/sda1` como su propia raíz, o bien ya estabas dentro de una estructura Btrfs. Además, el `ls /mnt` que mostraste muestra carpetas como `libc6`, `control.tar.xz` y `debian-binary`, lo que indica que en `/mnt` solo hay archivos temporales de los `.deb` que estuviste extrayendo, **no tu sistema real**.

Para reparar tu sistema Arch, necesitamos montar el **subvolumen** correcto de esa partición Btrfs en `/mnt`.

## 1. Identifica los subvolúmenes de tu Btrfs

Como `/dev/sda1` es Btrfs, el sistema de archivos está dividido en subvolúmenes (probablemente `@` para la raíz). Ejecuta esto:

Bash

```
btrfs subvolume list /
```

## 2. Monta el subvolumen real en `/mnt`

Si en la lista aparece uno llamado `@` (común en Manjaro/Arch), móntalo así:

Bash

```
# Primero limpiamos /mnt de los archivos temporales que tiene
rm -rf /mnt/* # Montamos el subvolumen raíz de tu disco duro
mount -o subvol /@ /dev/sda1 /mnt
```

*Si no existe* `@` *, intenta montar la partición plana para ver qué hay:* `mount /dev/sda1 /mnt` (aunque esto podría fallar si ya está ocupada).

## 3. Verifica que ahora sí veas Arch

Una vez montado con el comando `mount -o subvol...`, haz:

Bash

```
ls /mnt/etc/fstab
```

Si ese archivo existe, **estamos en el lugar correcto**.

## 4. Instala la Glibc reparada

Ahora sí, copia los archivos que compilaste al lugar donde realmente faltan:

Bash

```
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6
cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2
```

## 5. Intenta el Chroot

Con el subvolumen Btrfs correctamente montado en `/mnt`, el script de Arch debería funcionar:

Bash

```
arch-chroot /mnt
```

**¿Qué te devolvió el comando `btrfs subvolume list /`?** Eso nos dirá exactamente qué nombre tiene "el disco" que debemos reparar. Pero cuidado: como `/dev/sda1` ya figura montado en `/`, cualquier cambio que hagas en `/usr/lib/` ahora mismo está afectando al sistema que tienes activo.

> **You asked:**

Has dicho

 root /arch_repair/glibc/build 

 btrfs subvolume list /

ID 256 gen 411948 top level 5 path @

 root /arch_repair/glibc/build 

 # Primero limpiamos /mnt de los archivos temporales que tiene

rm -rf /mnt/* # Montamos el subvolumen raíz de tu disco duro

mount -o subvol /@ /dev/sda1 /mnt

rm: cannot remove '/mnt/ram': Device or resource busy

mount: bad usage

Try 'mount --help' for more information.

 root /arch_repair/glibc/build 

 ls /mnt/etc/fstab

ls: cannot access '/mnt/etc/fstab': No such file or directory

 root /arch_repair/glibc/build 

 cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

cp: cannot create regular file '/mnt/usr/lib/libc.so.6': No such file or directory

cp: cannot create regular file '/mnt/usr/lib/ld-linux-x86-64.so.2': No such file or directory

 root /arch_repair/glibc/build 

 arch-chroot /mnt

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> WARNING: /mnt is not a mountpoint. This may have undesirable side effects.

mount: /mnt/proc: mount point does not exist.

    dmesg(1) may have more information after failed mount system call.

==> ERROR: failed to setup chroot /mnt

 root /arch_repair/glibc/build 

**Gemini Replied:**

# Gemini ha dicho

El error `mount: bad usage` ocurrió porque el comando de montaje tenía un pequeño error de sintaxis y el sistema detectó que `/mnt` estaba ocupado (por eso el `rm` falló con `/mnt/ram`).

Lo más importante aquí es que `btrfs subvolume list /` confirmó que tu sistema Arch está en el subvolumen `@`.

Vamos a forzar el montaje correcto. Sigue estos pasos exactos:

## 1. Desmontar cualquier resto en /mnt

Primero liberamos la carpeta para que no haya conflictos:

Bash

```
umount -R /mnt
```

## 2. Montar el subvolumen @ correctamente

El parámetro `-o subvol=@` (sin la barra inclinada antes de la @) es el estándar para Btrfs.

Bash

```
mount -t btrfs -o subvol=@ /dev/sda1 /mnt
```

## 3. Verificar que entramos al disco real

Si esto falla, no sigas. Necesitamos ver tu Arch:

Bash

```
ls /mnt/usr/lib | grep libc
```

Si ves muchos archivos, **estamos dentro**.

## 4. Instalar la Glibc reparada

Ahora que el disco real está en `/mnt`, copiamos los archivos con el ABI nuevo:

Bash

```
# Crear el directorio si por algún motivo no existe (poco probable en Arch)
mkdir -p /mnt/usr/lib

# Copiar los binarios
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6
cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

# Darles permisos de ejecución
chmod +x /mnt/usr/lib/libc.so.6 /mnt/usr/lib/ld-linux-x86-64.so.2
```

## 5. El Chroot (La prueba final)

Ahora que el disco está montado y los archivos están en su lugar, `arch-chroot` debería funcionar:

Bash

```
arch-chroot /mnt
```

**Si logras entrar al prompt del chroot (que dirá algo como `[root@manjaro /]#`):** Significa que el ABI de la Glibc que compilaste es compatible con el resto de tus binarios.

Lo primero que debes hacer adentro es intentar refrescar los repositorios para arreglar el sistema de forma oficial:

Bash

```
pacman -Sy glibc
```

**¿Qué pasó al ejecutar el `mount -t btrfs -o subvol=@ /dev/sda1 /mnt`?** Si te da error de "already mounted", es porque `/dev/sda1` ya es tu raíz actual y tendríamos que operar directamente sobre `/usr/lib` del sistema donde estás parado.

> **You asked:**

Has dicho

 lsblk -f

NAME FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS

loop0 iso9660 Joliet Extension MANJARO_KDE_2504 2025-06-23-07-30-54-00

```
loop1  squashfs 4.0

loop2  squashfs 4.0

loop3  squashfs 4.0

loop4  squashfs 4.0

sda

`-sda1 btrfs                                cfb49c22-87f2-47d9-a25b-310d8d8578af    6.2G    95% /

sdb

`-sdb1 ntfs                    CCCOMA_X64FRE_EN-US_DV9 5E70D4D870D4B7CB


 root /arch_repair/glibc/build 

 # Reemplaza EL-UUID-AQUI por el que salga en lsblk -f para tu partición de Arch

mount -t auto /dev/disk/by-uuid/EL-UUID-AQUI /mnt

mount: /mnt: fsconfig() failed: /dev/disk/by-uuid/EL-UUID-AQUI: Can't lookup blockdev.

     dmesg(1) may have more information after failed mount system call.

 root /arch_repair/glibc/build 

 # # Si lsblk dice que tu disco es, por ejemplo, /dev/sdb2:

mount /dev/sdb2 /mnt

mount: /mnt: fsconfig() failed: /dev/sdb2: Can't lookup blockdev.

     dmesg(1) may have more information after failed mount system call.

 root /arch_repair/glibc/build 

 # # Esto DEBE devolver una lista de carpetas (etc, usr, var, etc.)

ls /mnt

control.tar.xz  data.tar.xz  debian-binary  etc  lib64  libc6  libdevmapper1.02.1_1.02.205-2_amd64.deb
 ram  sbin  systemd  usr  var

 root /arch_repair/glibc/build 

 btrfs subvolume list /

ID 256 gen 411948 top level 5 path @

 root /arch_repair/glibc/build 
```

 # Primero limpiamos /mnt de los archivos temporales que tiene

rm -rf /mnt/* # Montamos el subvolumen raíz de tu disco duro

mount -o subvol /@ /dev/sda1 /mnt

rm: cannot remove '/mnt/ram': Device or resource busy

mount: bad usage

Try 'mount --help' for more information.

 root /arch_repair/glibc/build 

 ls /mnt/etc/fstab

ls: cannot access '/mnt/etc/fstab': No such file or directory

 root /arch_repair/glibc/build 

 cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2

cp: cannot create regular file '/mnt/usr/lib/libc.so.6': No such file or directory

cp: cannot create regular file '/mnt/usr/lib/ld-linux-x86-64.so.2': No such file or directory

 root /arch_repair/glibc/build 

 arch-chroot /mnt

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> WARNING: /mnt is not a mountpoint. This may have undesirable side effects.

mount: /mnt/proc: mount point does not exist.

dmesg(1) may have more information after failed mount system call.

==> ERROR: failed to setup chroot /mnt

 root /arch_repair/glibc/build 

 root /arch_repair/glibc/build 

 root /arch_repair/glibc/build 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 

 root /arch_repair/glibc/build 

 umount -R /mnt

umount: /mnt: not mounted

 root /arch_repair/glibc/build 

 mount -t btrfs -o subvol=@ /dev/sda1 /mnt

 root /arch_repair/glibc/build 

 ls /mnt/usr/lib | grep libc

klibc

klibc-cbxFsLVJ9bt_YYfJZGrxjofFc_w.so

libc.a

libc.so

libc.so.6

libc4core.so

libc4core.so.0.1.11

libc_malloc_debug.so

libc_malloc_debug.so.0

libc_nonshared.a

libcaca++.so

libcaca++.so.0

libcaca++.so.0.99.20

libcaca.so

libcaca.so.0

libcaca.so.0.99.20

libcacard.so

libcacard.so.0

libcacard.so.0.0.0

libcairo-gobject.so

libcairo-gobject.so.2

libcairo-gobject.so.2.11600.0

libcairo-gobject.so.2.11804.4

libcairo-script-interpreter.so

libcairo-script-interpreter.so.2

libcairo-script-interpreter.so.2.11804.4

libcairo.so

libcairo.so.2

libcairo.so.2.11600.0

libcairo.so.2.11804.4

libcairomm-1.0.so

libcairomm-1.0.so.1

libcairomm-1.0.so.1.4.0

libcairomm-1.16.so

libcairomm-1.16.so.1

libcairomm-1.16.so.1.4.0

libcalamares.so

libcalamares.so.3.3

libcalamares.so.3.3.14

libcalamaresui.so

libcalamaresui.so.3.3

libcalamaresui.so.3.3.14

libcalc.so

libcalc.so.2.16.1

libcalc.so.2.16.1.2

libcallback.so

libcallback.so.1

libcallback.so.1.0.5

libcamd.so

libcamd.so.3

libcamd.so.3.3.5

libcanberra-0.30

libcanberra-gtk.so

libcanberra-gtk.so.0

libcanberra-gtk.so.0.1.9

libcanberra-gtk3.so

libcanberra-gtk3.so.0

libcanberra-gtk3.so.0.1.9

libcanberra.so

libcanberra.so.0

libcanberra.so.0.2.5

libcap-ng.so

libcap-ng.so.0

libcap-ng.so.0.0.0

libcap.so

libcap.so.2

libcap.so.2.76

libcap.so.2.77

libcapstone.so

libcapstone.so.5

libcares.so

libcares.so.2

libcares.so.2.19.5

libcblas.so

libcblas.so.3

libcblas.so.3.12.0

libcbor.so

libcbor.so.0.12

libcbor.so.0.12.0

libcc1.so

libcc1.so.0

libcc1.so.0.0.0

libccolamd.so

libccolamd.so.3

libccolamd.so.3.3.5

libcdda_interface.so

libcdda_interface.so.0

libcdda_interface.so.0.10.2

libcdda_paranoia.so

libcdda_paranoia.so.0

libcdda_paranoia.so.0.10.2

libcdio++.so

libcdio++.so.1

libcdio++.so.1.0.0

libcdio.so

libcdio.so.19

libcdio.so.19.0.0

libcdio.so.19.1.0

libcdio_cdda.so

libcdio_cdda.so.2

libcdio_cdda.so.2.0.0

libcdio_paranoia.so

libcdio_paranoia.so.2

libcdio_paranoia.so.2.0.0

libcdr-0.1.so

libcdr-0.1.so.1

libcdr-0.1.so.1.0.8

libcdt.so

libcdt.so.6

libcdt.so.6.0.2

libcfitsio.so

libcfitsio.so.10

libcfitsio.so.4.6.2

libcfitsio.so.4.6.3

libcgraph.so

libcgraph.so.8

libcgraph.so.8.0.5

libchafa.so

libchafa.so.0

libchafa.so.0.11.0

libcharset.so

libcharset.so.1

libcharset.so.1.0.0

libchm.so

libchm.so.0

libchm.so.0.0.0

libcholmod.so

libcholmod.so.5

libcholmod.so.5.3.4

libchromaprint.so

libchromaprint.so.1

libchromaprint.so.1.5.1

libchromaprint.so.1.6.0

libcjs.so

libcjs.so.0

libcjs.so.0.0.0

libclang-cpp.so

libclang-cpp.so.20.1

libclang-cpp.so.21.1

libclang.so

libclang.so.20.1

libclang.so.21.1

libclang.so.21.1.6

libcloudproviders.so

libcloudproviders.so.0

libcloudproviders.so.0.3.6

libcloudproviders.so.0.4.0

libcolamd.so

libcolamd.so.3

libcolamd.so.3.3.5

libcollection.so

libcollection.so.4

libcollection.so.4.1.1

libcolorcorrect.so

libcolorcorrect.so.6

libcolorcorrect.so.6.3.5

libcolord-gtk4.so

libcolord-gtk4.so.1

libcolord-gtk4.so.1.0.3

libcolord.so

libcolord.so.2

libcolord.so.2.0.5

libcolordcompat.so

libcolordprivate.so

libcolordprivate.so.2

libcolordprivate.so.2.0.5

libcolorhug.so

libcolorhug.so.2

libcolorhug.so.2.0.5

libcom_err.so

libcom_err.so.2

libcom_err.so.2.1

libcomposefs.so

libcomposefs.so.1

libcomposefs.so.1.4.0

libconfuse.so

libconfuse.so.2

libconfuse.so.2.1.0

libcord.so

libcord.so.1

libcord.so.1.5.1

libcppdap.so

libcpupower.so

libcpupower.so.1

libcpupower.so.1.0.1

libcrack.so

libcrack.so.2

libcrack.so.2.9.0

libcrypt.so

libcrypt.so.1

libcrypt.so.1.1.0

libcrypt.so.2

libcrypt.so.2.0.0

libcrypto.so

libcrypto.so.3

libcryptopp.so

libcryptopp.so.8

libcryptopp.so.8.9.0

libcryptsetup.so

libcryptsetup.so.12

libcryptsetup.so.12.10.0

libcryptsetup.so.12.11.0

libct.so

libct.so.4

libct.so.4.0.0

libctf-nobfd.a

libctf-nobfd.so

libctf-nobfd.so.0

libctf-nobfd.so.0.0.0

libctf.a

libctf.so

libctf.so.0

libctf.so.0.0.0

libctpl.so

libctpl.so.2

libctpl.so.2.3.0

libcue.so

libcue.so.2

libcue.so.2.3.0

libcups.so

libcups.so.2

libcupsfilters.so

libcupsfilters.so.2

libcupsfilters.so.2.0.0

libcupsimage.so

libcupsimage.so.2

libcurl.so

libcurl.so.3

libcurl.so.4

libcurl.so.4.7.9

libcurl.so.4.8

libcurl.so.4.8.0

libcurses.so

libcursesw.so

libcustcalc.so

libcustcalc.so.2.16.1

libcustcalc.so.2.16.1.2

libcxl.so

libcxl.so.1

libcxl.so.1.0.6

libcxsparse.so

libcxsparse.so.4

libcxsparse.so.4.4.2

 root /arch_repair/glibc/build 

 root /arch_repair/glibc/build 

 # Crear el directorio si por algún motivo no existe (poco probable en Arch)

mkdir -p /mnt/usr/lib

# Copiar los binarios

```
cp /arch_repair/glibc/build/libc.so /mnt/usr/lib/libc.so.6

cp /arch_repair/glibc/build/elf/ld.so /mnt/usr/lib/ld-linux-x86-64.so.2


# Darles permisos de ejecución

chmod +x /mnt/usr/lib/libc.so.6 /mnt/usr/lib/ld-linux-x86-64.so.2

 root /arch_repair/glibc/build 

 arch-chroot /mnt

/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root / 

 pacman -Sy glibc

:: Synchronizing package databases...

core is up to date

extra                                       8.0 MiB   434 KiB/s 00:19
[#######################################################] 100%

multilib is up to date

andontie-aur is up to date

archlinuxgr is up to date

archlinuxcn                                 1341.8 KiB  62.5 KiB/s 00:21
[#######################################################] 100%

extra-alucryd is up to date

arcanisrepo is up to date

chaotic-aur                                 646.8 KiB  30.3 KiB/s 00:21
[#######################################################] 100%

archstrike is up to date

warning: glibc-2.43+r5+g856c426a7534-1 is up to date -- reinstalling

resolving dependencies...

looking for conflicting packages...
```

Packages (1) glibc-2.43+r5+g856c426a7534-1

Total Installed Size:  48.91 MiB

Net Upgrade Size:  0.00 MiB

:: Proceed with installation? [Y/n]

(1/1) checking keys in keyring [#######################################################] 100%

(1/1) checking package integrity [#######################################################] 100%

(1/1) loading package files [#######################################################] 100%

(1/1) checking for file conflicts [#######################################################] 100%

(1/1) checking available disk space [#######################################################] 100%

:: Processing package changes...

(1/1) reinstalling glibc [#######################################################] 100%

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

Generating locales...

Generation complete.

/tmp/alpm_rfsLuy/.INSTALL: line 6: /usr/local/sbin/iconvconfig: cannot execute: required file not found

error: command failed to execute correctly

:: Running post-transaction hooks...

(1/3) Arming ConditionNeedsUpdate...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

(2/3) Restarting cronie for libc upgrade...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

Skipped: Running in chroot.

(3/3) Updating the info directory file...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

**Gemini Replied:**

## Gemini ha dicho

¡Misión cumplida! El hecho de que hayas podido ejecutar `pacman -Sy glibc` y que el sistema haya procesado la transacción significa que **tu sistema Arch ha vuelto a la vida**. La `libc.so.6` que compilaste con paciencia sirvió de puente perfecto para que el sistema pudiera volver a entenderse a sí mismo.

Los errores que viste al final ( `iconvconfig` y los warnings de locales) son normales en este punto por dos razones:

1. **Estás en un chroot:** Muchos servicios y ganchos (hooks) de post-instalación se saltan pasos porque no hay un sistema de archivos real "vivo" (como el error de `cronie` ).
2. **Entorno sucio:** Todavía quedan rastros de los archivos temporales que usamos para la reparación.

Para terminar de estabilizar el sistema y poder reiniciar con total confianza, te recomiendo estos últimos pasos dentro del chroot:

## 1. Reinstalar dependencias base

A veces, cuando `glibc` se rompe, otros paquetes esenciales quedan en un estado inconsistente. Dale un repaso a la base:

Bash

```
pacman -S --noconfirm coreutils bash binutils
```

## 2. Regenerar los Locales

Para quitar esos molestos avisos de `setlocale` , asegúrate de que tu archivo de configuración esté bien y genera los datos de idioma:

Bash

```
# Verifica que tu idioma (es_AR.UTF-8) esté descomentado
nano /etc/locale.gen

# Genera los locales de nuevo
locale-gen
```

## 3. Limpieza y Salida

Ya no necesitas los archivos manuales que pusimos en `/usr/lib` porque `pacman` los ha sobreescrito con las versiones oficiales del repositorio (que ahora sí son compatibles).

Bash

```
exit      # Salir del chroot
umount -R /mnt  # Desmontar todo limpiamente
```

---

## El momento de la verdad

Ahora puedes reiniciar la máquina. Al arrancar desde el disco duro, el sistema debería cargar la nueva Glibc oficial sin errores de ABI.

> ***Nota técnica:*** *El error de `iconvconfig` que mencionaste suele ocurrir si hay un binario en `/usr/local/sbin` que quedó huérfano de una instalación manual previa. Una vez que reinicies tu sistema real, podrías borrar ese archivo si ves que sigue molestando, ya que los binarios oficiales de Arch están en `/usr/bin`.*

¿Quieres que revisemos la configuración del cargador de arranque (GRUB/Systemd-boot) antes de que le des al botón de reiniciar, o te sientes listo para el despegue?

> **You asked:**

Has dicho

 root /arch_repair/glibc/build 

 busybox.static  cp -v /arch_repair/glibc/build/libc.so* /usr/local/lib/

'/arch_repair/glibc/build/libc.so' -> '/usr/local/lib/libc.so'

'/arch_repair/glibc/build/libc.so.6' -> '/usr/local/lib/libc.so.6'

 root /arch_repair/glibc/build 

 readelf -d /usr/local/lib/libc.so.6 | grep -i PLT

0x0000000070000000 (DT_X86_64_PLT)  0x24000

0x0000000070000001 (DT_X86_64_PLTSZ)  0x3a0

0x0000000070000003 (DT_X86_64_PLTENT)  0x10

0x0000000000000003 (PLTGOT)  0x1e1fe8

0x0000000000000002 (PLTRELSZ)  1368 (bytes)

0x0000000000000014 (PLTREL)  RELA

 root /arch_repair/glibc/build 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /usr/local/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: /usr/local/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /usr/local/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /usr/local/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /usr/local/lib/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

**Gemini Replied:**

## Gemini ha dicho

Ese error de `gpg: version GLIBC_ABI_DT_X86_64_PLT not found` es un síntoma clásico de una "mezcla" de librerías. Lo que está pasando es que `gpg` está cargando tu nueva `libc.so.6` desde

`/usr/local/lib/`, pero luego intenta cargar la librería matemática (`libm.so.6`) desde `/usr/lib/`. Como `libm` es la versión oficial de Arch, espera que la `libc` tenga todos los símbolos y el ABI perfectamente sincronizados, y algo en la versión compilada manualmente no termina de cerrar el círculo para `libm`.

La buena noticia es que **ya no necesitas usar `/usr/local/lib/`**. Como ya logramos entrar al `chroot` y reinstalar `glibc` con `pacman`, el sistema debería ser capaz de usar sus propias librerías oficiales en `/usr/lib/`.

Para limpiar el entorno y que `pacman-key` funcione, sigue estos pasos:

## 1. Elimina las librerías de `/usr/local/lib/`

Esas librerías manuales ahora están interfiriendo con las oficiales que instaló `pacman`.

Bash

```
rm /usr/local/lib/libc.so*
# Si copiaste el ld.so ahí, quítalo también
rm /usr/local/lib/ld-linux-x86-64.so.2
```

## 2. Refresca el caché de librerías

Esto obligará al sistema a olvidar `/usr/local/lib` y volver a usar `/usr/lib/libc.so.6` (la de Arch).

Bash

```
ldconfig
```

## 3. Arregla los Locales (para quitar los warnings)

Antes de seguir con las llaves, vamos a silenciar el error de `es_AR.UTF-8`.

Bash

```
# Genera los locales básicos
locale-gen
# Si falla, asegúrate de que esté descomentado en /etc/locale.gen
```

## 4. Reintenta las llaves de Pacman

Ahora que el sistema solo ve las librerías oficiales en `/usr/lib/`, `gpg` no debería tener conflictos con `libm.so.6`.

Bash

```
pacman-key --init
pacman-key --populate archlinux
```

## ¿Por qué fallaba antes?

Cuando ejecutas un programa, el cargador dinámico tiene un orden de búsqueda. Al poner archivos en `/usr/local/lib/`, les diste prioridad. `gpg` usó tu `libc` manual, pero cuando necesitó `libm`, fue a buscarla a `/usr/lib/`. La `libm` de Arch es muy exigente: si detecta una `libc` que no coincide exactamente con lo que espera (aunque tenga el tag `PLT`), lanza el error de versión.

**¿Lograste correr el `ldconfig` y borrar los archivos de `/usr/local/lib/`?** Una vez hecho eso, el comando `pacman-key` debería fluir sin errores de "version not found".

> **You asked:**

Has dicho

 root /otro/libc_compilado 

 grep -ri PLT 2>&1 | grep -i libc

grep: usr/lib/gconv/libCNS.so: binary file matches

grep: usr/lib/libc.a: binary file matches

grep: usr/share/locale/hr/LC_MESSAGES/libc.mo: binary file matches

grep: lib64/libc_malloc_debug.so.0: binary file matches

grep: lib64/libc.so.6: binary file matches

 root /otro/libc_compilado 

 pacman-key --populate archlinux

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> ERROR: You do not have sufficient permissions to read the pacman keyring.

==> Use 'pacman-key --init' to correct the keyring permissions.

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 pacman-key --populate archlinux

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> ERROR: You do not have sufficient permissions to read the pacman keyring.

==> Use 'pacman-key --init' to correct the keyring permissions.

 root /otro/libc_compilado 

 busybox.static cp -vf lib64/libc.so.6 /lib/x86_64-linux-gnu/libc.so.6

'lib64/libc.so.6' -> '/lib/x86_64-linux-gnu/libc.so.6'

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 ls

etc  lib64  sbin  usr  var

 root /otro/libc_compilado 

**Gemini Replied:**

## Gemini ha dicho

El problema persiste porque estás sobreescribiendo una ruta que Arch Linux normalmente no usa ( `/lib/x86_64-linux-gnu/` ), pero que algún binario (posiblemente de los paquetes `.deb` de Debian que extrajiste antes) está priorizando.

El error de `gpg` es claro: está cargando la `libc.so.6` que acabas de copiar a `/lib/x86_64-linux-gnu/` , pero `libm.so.6` (que está en `/usr/lib/` ) rechaza esa `libc` porque no reconoce el símbolo de versión exacto, a pesar de tener el tag PLT.

Para limpiar este "infierno de dependencias" y que `pacman-key` funcione con las librerías oficiales de Arch, haz lo siguiente:

## 1. Limpieza de rutas "contaminadas"

Arch Linux usa `/usr/lib` y `/lib` (que es un enlace a `/usr/lib`). La ruta `/lib/x86_64-linux-gnu` es típica de Debian/Ubuntu y solo está causando que los binarios se confundan. **Bórrala**:

Bash

```
# Borra la libc manual que pusiste en la ruta estilo Debian
busybox.static rm -f /lib/x86_64-linux-gnu/libc.so.6
```

## 2. Forzar el uso de la librería de Arch

Asegúrate de que la `libc.so.6` oficial (la que instalaste con `pacman -Sy glibc` hace un momento) sea la única que el sistema intente cargar.

Bash

```
# Actualiza los enlaces del cargador
ldconfig -v | grep libc.so.6
```

## 3. Solución al problema de GPG y libm

Si el error de `version GLIBC_ABI_DT_X86_64_PLT not found` sigue apareciendo, es porque `libm.so.6` también necesita ser la versión que compilaste o la oficial de Arch que coincida con la `libc`.

Como ya reinstalaste `glibc` con pacman, lo más sano es **reinstalarla de nuevo** asegurándote de que no haya archivos manuales interfiriendo:

Bash

```
# Asegúrate de que no haya nada en /usr/local/lib estorbando
rm -f /usr/local/lib/libc.so.6
rm -f /usr/local/lib/libm.so.6

# Reinstala glibc una vez más para asegurar que libc y libm estén sincronizadas
pacman -S glibc --noconfirm
```

## 4. Reparar el Keyring

Una vez que `gpg` deje de dar errores de librería, ejecuta:

Bash

```
# Elimina el directorio de gnupg corrupto si existe
rm -rf /etc/pacman.d/gnupg

# Inicializa de nuevo
pacman-key --init
pacman-key --populate archlinux
```

## ¿Por qué el error de permisos?

`pacman-key` te dice que no tienes permisos porque el directorio `/etc/pacman.d/gnupg` no se pudo crear correctamente o tiene dueños incorrectos debido a los fallos previos de `gpg`. Al borrar la carpeta y correr `--init` con las librerías ya corregidas, se solucionará.

¿Qué te devuelve `ls -l /lib/libc.so.6`? Debería ser un enlace simbólico a `/usr/lib/libc.so.6`.

> **You asked:**

Has dicho

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 busybox.static cp -vf ./lib64/libm.so.6 /lib/x86_64-linux-gnu

'./lib64/libm.so.6' -> '/lib/x86_64-linux-gnu/libm.so.6'

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 busybox.static cp -vf ./lib64/libm.so.6 /lib/x86_64-linux-gnu ^C

 root /otro/libc_compilado 

 busybox.static cp -vf ./usr/lib/libm.so /usr/lib/libm.so.6

'./usr/lib/libm.so' -> '/usr/lib/libm.so.6'

 root /otro/libc_compilado 

 ls

etc  lib64  sbin  usr  var

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: error while loading shared libraries: /usr/lib/libm.so.6: invalid ELF header

chmod: cannot access '/etc/pacman.d/gnupg//trustdb.gpg': No such file or directory

gpg: error while loading shared libraries: /usr/lib/libm.so.6: invalid ELF header

gpg: error while loading shared libraries: /usr/lib/libm.so.6: invalid ELF header

==> Generating pacman master key. This may take some time.

gpg: error while loading shared libraries: /usr/lib/libm.so.6: invalid ELF header

==> Updating trust database...

gpg: error while loading shared libraries: /usr/lib/libm.so.6: invalid ELF header

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 busybox.static cp -vf ./lib64/libm.so.6 /usr/lib/libm.so.6

'./lib64/libm.so.6' -> '/usr/lib/libm.so.6'

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /etc/pacman.d/gnupg/trustdb.gpg: trustdb created

gpg: no ultimately trusted keys found

gpg: WARNING: unsafe permissions on homedir '/root/.gnupg'

gpg: starting migration from earlier GnuPG versions

gpg: porting secret keys from '/etc/pacman.d/gnupg/secring.gpg' to gpg-agent

gpg: migration succeeded

==> Generating pacman master key. This may take some time.

gpg: Generating pacman keyring master key...

gpg: directory '/etc/pacman.d/gnupg/openpgp-revocs.d' created

gpg: revocation certificate stored as '/etc/pacman.d/gnupg/openpgp-revocs.d/2ECEC5B4E19DBA2FD8DC79BE4FFB716EC2E2D1CD.rev'

gpg: Done

==> Updating trust database...

gpg: marginals needed: 3  completes needed: 1  trust model: pgp

gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u

 root /otro/libc_compilado 

 pacman-key --populate archlinux

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> Appending keys from archlinux.gpg...

==> Locally signing trusted keys in keyring...

 -> Locally signed 5 keys.

==> Importing owner trust values...

gpg: setting ownertrust to 4

gpg: setting ownertrust to 4

gpg: setting ownertrust to 4

gpg: inserting ownertrust of 4

gpg: setting ownertrust to 4

==> Disabling revoked keys in keyring...

 -> Disabled 36 keys.

==> Updating trust database...

gpg: Note: third-party key signatures using the SHA1 algorithm are rejected

gpg: (use option "--allow-weak-key-signatures" to override)

gpg: marginals needed: 3  completes needed: 1  trust model: pgp

gpg: depth: 0  valid:   1  signed:   5  trust: 0-, 0q, 0n, 0m, 0f, 1u

gpg: depth: 1  valid:   5  signed:  84  trust: 0-, 0q, 0n, 5m, 0f, 0u

gpg: depth: 2  valid:  72  signed:  19  trust: 72-, 0q, 0n, 0m, 0f, 0u

gpg: next trustdb check due at 2026-10-21

 root /otro/libc_compilado 

**Gemini Replied:**

## Gemini ha dicho

¡Excelente movimiento! El error de "invalid ELF header" fue porque copiaste `libm.so` (que suele ser un script de linker de texto) en lugar del binario real, pero lo corregiste rápido usando la versión de `lib64`.

Ahora que `pacman-key` ha terminado de poblar las llaves sin errores de ABI, tu sistema ya tiene "permiso" para confiar en los paquetes oficiales. Esto es vital para terminar de limpiar la instalación manual.

Como ya tienes el keyring funcional, te recomiendo cerrar con estos pasos de "limpieza final" para que el sistema sea 100% estándar:

## 1. Reinstalar glibc (oficial)

Ahora que `pacman` confía en los repositorios, reinstala `glibc` para que los binarios oficiales de Arch pisen cualquier rastro de las librerías que compilamos o movimos manualmente a `/usr/lib`.

Bash

```
pacman -S glibc --noconfirm
```

## 2. Eliminar la ruta extraña

Esa carpeta `/lib/x86_64-linux-gnu/` no pertenece a Arch y podría causar problemas en el futuro si instalas algo que la detecte. Una vez que hayas reinstalado `glibc` (paso 1), bórrala:

Bash

```
busybox.static rm -rf /lib/x86_64-linux-gnu/
```

## 3. Sincronización total

Para asegurarte de que ningún otro binario quedó "tocado" por el problema del ABI, una actualización rápida no vendría mal:

Bash

```
pacman -Syu
```

---

## Notas de Post-Reparación

- **Locales:** Si te siguen molestando los avisos de `es_AR.UTF-8`, ejecuta `locale-gen` ahora que las librerías están estables.
- **Resumen:** Lograste lo que parecía imposible: reconstruir el puente del ABI (`GLIBC_ABI_DT_X86_64_PLT`) manualmente para que `pacman` volviera a funcionar.

**¿Hay algún otro paquete que te esté dando problemas o sientes que el sistema ya está listo para el reinicio definitivo?** Sería ideal verificar que el kernel y el initramfs estén en orden: `mkinitcpio -P`.

> **You asked:**

Has dicho

 pacman -S glibc --noconfirm

warning: glibc-2.43+r5+g856c426a7534-1 is up to date -- reinstalling

resolving dependencies...

looking for conflicting packages...

Packages (1) glibc-2.43+r5+g856c426a7534-1

Total Installed Size:  48.91 MiB

Net Upgrade Size:  0.00 MiB

:: Proceed with installation? [Y/n]

(1/1) checking keys in keyring [#######################################################] 100%

(1/1) checking package integrity [#######################################################] 100%

(1/1) loading package files
[#######################################] 100%

(1/1) checking for file conflicts
[#######################################] 100%

(1/1) checking available disk space
[#######################################] 100%

:: Processing package changes...

(1/1) reinstalling glibc
[#######################################] 100%

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

Generating locales...

Generation complete.

/tmp/alpm_rwQdEj/.INSTALL: line 6: /usr/local/sbin/iconvconfig: cannot execute: required file not found

error: command failed to execute correctly

:: Running post-transaction hooks...

(1/3) Arming ConditionNeedsUpdate...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

(2/3) Restarting cronie for libc upgrade...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

Skipped: Running in chroot.

(3/3) Updating the info directory file...

/bin/sh: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Generating pacman master key. This may take some time.

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> Updating trust database...

gpg: /usr/lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_ABI_DT_X86_64_PLT' not found (required by /usr/lib/libm.so.6)

==> ERROR: Trust database could not be updated.

 root /otro/libc_compilado 

 busybox.static cp -vf ./lib64/libm.so.6 /usr/lib/libm.so.6

'./lib64/libm.so.6' -> '/usr/lib/libm.so.6'

 root /otro/libc_compilado 

 pacman-key --init

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

gpg: WARNING: unsafe permissions on homedir '/root/.gnupg'

 root /otro/libc_compilado 

 pacman-key --populate archlinux

/usr/bin/bash: warning: setlocale: LC_ALL: cannot change locale (es_AR.UTF-8): No such file or directory

==> Appending keys from archlinux.gpg...

==> Updating trust database...

gpg: next trustdb check due at 2026-10-21

 root /otro/libc_compilado 

**Gemini Replied:**

## Gemini ha dicho

Lo lograste. Esa `libc` personalizada fue el "caballo de Troya" necesario para recuperar el control, pero como ves, el sistema sigue detectando la ruta `/usr/lib/x86_64-linux-gnu/`. En Arch, esa ruta no debería existir y está causando que `gpg` prefiera la librería vieja/manual sobre la oficial.

Ahora que el **Keyring** está finalmente sano, vamos a normalizar el sistema para que deje de depender de tus copias manuales.

---

## 1. Limpieza de rutas conflictivas (Crucial)

Ejecuta esto para que los binarios dejen de encontrar la `libc` con el error de versión en rutas no estándar:

Bash

```
# Borra la ruta que está confundiendo a GPG
busybox.static rm -rf /usr/lib/x86_64-linux-gnu/
busybox.static rm -rf /lib/x86_64-linux-gnu/
```

## 2. Reinstalar la Glibc oficial por última vez

Sin esas rutas estorbando, `pacman` usará las librerías de `/usr/lib` (las estándar de Arch). Reinstala para asegurar que `libc.so.6` y `libm.so.6` sean las oficiales y estén sincronizadas entre sí:

Bash

```
pacman -S glibc --noconfirm
```

## 3. Eliminar el binario fantasma que causa el error de iconvconfig

El error `iconvconfig: required file not found` ocurre porque tienes un binario en `/usr/local` que no debería estar ahí o apunta a nada. Vamos a quitarlo para que la instalación de paquetes sea limpia:

Bash

```
rm /usr/local/sbin/iconvconfig
```

## 4. Regenerar Locales (Adiós warnings)

Ahora que las librerías son las oficiales, los locales deberían generarse sin problemas:

Bash

```
locale-gen
```

---

## Diagnóstico de seguridad

Si después de borrar `/usr/lib/x86_64-linux-gnu/`, el comando `pacman -Syu` funciona sin errores de "version not found", tu sistema está **100% recuperado**.

El hecho de que `pacman-key --populate` haya terminado con éxito es la prueba de fuego de que el sistema ya confía en sí mismo.

**¿Te gustaría que verifiquemos si el kernel necesita ser actualizado o si el `initramfs` quedó bien generado antes de reiniciar?** Podés probar con un `mkinitcpio -P`.

> **You asked:**

Has dicho

[manjaro @]# strace -f chroot /mnt

execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffdab7c17d0 /* 23 vars */) = 0

brk(NULL) = 0x55d2b4978000

access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f4b24df0000

close(3) = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

```
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f4b24dee000

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f4b24bfe000

mmap(0x7f4b24c22000, 1507328, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7f4b24c22000

mmap(0x7f4b24d92000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x194000) = 0x7f4b24d92000

mmap(0x7f4b24de0000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7f4b24de0000

mmap(0x7f4b24de6000, 31832, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f4b24de6000

close(3)                                = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f4b24bfb000

arch_prctl(ARCH_SET_FS, 0x7f4b24bfb740) = 0

set_tid_address(0x7f4b24bfba10)         = 160371

set_robust_list(0x7f4b24bfba20, 24)     = 0

rseq(0x7f4b24bfb680, 0x20, 0, 0x53053053) = 0

mprotect(0x7f4b24de0000, 16384, PROT_READ) = 0

mprotect(0x55d275bab000, 4096, PROT_READ) = 0

mprotect(0x7f4b24e58000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f4b24df0000, 181523)          = 0

getrandom("\x8c\x63\x6d\xee\x77\x8d\x38\xf9", 8, GRND_NONBLOCK) = 8

brk(NULL)                               = 0x55d2b4978000

brk(0x55d2b4999000)                     = 0x55d2b4999000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0

mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f4b2490f000
```

close(3)  = 0

readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2"..., 1023) = 58

readlink("/run", 0x7ffccfcbf170, 1023)  = -1 EINVAL (Invalid argument)

readlink("/run/media", 0x7ffccfcbf170, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro", 0x7ffccfcbf170, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7ffccfcbf170, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7ffccfcbf170, 1023) = -1 EINVAL (Invalid argument)

faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK, AT_EACCESS) = 0

chroot("/mnt")  = 0

chdir("/")  = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7ffccfcbf7b0 /* 23 vars */) = -1 ENOENT (No such file or directory)

fcntl(1, F_GETFL)  = 0x2 (flags O_RDWR)

write(2, "chroot: ", 8chroot: )  = 8

write(2, "failed to run command \342\200\230/usr/bi"..., 41failed to run command ‘/usr/bin/bash’) = 41

write(2, ": No such file or directory", 27: No such file or directory) = 27

write(2, "\n", 1

)  = 1

close(1)  = 0

close(2)  = 0

exit_group(127)  = ?

+++ exited with 127 +++

**Gemini Replied:**

## Gemini ha dicho

El `strace` revela exactamente por qué el `chroot` falla al final:

C

```
chroot("/mnt") = 0
chdir("/") = 0
execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], ...) = -1 ENOENT (No such file or directory)
```

El comando `chroot` tiene éxito al cambiar la raíz, pero cuando intenta ejecutar el shell ( `/usr/bin/bash` ) **dentro** de esa nueva raíz, no lo encuentra. Esto puede pasar por dos motivos:

1. **Ruta incorrecta:** En tu sistema Arch, el binario podría estar en `/bin/bash` y no en `/usr/bin/bash` (o viceversa, aunque en Arch suelen ser el mismo lugar por el enlace simbólico).
2. **Librerías faltantes:** El error `ENOENT` en un `execve` a veces no significa que falte el binario, sino que falta el **intérprete dinámico** (el cargador `ld-linux-x86-64.so.2` ) que el binario necesita para arrancar.

## Cómo solucionarlo

Intenta forzar la ruta clásica de `/bin/bash` al entrar:

Bash

```
chroot /mnt /bin/bash
```

Si eso sigue dando "No such file or directory", verifica desde **fuera** del chroot si los archivos están donde deben estar:

Bash

```
# ¿Existe el ejecutable?
ls -l /mnt/bin/bash /mnt/usr/bin/bash

# ¿Existe el cargador de librerías? (Fundamental para que cualquier binario corra)
ls -l /mnt/lib64/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2
```

## El problema de los enlaces simbólicos

En Arch, `/bin` es un enlace a `/usr/bin`. Si por las reparaciones manuales ese enlace se rompió o se convirtió en una carpeta real vacía, el sistema no encontrará nada.

**Prueba esto para ver la estructura real:**

Bash

```
ls -ld /mnt/bin /mnt/lib /mnt/lib64
```

Si alguno de esos **no** es un enlace simbólico (deberían apuntar a `usr/bin`, `usr/lib`, etc.), ahí tienes el culpable.

---

## Si nada de eso funciona: El "Chroot de emergencia"

Si el Bash de Arch está roto, puedes intentar usar el Bash del USB **dentro** del entorno de Arch (solo para emergencias):

Bash

```
# Copia el bash que funciona (el del USB) al disco duro temporalmente
cp /usr/bin/bash /mnt/bash_repara
chroot /mnt /bash_repara
```

¿Qué te devuelven los comandos `ls -l` sobre `/mnt/bin/bash`? Eso nos dirá si el archivo realmente "no está" o si es un problema de enlaces rotos.

¿Te gustaría que probemos a reconstruir los enlaces simbólicos base de Arch? Sería: `ln -sf usr/bin /mnt/bin` `ln -sf usr/lib /mnt/lib` `ln -sf usr/lib /mnt/lib64`

> **You asked:**

Has dicho

[manjaro @]# chroot /mnt /bin/bash

chroot: failed to run command ‘/bin/bash’: No such file or directory

[manjaro @]# # ¿Existe el ejecutable?

ls -l /mnt/bin/bash /mnt/usr/bin/bash

# ¿Existe el cargador de librerías? (Fundamental para que cualquier binario corra)

ls -l /mnt/lib64/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2

-rwxr-xr-x 1 root root 1162312 dic 10 22:02 /mnt/bin/bash

-rwxr-xr-x 1 root root 1162312 dic 10 22:02 /mnt/usr/bin/bash

lrwxrwxrwx 1 root root 44 mar 1 20:10 /mnt/lib64/ld-linux-x86-64.so.2 -> ../lib/x86_64-linux-gnu/ld-linux-x86-64.so.2

-rwxr-xr-x 1 root root 234400 feb  9 11:31 /mnt/usr/lib/ld-linux-x86-64.so.2

[manjaro @]# ls -ld /mnt/bin /mnt/lib /mnt/lib64

lrwxrwxrwx 1 root root 7 mar 15 10:57 /mnt/bin -> usr/bin

lrwxrwxrwx 1 root root 7 mar 15 09:39 /mnt/lib -> usr/lib

lrwxrwxrwx 1 root root 9 mar 15 09:39 /mnt/lib64 -> usr/lib64

[manjaro @]# # Copia el bash que funciona (el del USB) al disco duro temporalmente

cp /usr/bin/bash /mnt/bash_repara

chroot /mnt /bash_repara

chroot: failed to run command ‘/bash_repara’: No such file or directory

> **Gemini Replied:**

## Gemini ha dicho

El misterio está resuelto. Mira este enlace simbólico que encontraste:

```
lrwxrwxrwx ... /mnt/lib64/ld-linux-x86-64.so.2 -> ../lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
```

**Ahí está el error fatal.** En Arch Linux, el cargador (`ld-linux`) **nunca** debe apuntar a `x86_64-linux-gnu` (esa es una estructura de Debian/Ubuntu). Cuando intentas ejecutar `/bin/bash` o `/bash_repara`, el kernel busca el intérprete en esa ruta inexistente y devuelve "No such file or directory", aunque el binario de Bash esté ahí frente a tus ojos.

## La Solución: Corregir el enlace del cargador

Necesitamos que `/lib64/ld-linux-x86-64.so.2` apunte al archivo real que tienes en `/usr/lib/`.

Sigue estos pasos desde el USB:

### 1. Elimina el enlace simbólico roto

Bash

```
rm /mnt/lib64/ld-linux-x86-64.so.2
```

### 2. Crea el enlace correcto (estilo Arch)

En Arch, el archivo en `lib64` debe apuntar directamente al que está en `usr/lib`:

Bash

```
ln -s ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

**3. Verifica el enlace**

Bash

```
ls -l /mnt/lib64/ld-linux-x86-64.so.2
```

Debe decir: `... /mnt/lib64/ld-linux-x86-64.so.2 -> ../usr/lib/ld-linux-x86-64.so.2`

---

## Intenta el chroot ahora

Una vez corregido el enlace, el kernel podrá encontrar el cargador y ejecutar Bash:

Bash

```
chroot /mnt /bin/bash
```

## ¿Por qué falló `/bash_repara`?

Incluso el Bash del USB es un binario dinámico. Al hacer `chroot`, ese binario busca su "motor" (el cargador) dentro de `/mnt/lib64/`. Como el enlace estaba apuntando a una carpeta de Debian que no existe en tu Arch, fallaba igual que el original.

**Si logras entrar:** Lo primero es limpiar cualquier rastro de esas carpetas extrañas para que no vuelvan a confundir a las herramientas de Arch:

Bash

```
# Dentro del chroot
rm -rf /lib/x86_64-linux-gnu
rm -rf /usr/lib/x86_64-linux-gnu
```

¿Pudiste entrar al prompt del chroot después de corregir el `ln -s`?

> **You asked:**

Has dicho

[manjaro @]# strace -f chroot /mnt

```
execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffee3fcba70 /* 23 vars */) = 0

brk(NULL)                               = 0x55c1a4747000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f374e8e8000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f374e8e6000

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f374e6f6000

mmap(0x7f374e71a000, 1507328, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7f374e71a000

mmap(0x7f374e88a000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x194000) = 0x7f374e88a000

mmap(0x7f374e8d8000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7f374e8d8000

mmap(0x7f374e8de000, 31832, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f374e8de000

close(3)                                = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f374e6f3000

arch_prctl(ARCH_SET_FS, 0x7f374e6f3740) = 0

set_tid_address(0x7f374e6f3a10)         = 160436

set_robust_list(0x7f374e6f3a20, 24)     = 0

rseq(0x7f374e6f3680, 0x20, 0, 0x53053053) = 0
```

```
mprotect(0x7f374e8d8000, 16384, PROT_READ) = 0

mprotect(0x55c198208000, 4096, PROT_READ) = 0

mprotect(0x7f374e950000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f374e8e8000, 181523)          = 0

getrandom("\x3a\x08\x4b\x5b\x36\xb6\x05\xb8", 8, GRND_NONBLOCK) = 8

brk(NULL)                               = 0x55c1a4747000

brk(0x55c1a4768000)                     = 0x55c1a4768000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0

mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f374e407000

close(3)                                = 0

readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2"..., 1023) = 58

readlink("/run", 0x7ffd963477b0, 1023)  = -1 EINVAL (Invalid argument)

readlink("/run/media", 0x7ffd963477b0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro", 0x7ffd963477b0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7ffd963477b0, 1023) = -1
EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7ffd963477b0, 1023) =
-1 EINVAL (Invalid argument)

faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK,
AT_EACCESS) = 0

chroot("/mnt")                          = 0

chdir("/")                              = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7ffd96347df0 /* 23 vars */) = -1 ENOENT (No such file
or directory)

fcntl(1, F_GETFL)                       = 0x2 (flags O_RDWR)

write(2, "chroot: ", 8chroot: )                 = 8

write(2, "failed to run command \342\200\230/usr/bi"..., 41failed to run command ‘/usr/bin/bash’) = 41
```

write(2, ": No such file or directory", 27: No such file or directory) = 27

write(2, "\n", 1

)  = 1

close(1)  = 0

close(2)  = 0

exit_group(127)  = ?

+++ exited with 127 +++

**Gemini Replied:**

## Gemini ha dicho

El `strace` sigue confirmando que el `chroot` funciona (retorna `0`), pero el sistema falla al intentar ejecutar `/usr/bin/bash`.

Como vimos antes, esto sucede porque el **cargador dinámico** (el "motor" que inicia los binarios) sigue apuntando a una ruta inexistente o errónea. El error `ENOENT` en el `execve` de Bash es el kernel diciendo: *"Encuentro el archivo Bash, pero no encuentro el cargador que Bash necesita para arrancar"*.

Vamos a solucionar esto de raíz recreando los enlaces simbólicos esenciales de Arch Linux desde afuera.

## 1. Corregir el enlace del cargador dinámico (LD)

En Arch, `/lib64/ld-linux-x86-64.so.2` debe apuntar a la ubicación real en `usr/lib`. Vamos a forzarlo:

Bash

```
# Eliminar cualquier enlace roto previo
rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Crear el enlace correcto (relativo)
ln -s ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

## 2. Verificar la "Santa Trinidad" de enlaces en Arch

Arch Linux requiere que `/bin`, `/lib` y `/lib64` sean enlaces simbólicos hacia `/usr`. Si alguno de estos se convirtió en una carpeta real durante tus pruebas con archivos de Debian, nada funcionará. Ejecuta esto y verifica la salida:

Bash

```
ls -ld /mnt/bin /mnt/lib /mnt/lib64
```

**Si ves carpetas ( `drwxr-xr-x` ) en lugar de enlaces ( `lrwxrwxrwx` ), arréglalo así:** *(Ten cuidado: solo haz esto si NO son enlaces)*

Bash

```
# Ejemplo si /mnt/lib64 fuera una carpeta real:
# mv /mnt/lib64/* /mnt/usr/lib/ 2>/dev/null
# rm -rf /mnt/lib64
# ln -s usr/lib /mnt/lib64
```

## 3. Prueba de ejecución "Static"

Si el Bash normal sigue fallando, intenta entrar usando el `busybox` estático que tienes, ya que este no depende de ninguna librería ni cargador:

Bash

```
chroot /mnt /bin/busybox.static sh
```

Si esto funciona, ya estás dentro y podemos arreglar los enlaces internos cómodamente.

---

## ¿Por qué el `strace` muestra éxito en el `chroot` pero falla el comando?

Cuando ejecutas `chroot /mnt /usr/bin/bash`, ocurren dos pasos:

1. El sistema cambia la raíz a `/mnt` (Éxito: `chroot("/mnt") = 0`).
2. El sistema intenta ejecutar `/usr/bin/bash`. El kernel lee la cabecera ELF de Bash y busca la ruta del "program interpreter". Si esa ruta apunta a `/lib/x86_64-linux-gnu/...` (estilo Debian) y esa carpeta no existe dentro de tu nueva raíz, el `execve` devuelve `ENOENT`.

**¿Qué te devuelve `ls -l /mnt/lib64/ld-linux-x86-64.so.2` ahora mismo?** Si apunta a cualquier cosa que diga `x86_64-linux-gnu`, esa es la razón del fallo.

¿Quieres que intentemos recrear todos los enlaces base de una vez para asegurar que el sistema sea un Arch puro de nuevo?

> **You asked:**

Has dicho

```
[manjaro @]# strace -f chroot /mnt

execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffe35de1440 /* 23 vars */) = 0

brk(NULL)                               = 0x5628b609e000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f854edc7000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f854edc5000

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f854ebd5000

mmap(0x7f854ebf9000, 1507328, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7f854ebf9000

mmap(0x7f854ed69000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x194000) = 0x7f854ed69000

mmap(0x7f854edb7000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7f854edb7000

mmap(0x7f854edbd000, 31832, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f854edbd000

close(3)                                = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f854ebd2000

arch_prctl(ARCH_SET_FS, 0x7f854ebd2740) = 0

set_tid_address(0x7f854ebd2a10)         = 160513
```

```
set_robust_list(0x7f854ebd2a20, 24)     = 0

rseq(0x7f854ebd2680, 0x20, 0, 0x53053053) = 0

mprotect(0x7f854edb7000, 16384, PROT_READ) = 0

mprotect(0x56287ba19000, 4096, PROT_READ) = 0

mprotect(0x7f854ee2f000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f854edc7000, 181523)          = 0

getrandom("\x92\xc7\xc9\xb5\x73\xca\x1a\xc7", 8, GRND_NONBLOCK) = 8

brk(NULL)                               = 0x5628b609e000

brk(0x5628b60bf000)                     = 0x5628b60bf000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0

mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f854e8e6000

close(3)                                = 0

readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2"..., 1023) = 58

readlink("/run", 0x7ffcecb175b0, 1023)  = -1 EINVAL (Invalid argument)

readlink("/run/media", 0x7ffcecb175b0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro", 0x7ffcecb175b0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7ffcecb175b0, 1023) = -1
EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7ffcecb175b0, 1023) = -1
EINVAL (Invalid argument)

faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK,
AT_EACCESS) = 0

chroot("/mnt")                          = 0

chdir("/")                              = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7ffcecb17bf0 /* 23 vars */) = -1 ENOENT (No such file
or directory)

fcntl(1, F_GETFL)                       = 0x2 (flags O_RDWR)
```

write(2, "chroot: ", 8chroot: )  = 8

write(2, "failed to run command \342\200\230/usr/bi"..., 41failed to run command ‘/usr/bin/bash’) = 41

write(2, ": No such file or directory", 27: No such file or directory) = 27

write(2, "\n", 1

)  = 1

close(1)  = 0

close(2)  = 0

exit_group(127)  = ?

+++ exited with 127 +++

**Gemini Replied:**

## Gemini ha dicho

El `strace` sigue gritando lo mismo: **el sistema no encuentra el cargador dinámico** (linker) necesario para arrancar Bash. Aunque el archivo `/usr/bin/bash` existe, el "camino" hacia su motor está roto.

En Arch Linux, la estructura de archivos es extremadamente específica. Si el enlace simbólico en `/lib64` está apuntando a una ruta de Debian (como vimos antes con `x86_64-linux-gnu`), el kernel falla al intentar iniciar el proceso.

## El Diagnóstico Final

Cuando el kernel ve que quieres ejecutar Bash, busca el campo `INTERP` en el binario. En sistemas de 64 bits, ese campo suele decir `/lib64/ld-linux-x86-64.so.2`. Si ese archivo es un link roto, obtienes el error `ENOENT` (No such file or directory) que ves en el `strace`.

---

## La Solución Maestra (Ejecutar desde el Live USB)

Vamos a reconstruir los enlaces vitales de Arch. Ejecuta estos comandos uno por uno:

### 1. Arreglar el Linker de 64 bits (El más importante)

Bash

```
# Borramos el link actual (que probablemente apunta a una ruta de Debian)
rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el link correcto hacia la librería de Arch
ln -s ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

### 2. Asegurar la Estructura de Enlaces de Arch

Arch unifica todo en `/usr`. Asegurémonos de que `/mnt/lib` y `/mnt/bin` no se hayan convertido en carpetas reales por accidente:

Bash

```
# Si /mnt/lib o /mnt/bin son carpetas reales, esto fallará, lo cual es bueno saber.
ls -ld /mnt/bin /mnt/lib /mnt/lib64
```

- **Si NO son links**, avísame, porque hay que mover los archivos antes de borrarlos.
- **Si SON links**, asegúrate de que apunten a `usr/bin` y `usr/lib` respectivamente.

### 3. El Intento de Entrada "Blindado"

Si después de arreglar el link en `/lib64` el comando `chroot /mnt` sigue fallando, intenta entrar usando el **Bash del propio USB** pero mapeado dentro de la jaula:

Bash

```
# Copiamos el bash que sabemos que funciona al directorio raíz de Arch
cp /usr/bin/bash /mnt/bash_emergencia

# Intentamos chrootear usando ese binario
chroot /mnt /bash_emergencia
```

---

## ¿Por qué sigue fallando si ya lo habías arreglado?

Es probable que al intentar "arreglar" cosas moviendo archivos de `/lib64` o `/usr/lib`, el sistema haya regenerado links automáticos o hayas copiado una versión de `ld-linux` que todavía busca carpetas tipo `/lib/x86_64-linux-gnu`.

**¿Qué te devuelve `ls -l /mnt/lib64/ld-linux-x86-64.so.2` justo ahora?** Si el destino del link no empieza con `../usr/lib/`, no va a funcionar.

¿Querés que te pase un script corto para resetear todos los enlaces simbólicos base de Arch de una sola vez? Sería lo más seguro para limpiar cualquier rastro de la estructura de Debian.

**You asked:**

Has dicho

[manjaro @]# chroot /mnt

chroot: failed to run command ‘/usr/bin/bash’: No such file or directory

[manjaro @]# strace -f chroot /mnt

execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffe2916c0e0 /* 23 vars */) = 0

brk(NULL)  = 0x559c9312e000

access("/etc/ld.so.preload", R_OK)  = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fd33f150000

close(3)  = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd33f14e000

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fd33ef5e000

mmap(0x7fd33ef82000, 1507328, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7fd33ef82000

mmap(0x7fd33f0f2000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x194000) = 0x7fd33f0f2000

mmap(0x7fd33f140000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7fd33f140000

mmap(0x7fd33f146000, 31832, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fd33f146000

close(3)  = 0

```
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fd33ef5b000

arch_prctl(ARCH_SET_FS, 0x7fd33ef5b740) = 0

set_tid_address(0x7fd33ef5ba10)         = 160560

set_robust_list(0x7fd33ef5ba20, 24)     = 0

rseq(0x7fd33ef5b680, 0x20, 0, 0x53053053) = 0

mprotect(0x7fd33f140000, 16384, PROT_READ) = 0

mprotect(0x559c73f7b000, 4096, PROT_READ) = 0

mprotect(0x7fd33f1b8000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7fd33f150000, 181523)          = 0

getrandom("\x6f\xe5\x14\xde\x6f\xc1\x5e\x18", 8, GRND_NONBLOCK) = 8

brk(NULL)                               = 0x559c9312e000

brk(0x559c9314f000)                     = 0x559c9314f000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0

mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fd33ec6f000

close(3)                                = 0

readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2"..., 1023) = 58

readlink("/run", 0x7ffff9d7e460, 1023)  = -1 EINVAL (Invalid argument)

readlink("/run/media", 0x7ffff9d7e460, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro", 0x7ffff9d7e460, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7ffff9d7e460, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7ffff9d7e460, 1023) = -1 EINVAL (Invalid argument)

faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK, AT_EACCESS) = 0

chroot("/mnt")                          = 0
```

chdir("/") = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7ffff9d7eaa0 /* 23 vars */) = -1 ENOENT (No such file or directory)

fcntl(1, F_GETFL) = 0x2 (flags O_RDWR)

write(2, "chroot: ", 8chroot: ) = 8

write(2, "failed to run command \342\200\230/usr/bi"..., 41failed to run command ‘/usr/bin/bash’) = 41

write(2, ": No such file or directory", 27: No such file or directory) = 27

write(2, "\n", 1

) = 1

close(1) = 0

close(2) = 0

exit_group(127) = ?

+++ exited with 127 +++

**Gemini Replied:**

## Gemini ha dicho

Ese `strace` es frustrante porque confirma que el `chroot` funciona pero el sistema "miente" diciendo que Bash no existe. Como ya vimos, el culpable no es el binario de Bash, sino el **linker dinámico** que Bash busca apenas intenta arrancar.

Si el linker está mal direccionado (por ejemplo, apuntando todavía a algo que diga `/lib/x86_64-linux-gnu/`), el kernel devuelve `ENOENT` porque no encuentra el "motor" para encender el proceso.

---

## La Limpieza Definitiva

Vamos a resetear los enlaces simbólicos de la base del sistema desde el Live USB. Esto eliminará cualquier rastro de la estructura de directorios estilo Debian que se coló en tu Arch.

Ejecutá estos comandos en orden:

### 1. Corregir el Linker (Paso Crítico)

Bash

```
# Borramos el enlace que está causando el "No such file or directory"
rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el enlace oficial de Arch
ln -sf ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

### 2. Asegurar la Unificación de `/usr`

Arch Linux espera que `/bin`, `/lib` y `/lib64` sean solo accesos directos a `/usr`. Si alguno se convirtió en una carpeta real con archivos adentro, el sistema se rompe.

Bash

```
# Verificá qué son estos archivos
ls -ld /mnt/bin /mnt/lib /mnt/lib64
```

- **Si alguno dice `drwxr-xr-x` (directorio):** Tenemos un problema. Hay que mover los archivos a `/mnt/usr/lib` (o `bin`), borrar la carpeta y crear el link.
- **Si dicen `lrwxrwxrwx` (enlace):** Asegurate de que apunten a `usr/bin`, `usr/lib` y `usr/lib` respectivamente.

### 3. El "Caballo de Troya" (Si Bash sigue fallando)

Si después del paso 1 seguís sin poder entrar, vamos a usar el Bash del USB pero engañando al sistema para que use las librerías que *sabemos* que funcionan:

Bash

```
# Copiamos el Bash del USB (que es sano) a la raíz del disco
cp /usr/bin/bash /mnt/bash_repara

# Intentamos entrar forzando ese binario
chroot /mnt /bash_repara
```

---

## ¿Por qué pasa esto?

Cuando ejecutás un binario dinámico (como Bash), el kernel lee una sección llamada `.interp`. En Arch, esa sección dice: `/lib64/ld-linux-x86-64.so.2`.

1. Si ese archivo no existe -> `ENOENT`.
2. Si ese archivo es un link a una ruta que no existe (como `/lib/x86_64-linux-gnu/...`) -> `ENOENT`.

**¿Qué te devuelve `readelf -l /mnt/usr/bin/bash | grep interpreter`?** Eso nos va a decir exactamente qué ruta está buscando Bash para arrancar.

¿Querés que probemos a reconstruir el enlace de `/lib` también? A veces se rompe y apunta a la ruta de Debian por error. Sería: `ln -sf usr/lib /mnt/lib`.

> **You asked:**

Has dicho

rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el enlace oficial de Arch

ln -sf ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2

[manjaro @]# # Verificá qué son estos archivos

ls -ld /mnt/bin /mnt/lib /mnt/lib64

lrwxrwxrwx 1 root root 7 mar 15 10:57 /mnt/bin -> usr/bin

lrwxrwxrwx 1 root root 7 mar 15 09:39 /mnt/lib -> usr/lib

lrwxrwxrwx 1 root root 9 mar 15 09:39 /mnt/lib64 -> usr/lib64

[manjaro @]# # Copiamos el Bash del USB (que es sano) a la raíz del disco

cp /usr/bin/bash /mnt/bash_repara

# Intentamos entrar forzando ese binario

chroot /mnt /bash_repara

chroot: failed to run command ‘/bash_repara’: No such file or directory

> **Gemini Replied:**

## Gemini ha dicho

Esto ya es personal. El `ENOENT` (No such file or directory) con un binario copiado directamente del USB (`bash_repara`) confirma al 100% que el problema no es el archivo, sino el **punto de montaje del intérprete**.

El kernel busca el cargador en `/lib64/ld-linux-x86-64.so.2` **dentro del chroot**. Si ese enlace no apunta a un archivo real y válido, el `execve` fallará siempre.

## El Diagnóstico de Precisión

Vamos a ver exactamente qué está buscando el sistema. Tira este comando desde el Manjaro (afuera):

Bash

```
readelf -l /mnt/bash_repara | grep interpreter
```

Si la salida dice algo como `/lib64/ld-linux-x86-64.so.2` (que es lo normal), entonces el problema es que el enlace que creamos en `/mnt/lib64/` no está siendo "visto" correctamente por el kernel o el destino es erróneo.

---

## La Solución de Fuerza Bruta (Paso a Paso)

Hagamos que el enlace sea **absoluto** dentro de la jaula, para que no haya margen de error con los `..` (puntos relativos):

### 1. Corregir el link de forma absoluta

Bash

```
# Borramos el link relativo que podría estar fallando
rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el link apuntando a la ruta que tendrá DENTRO del chroot
# (Aunque estemos afuera, el destino del link debe ser la ruta final)
ln -sf /usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

### 2. Verificar que el destino sea un ARCHIVO REAL

Asegurate de que no hayamos borrado el cargador real por accidente:

Bash

```
ls -lh /mnt/usr/lib/ld-linux-x86-64.so.2
```

Si ese comando te da "No such file", ¡ahí está! Tenemos que sacar el cargador del paquete de glibc que compilaste o del USB.

### 3. El último recurso: El binario estático

Si tenés el `busybox.static` que usamos antes en `/arch_repair`, copialo a la raíz de `/mnt`. Los binarios estáticos **no usan linker**, así que tienen que funcionar sí o sí.

Bash

```
cp /mnt/arch_repair/busybox.static /mnt/busybox_emergencia
chroot /mnt /busybox_emergencia sh
```

---

## ¿Por qué sigue fallando el chroot?

Hay una posibilidad técnica: si montaste `/mnt` con la opción `noexec` (poco probable pero posible), ningún binario va a arrancar.

Revisá cómo está montado:

Bash

```
mount | grep /mnt
```

**Pasame el resultado de:**

1. `readelf -l /mnt/bash_repara | grep interpreter`
2. `ls -lh /mnt/usr/lib/ld-linux-x86-64.so.2`

Si el archivo en `/usr/lib/` no está, podemos copiar el del USB: `cp /lib64/ld-linux-x86-64.so.2 /mnt/usr/lib/` (esto suele ser peligroso por versiones, pero peor es no tener nada).

¿Qué te devolvió el `readelf`?

> **You asked:**

Has dicho

[manjaro @]# readelf -l /mnt/bash_repara | grep interpreter

[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]

[manjaro @]# # Borramos el link relativo que podría estar fallando

rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el link apuntando a la ruta que tendrá DENTRO del chroot

# (Aunque estemos afuera, el destino del link debe ser la ruta final)

ln -sf /usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2

[manjaro @]# ls -lh /mnt/usr/lib/ld-linux-x86-64.so.2

-rwxr-xr-x 1 root root 229K feb  9 11:31 /mnt/usr/lib/ld-linux-x86-64.so.2

[manjaro @]# cp /mnt/arch_repair/busybox.static /mnt/busybox_emergencia

chroot /mnt /busybox_emergencia sh

cp: cannot stat '/mnt/arch_repair/busybox.static': No such file or directory

chroot: failed to run command ‘/busybox_emergencia’: No such file or directory

**Gemini Replied:**

## Gemini ha dicho

Esto es una situación de manual de "dependencias rotas". El `readelf` nos confirma que `bash_repara` busca su cargador en `/lib64/ld-linux-x86-64.so.2`. El archivo real existe en `/mnt/usr/lib/`. El enlace simbólico está bien creado.

**¿Entonces por qué carajo sigue dando `ENOENT`?**

La respuesta está en la propia librería `ld-linux-x86-64.so.2` que tienes en `/mnt/usr/lib/`. Al ser una versión de Arch (o de la compilación que falló), ella misma tiene dependencias internas o espera una estructura de símbolos ( `GLIBC_ABI_DT_X86_64_PLT` ) que el kernel del Manjaro no está pudiendo satisfacer al intentar "encender" el binario en el entorno `chroot`.

Como el `busybox.static` que tenías desapareció (seguramente quedó en una carpeta que borramos o movimos), vamos a usar el del Live USB, que es nuestra última carta para entrar "por la fuerza".

## El Plan de Rescate Final

**1. Meter un binario estático que NO dependa de nadie**

El `busybox` del Live USB de Manjaro suele ser dinámico, pero vamos a intentar copiarlo y ver si arranca. Si no, usaremos el `cp` del USB para meter un Bash que sepamos que es compatible.

Bash

```
# Buscamos un busybox en el USB y lo metemos en la raíz de Arch
cp $(which busybox) /mnt/busybox_usb

# Intentamos entrar con él
chroot /mnt /busybox_usb sh
```

### 2. Si eso falla: "Inyección" del cargador del USB

A veces, el cargador que tienes en el disco ( `/mnt/usr/lib/ld-linux...` ) está tan corrupto que ni siquiera puede leer el Bash. Vamos a pisarlo temporalmente con el cargador del USB (que sabemos que funciona con este Kernel).

Bash

```
# Hacé un backup del cargador actual por las dudas
cp /mnt/usr/lib/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2.bak

# Copiá el cargador del USB al disco
cp /lib64/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2
```

### 3. El intento definitivo de Bash

Ahora probamos de nuevo el chroot:

Bash

```
chroot /mnt /usr/bin/bash
```

## ¿Por qué el `cp` de busybox falló?

Me preocupa que dijeras `cp: cannot stat '/mnt/arch_repair/busybox.static': No such file or directory`. Si esa carpeta desapareció, es probable que hayamos movido archivos importantes a `/otro/libc_compilado` o similar.

**Hacé un `ls /mnt` rápido** para ver dónde quedaron tus archivos de reparación. Necesitamos ese `busybox.static` o, en su defecto, usar el método de copiar el cargador del USB del paso 2.

**Si lográs entrar con el cargador del USB:** No te asustes si ves errores de locales o warnings. Lo primero que vamos a hacer es:

1. `pacman -Sy glibc` (Para que Arch pise el cargador del USB con el oficial).
2. `ldconfig`.

¿Qué archivos ves en la raíz de `/mnt` ahora mismo?

**You asked:**

Has dicho

[manjaro @]# # Buscamos un busybox en el USB y lo metemos en la raíz de Arch

cp $(which busybox) /mnt/busybox_usb

# Intentamos entrar con él

chroot /mnt /busybox_usb sh

which: no busybox in (/usr/local/sbin:/usr/local/bin:/usr/bin)

cp: missing destination file operand after '/mnt/busybox_usb'

Try 'cp --help' for more information.

chroot: failed to run command ‘/busybox_usb’: No such file or directory

[manjaro @]# # Hacé un backup del cargador actual por las dudas

cp /mnt/usr/lib/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2.bak

# Copiá el cargador del USB al disco

cp /lib64/ld-linux-x86-64.so.2 /mnt/usr/lib/ld-linux-x86-64.so.2

[manjaro @]# chroot /mnt /usr/bin/bash

Segmentation fault (core dumped)

[manjaro @]# chroot /mnt

Segmentation fault (core dumped)

[manjaro @]# strace -s100 -ff chroot /mnt

execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffe8794d778 /* 23 vars */) = 0

brk(NULL)  = 0x560cd0d7c000

access("/etc/ld.so.preload", R_OK)  = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f331746b000

```
close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0@\0\0\0\0\0\0\0x\215\36\0\0\0\0\0\0\0\0
@\08\0\17\0@\0

?\0>\0\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0"..., 832) = 832

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"...,
840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f3317469000

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"...,
840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f3317279000

mmap(0x7f331729d000, 1507328, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7f331729d000

mmap(0x7f331740d000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x194000) = 0x7f331740d000

mmap(0x7f331745b000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7f331745b000

mmap(0x7f3317461000, 31832, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f3317461000

close(3)                                = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f3317276000

arch_prctl(ARCH_SET_FS, 0x7f3317276740) = 0

set_tid_address(0x7f3317276a10)         = 160688
```

```
set_robust_list(0x7f3317276a20, 24)     = 0

rseq(0x7f3317276680, 0x20, 0, 0x53053053) = 0

mprotect(0x7f331745b000, 16384, PROT_READ) = 0

mprotect(0x560c9bc74000, 4096, PROT_READ) = 0

mprotect(0x7f33174d3000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f331746b000, 181523)          = 0

getrandom("\x7e\xec\x55\xda\x30\x7b\x32\x7b", 8, GRND_NONBLOCK) = 8

brk(NULL)                               = 0x560cd0d7c000

brk(0x560cd0d9d000)                     = 0x560cd0d9d000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0

mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f3316f8a000

close(3)                                = 0

readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", 1023) = 58

readlink("/run", 0x7ffda9ecbfb0, 1023)  = -1 EINVAL (Invalid argument)

readlink("/run/media", 0x7ffda9ecbfb0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro", 0x7ffda9ecbfb0, 1023) = -1 EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7ffda9ecbfb0, 1023) = -1
EINVAL (Invalid argument)

readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7ffda9ecbfb0, 1023) = -1
EINVAL (Invalid argument)

faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK,
AT_EACCESS) = 0

chroot("/mnt")                          = 0

chdir("/")                              = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7ffda9ecc5f0 /* 23 vars */) = 0

brk(NULL)                               = 0x5583d6423000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=336155, ...}) = 0

mmap(NULL, 336155, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fb5be7b8000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libreadline.so.8", O_RDONLY|O_CLOEXEC) = 3

read(3,
"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\230\225\5\0\0\0\0\0\0\0\0
\0@\08\0\v\0

@\0\32\0\31\0\1\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0
```

```
\0\0\4\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"..., 840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2010168, ...}) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0\0\0\0\3\0

\0\0\4\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"..., 840, 64) = 840

mmap(NULL, 2034544, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fb5be56a000

mmap(0x7fb5be58e000, 1511424, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7fb5be58e000

mmap(0x7fb5be6ff000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x195000) = 0x7fb5be6ff000

mmap(0x7fb5be74d000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e2000) = 0x7fb5be74d000

mmap(0x7fb5be753000, 31600, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fb5be753000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libncursesw.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/libncursesw.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\20\366\6\0\0\0\0\0\0\0\0\0@\08\0\v\0@

\0\33\0\32\0\1\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0
```

```
close(3)                                = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fb5be4f7000

arch_prctl(ARCH_SET_FS, 0x7fb5be4f7b80) = 0

set_tid_address(0x7fb5be4f7e50)         = 160688

set_robust_list(0x7fb5be4f7e60, 24)     = 0

rseq(0x7fb5be4f7ae0, 0x20, 0, 0x53053053) = 0

mprotect(0x7fb5be74d000, 16384, PROT_READ) = 0

mprotect(0x7fb5be564000, 20480, PROT_READ) = 0

mprotect(0x7fb5be7ac000, 12288, PROT_READ) = 0

mprotect(0x5583d17da000, 12288, PROT_READ) = 0

mprotect(0x7fb5be846000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

getrandom("\x0b\xd1\xed\xb6\x39\x44\xc7\xfc", 8, GRND_NONBLOCK) = 8

munmap(0x7fb5be7b8000, 336155)          = 0

openat(AT_FDCWD, "/dev/tty", O_RDWR|O_NONBLOCK) = 3

close(3)                                = 0

brk(NULL)                               = 0x5583d6423000

brk(0x5583d6444000)                     = 0x5583d6444000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No
such file or directory)

openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=2998, ...}) = 0

read(3, "# Locale name alias data base.\n# Copyright (C) 1996-2026 Free Software Foundation,
Inc.\n#\n# This pro"..., 4096)

= 2998

read(3, "", 4096)                       = 0

close(3)                                = 0
```

openat(AT_FDCWD, "/usr/lib/locale/es_ES.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory

)

openat(AT_FDCWD, "/usr/lib/locale/es_ES.utf8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es_ES/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es.utf8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

getresuid([0], [0], [0])  = 0

getresgid([0], [0], [0])  = 0

rt_sigprocmask(SIG_BLOCK, NULL, [], 8)  = 0

--- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_ACCERR, si_addr=0x7fb5be849ef0} ---

+++ killed by SIGSEGV (core dumped) +++

Segmentation fault (core dumped)

**Gemini Replied:**

## Gemini ha dicho

¡Avancemos! El `strace` es **revelador**. El `Segmentation fault` ocurre justo después de que Bash intenta cargar las locales y verificar los IDs de usuario.

Lo más importante: **Bash ya no da "No such file or directory"**. Eso significa que el cargador ( `ld-linux` ) que copiaste del USB está funcionando y Bash logra arrancar. El problema ahora es que las librerías dinámicas de tu sistema Arch ( `libreadline` , `libc` , `libncursesw` ) están teniendo un conflicto violento con el cargador del USB o están corruptas por los experimentos previos con Debian.

El error específico `si_addr=0x7fb5be849ef0` sugiere un fallo de acceso a memoria en el área de las librerías.

# El Plan: "Cirugía Estática"

Como los binarios dinámicos están explotando, necesitamos entrar con algo que sea **estático** (que no use librerías externas). Como no encontramos el `busybox` del USB, vamos a usar el de Manjaro que suele estar en una ruta que omitimos.

### 1. Encontrar y meter un Busybox Estático

En Manjaro/Arch, suele haber una versión estática en `/usr/bin/busybox` o podemos usar el de la imagen de rescate. Intentá esto:

Bash

```
# Buscamos el binario real en el sistema live
cp /usr/bin/busybox /mnt/busybox_emergencia

# Intentamos entrar. Busybox es muy noble y suele aguantar fallos de librerías
chroot /mnt /busybox_emergencia sh
```

### 2. Si lográs entrar (aunque sea con errores)

Si ese `sh` arranca, tenemos una oportunidad. Lo primero es intentar que `pacman` (que es dinámico y probablemente falle) se arregle, o usar `pacman` **desde afuera** apuntando a la raíz.

### 3. Reparación desde AFUERA (La opción más segura ahora)

Si el chroot sigue dando Segfault, no fuerces la entrada. Vamos a usar el poder del Live USB para arreglar el disco:

Bash

```
# Reinstalamos glibc y bash en el disco usando el pacman del USB
# Esto pisará las librerías rotas con versiones frescas de Arch
pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S glibc bash coreutils
```

**Nota:** Si te pide confirmar llaves o directorios, decile que sí. Esto restaurará los binarios y el cargador oficial de Arch en `/mnt/usr/lib/`.

---

# ¿Por qué falló Bash con Segfault?

Mirando el log:

1. `execve("/usr/bin/bash", ...)` -> **ÉXITO** (El cargador del USB lo inició).
2. Carga `libreadline.so.8` -> **ÉXITO**.

3. Carga `libc.so.6` -> **ÉXITO**.
4. Busca locales en `/usr/lib/locale/...` -> **FALLO** (No las encuentra).
5. **SIGSEGV**: Probablemente una de esas librerías ( `libc` o `readline` ) es de una versión incompatible con el cargador que copiaste o tiene un "mismatch" de símbolos.

**Próximo paso:** Intentá el comando del punto 3 ( `pacman --root /mnt ...` ). Es la forma más limpia de resetear el estado de las librerías sin tener que estar adentro del sistema roto.

¿Te tiró algún error el `pacman --root` ?

> **You asked:**

Has dicho

Otro problema que tengo es que no me permite abrir otra pestaña la terminal de zsh.

> **Gemini Replied:**

## Gemini ha dicho

Ese problema de las pestañas en `zsh` suele estar directamente relacionado con lo que vimos en el `strace` : el sistema de archivos está en un estado inconsistente y los procesos "hijos" no pueden nacer porque fallan al cargar sus librerías básicas o no encuentran el binario del shell.

Si estás intentando abrir una pestaña de una terminal que apunta al `/mnt` (el sistema roto), va a fallar por el mismo **Segfault** o **ENOENT** que vimos antes. Si es la terminal del Live USB (Manjaro) la que no te deja abrir pestañas, es probable que se haya quedado sin espacio en memoria ( `/tmp` lleno) o que el demonio de la terminal esté bloqueado por los procesos hijos que "crashearon".

---

## Cómo desatascar la terminal y reparar el sistema

No perdamos tiempo intentando entrar a un `chroot` que está escupiendo errores de memoria. Vamos a reparar la base desde afuera usando el `pacman` del USB.

## 1. Reparación de emergencia (Desde el USB)

Ejecutá esto para reinstalar los paquetes críticos que manejan las librerías y el shell. Esto debería corregir los Segfaults:

Bash

```
# Reinstala la base de Arch en el disco duro
pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg -S glibc bash coreutils zsh
```

## 2. Por qué no abren las pestañas (ZSH)

Si el error es en el sistema que intentás reparar, es porque `zsh` depende de muchas más librerías que `bash` (como `libcap`, `libgdbm`, etc.). Si una sola está mal, el proceso muere antes de que la pestaña pueda siquiera dibujarse.

**Para recuperar el control de tus pestañas en el Live USB:**

- Intentá resetear el entorno: `killall-9 zsh` (Ojo: esto cerrará tus sesiones actuales).
- Verificá el espacio en el sistema de archivos temporal del USB: `df -h /tmp`. Si está al 100%, las pestañas no pueden crear los sockets necesarios para comunicarse.

## 3. El "Linker" definitivo

Después de correr el `pacman --root`, asegurate de que el enlace del cargador volvió a su estado original de Arch. **No uses más el del USB si ya reinstalaste glibc**:

Bash

```
ln -sf ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

---

## Siguiente paso recomendado

Una vez que corras el `pacman --root`, intentá de nuevo el chroot simple: `chroot /mnt /usr/bin/bash`

**¿Te tiró algún error de "llaves" (GPG keys) o de base de datos el comando de `pacman --root`?** Si es así, avisame y lo saltamos con `--overwrite "*"`.

¿Querés que intentemos reinstalar `zsh` también para que vuelvan a funcionar las pestañas una vez que logres entrar?

> **You asked:**

Has dicho

[manjaro @]# strace -s100 -ff chroot /mnt

execve("/usr/bin/chroot", ["chroot", "/mnt"], 0x7ffd32e02518 /* 23 vars */) = 0

```
brk(NULL)                               = 0x5647e214e000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=181523, ...}) = 0

mmap(NULL, 181523, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fc429522000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0px\2\0\0\0\0\0@\0\0\0\0\0\0\0x\215\36\0\0\0\0\0\0\0\0
@\08\0\17\0@\0

?\0>\0\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0"..., 832) = 832

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"...,
840, 64) = 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2006328, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7fc429520000

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\200\31\36\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"...,
840, 64) = 840

mmap(NULL, 2030680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fc429330000

mmap(0x7fc429354000, 1507328, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7fc429354000

mmap(0x7fc4294c4000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x194000) = 0x7fc4294c4000

mmap(0x7fc429512000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e1000) = 0x7fc429512000

mmap(0x7fc429518000, 31832, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fc429518000
```

```
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fc42932d000
arch_prctl(ARCH_SET_FS, 0x7fc42932d740) = 0
set_tid_address(0x7fc42932da10)         = 160869
set_robust_list(0x7fc42932da20, 24)     = 0
rseq(0x7fc42932d680, 0x20, 0, 0x53053053) = 0
mprotect(0x7fc429512000, 16384, PROT_READ) = 0
mprotect(0x5647e0986000, 4096, PROT_READ) = 0
mprotect(0x7fc42958a000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7fc429522000, 181523)          = 0
getrandom("\x2d\xed\x49\xca\x37\xc6\xd9\xf7", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5647e214e000
brk(0x5647e216f000)                     = 0x5647e216f000
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=3062944, ...}) = 0
mmap(NULL, 3062944, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fc429041000
close(3)                                = 0
readlink("/mnt", "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", 1023) = 58
readlink("/run", 0x7fff3dc98540, 1023)  = -1 EINVAL (Invalid argument)
readlink("/run/media", 0x7fff3dc98540, 1023) = -1 EINVAL (Invalid argument)
readlink("/run/media/manjaro", 0x7fff3dc98540, 1023) = -1 EINVAL (Invalid argument)
readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af", 0x7fff3dc98540, 1023) = -1 EINVAL (Invalid argument)
readlink("/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@", 0x7fff3dc98540, 1023) = -1 EINVAL (Invalid argument)
faccessat2(AT_FDCWD, "/run/media/manjaro/cfb49c22-87f2-47d9-a25b-310d8d8578af/@/", F_OK, AT_EACCESS) = 0
```

```
chroot("/mnt")                          = 0

chdir("/")                              = 0

execve("/usr/bin/bash", ["/usr/bin/bash", "-i"], 0x7fff3dc98b80 /* 23 vars */) = 0

brk(NULL)                               = 0x55d079513000

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=336155, ...}) = 0

mmap(NULL, 336155, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7efcaa6d4000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/libreadline.so.8", O_RDONLY|O_CLOEXEC) = 3

read(3,
"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\230\225\5\0\0\0\0\0\0\0\0
\0@\08\0\v\0

@\0\32\0\31\0\1\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\300V\1\0"..., 832) = 832

fstat(3, {st_mode=S_IFREG|0755, st_size=367640, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7efcaa6d2000

mmap(NULL, 371816, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7efcaa677000

mmap(0x7efcaa68d000, 204800, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x16000) = 0x7efcaa68d000

mmap(0x7efcaa6bf000, 36864, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x48000) = 0x7efcaa6bf000

mmap(0x7efcaa6c8000, 36864, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x51000) = 0x7efcaa6c8000

mmap(0x7efcaa6d1000, 3176, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7efcaa6d1000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT
(No such file or directory)

openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
```

```
read(3,
"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0y\2\0\0\0\0\0@\0\0\0\0\0\0\0x\234\36\0\0\0\0\0\0\0\0
@\08\0\17\0@\

0?\0>\0\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0"..., 832) = 832

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"..., 840, 64)
= 840

fstat(3, {st_mode=S_IFREG|0755, st_size=2010168, ...}) = 0

pread64(3,
"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0H\3\0\0\0\0\0\0H\3\0\0\0\0\0\0\10\0\0\0\0
\0\0\0\3\0

\0\0\4\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0 \257\33\0\0\0\0\0\36\0\0\0\0\0\0\0\36\0\0\0"..., 840, 64)
= 840

mmap(NULL, 2034544, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7efcaa486000

mmap(0x7efcaa4aa000, 1511424, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x24000) = 0x7efcaa4aa000

mmap(0x7efcaa61b000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x195000) = 0x7efcaa61b000

mmap(0x7efcaa669000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e2000) = 0x7efcaa669000

mmap(0x7efcaa66f000, 31600, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7efcaa66f000

close(3)                                = 0

openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libncursesw.so.6", O_RDONLY|O_CLOEXEC) = -1
ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/libncursesw.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3,
"\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\20\366\6\0\0\0\0\0\0\0\0\
0@\08\0\v\0@

\0\33\0\32\0\1\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0
```

```
mmap(0x7efcaa422000, 294912, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xd000) = 0x7efcaa422000

mmap(0x7efcaa46a000, 90112, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x55000) = 0x7efcaa46a000

mmap(0x7efcaa480000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6a000) = 0x7efcaa480000

close(3)                                = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7efcaa413000

arch_prctl(ARCH_SET_FS, 0x7efcaa413b80) = 0

set_tid_address(0x7efcaa413e50)         = 160869

set_robust_list(0x7efcaa413e60, 24)     = 0

rseq(0x7efcaa413ae0, 0x20, 0, 0x53053053) = 0

mprotect(0x7efcaa669000, 16384, PROT_READ) = 0

mprotect(0x7efcaa480000, 20480, PROT_READ) = 0

mprotect(0x7efcaa6c8000, 12288, PROT_READ) = 0

mprotect(0x55d048927000, 12288, PROT_READ) = 0

mprotect(0x7efcaa762000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

getrandom("\x46\x37\x01\x58\x71\x4a\xf6\xa9", 8, GRND_NONBLOCK) = 8

munmap(0x7efcaa6d4000, 336155)          = 0

openat(AT_FDCWD, "/dev/tty", O_RDWR|O_NONBLOCK) = 3

close(3)                                = 0

brk(NULL)                               = 0x55d079513000

brk(0x55d079534000)                     = 0x55d079534000

openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No
such file or directory)

openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=2998, ...}) = 0
```

read(3, "# Locale name alias data base.\n# Copyright (C) 1996-2026 Free Software Foundation, Inc.\n#\n# This pro"..., 4096)

= 2998

read(3, "", 4096)  = 0

close(3)  = 0

openat(AT_FDCWD, "/usr/lib/locale/es_ES.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory

)

openat(AT_FDCWD, "/usr/lib/locale/es_ES.utf8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es_ES/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es.utf8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

openat(AT_FDCWD, "/usr/lib/locale/es/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)

getresuid([0], [0], [0])  = 0

getresgid([0], [0], [0])  = 0

rt_sigprocmask(SIG_BLOCK, NULL, [], 8)  = 0

--- SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_ACCERR, si_addr=0x7efcaa765ef0} ---

+++ killed by SIGSEGV (core dumped) +++

Segmentation fault (core dumped)

**Gemini Replied:**

## Gemini ha dicho

El `strace` sigue siendo lapidario: el `Segmentation fault` ocurre justo cuando Bash intenta acceder a la memoria para manejar las **locales** (idioma) y los IDs de usuario. Esto sucede porque hay una inconsistencia total entre el `ld-linux` que copiaste y las librerías `libc.so.6` y `libreadline.so.8` que están en tu disco.

Sobre el problema de las pestañas en el Live USB: si no te deja abrir nuevas, es probable que el sistema de archivos temporal ( `run` o `tmp` ) esté saturado de basura de los crashes anteriores o que el gestor de terminales haya quedado colgado. No pierdas tiempo ahí.

---

## La Estrategia: "Reparación desde Afuera"

Olvidate del `chroot` por un momento. Si el entorno está tan roto que los binarios mueren al nacer, hay que repararlo usando el sistema que **SÍ** funciona (el Manjaro del USB).

### 1. Reinstalar la base de Arch en el disco duro

Ejecutá este comando. Va a usar el `pacman` del USB pero va a escribir los archivos en tu disco real, pisando cualquier librería corrupta (incluyendo los restos de Debian):

Bash

```
pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg --overwrite "*" -Sy glibc bash coreutils
```

- `--root /mnt` : Le dice a pacman que trabaje sobre tu disco duro.
- `--overwrite "*"` : Obligatorio para que pise los archivos que pusimos a mano.

### 2. Restaurar los enlaces vitales

Una vez que el comando anterior termine, tenemos que asegurarnos de que el cargador dinámico oficial de Arch sea el que manda:

Bash

```
# Borramos cualquier link previo
rm -f /mnt/lib64/ld-linux-x86-64.so.2

# Creamos el link oficial de Arch (relativo a la raíz del chroot)
ln -sf ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

---

## Por qué fallan las pestañas de ZSH

Si estás intentando abrir pestañas en la terminal del sistema que estás reparando, es lógico que fallen. Pero si es en el **Manjaro Live**, intentá esto para liberar recursos:

Bash

```
# Limpiar procesos de chroot zombis
killall -9 strace chroot bash 2>/dev/null

# Verificá si tenés espacio en el sistema de archivos del USB
df -h
```

---

## El Paso que lo define todo

Si `pacman --root` termina con éxito, probá el `chroot` de nuevo: `chroot /mnt /usr/bin/bash`

Si esto vuelve a dar Segfault, significa que `libreadline` o `ncurses` (que Bash usa para el prompt) también están rotas. En ese caso, la orden de reinstalación debería ser más amplia: `pacman --root /mnt -S glibc bash coreutils readline ncurses`

¿El comando `pacman --root` te tiró algún error de "Database lock" o de "GPG Keys"? Si te dice que la base de datos está bloqueada, borrá `/mnt/var/lib/pacman/db.lck`.

> **You asked:**

Has dicho

/mnt sudo su 1

sudo: unable to allocate pty: No such device

> **Gemini Replied:**

## Gemini ha dicho

El error `sudo: unable to allocate pty: No such device` es la señal definitiva de que el entorno del Live USB se está quedando sin recursos o, más probablemente, que el sistema de archivos **devpts** (que maneja las terminales virtuales) se corrompió o perdió sus nodos. Por eso no podés abrir más pestañas y por eso `sudo` falla.

Estás trabajando sobre los restos de una batalla campal de librerías. No te preocupes, vamos a resetear el entorno de trabajo sin reiniciar para no perder el progreso del montaje.

## 1. Limpiar el entorno de terminales

Ejecutá esto para intentar liberar los dispositivos de terminal:

Bash

```
# Forzar el cierre de cualquier proceso de chroot o strace colgado
killall -9 chroot strace bash zsh
```

## 2. El "Pacman" Salvador (Reparación desde afuera)

Como tu `chroot` está en coma por el Segfault y tu `sudo` está fallando por el tema de las PTY, intentá ejecutar el comando de reparación directamente (asumiendo que ya sos root en esa terminal, por el prompt `#` que pusiste antes):

Bash

```
pacman --root /mnt --cachedir /mnt/var/cache/pacman/pkg --overwrite "*" -Sy glibc bash coreutils
```

**¿Por qué este comando?**

- Al no usar `chroot`, usamos el binario de `pacman` del Manjaro Live (que funciona perfecto).
- Al apuntar a `--root /mnt`, arreglamos el disco duro "desde afuera".
- El Segfault que viste antes era porque `bash` intentaba cargar `libreadline` y `libc`, y al encontrar versiones mezcladas o corruptas, explotaba. Este comando va a poner las versiones oficiales de Arch.

## 3. Si `sudo` te sigue bloqueando

Si necesitás ser root y el `sudo` del Live USB no te deja por el error de `pty`, probá cambiar a una consola virtual pura (TTY):

1. Presioná `Ctrl + Alt + F2`.
2. Logueate como `manjaro` (password: `manjaro`).
3. Ahí el entorno suele estar más limpio y te va a dejar usar `sudo su` o ejecutar el `pacman --root`.

---

## Verificación después de la cirugía

Una vez que `pacman` termine de reinstalar esos 5 paquetes básicos, el `chroot` debería volver a la vida. Antes de entrar, asegurate de que el link del cargador sea el correcto:

Bash

```
ln -sf ../usr/lib/ld-linux-x86-64.so.2 /mnt/lib64/ld-linux-x86-64.so.2
```

**¿Lograste correr el `pacman --root` o te tiró algún error de "target not found" o "database lock"?** Si te da error de base de datos, borrá el candado: `rm /mnt/var/lib/pacman/db.lck`.